



COM1682

FINAL YEAR PROJECTS

Prepared By :

Nguyen Duc Hieu

001343306

Instructors:

Tran Thi Thoa

Table of Contents

1. Abstract	7
2. Introduction.....	7
3. Literature Review.....	9
3.1 Review of the Domain	9
3.2 Technologies for flexible cost splitting and shared subscriptions	12
3.2.1 Front-end technology comparison: Selection rationale and criteria	12
3.2.2 Backend technology comparison: Selection rationale and key challenge	16
3.2.3 Database architecture comparison	19
3.3 Review of methodologies	21
3.4 Choosing solutions.....	25
4 Requirement Analysis	28
4.1 Splitwise Application.....	29
4.2 Tricount Application.....	31
4.3 Market gaps identified from similar applications	33
4.4 Conclusion	34
4.5 Stakeholder Requirements	35
4.6 Business Requirement.....	36
4.7 Solution Requirement	40
4.8 Non-Functional Requirement.....	42
4.9 Legal, Social, Ethical and Professional Issues.....	44
4.9.1 Personal Data and Privacy	44
4.9.2 Payment, Refund and Withdrawal Responsibility	44
4.9.3 AI-Assisted Invoice Extraction.....	44
4.9.4 Authentication and JWT Security.....	45
4.9.5 Role-Based Access Control	45
4.9.6 Financial Audit Log and Accountability.....	45
4.9.7 Professional Responsibility.....	46
5 Software design.....	46
5.1 Architecture.....	46
5.1.1 Overall Architecture.....	46
5.1.2 Architectural rationale and Key decisions	47
5.1.3 API, Realtime and Notification communication design	48

5.1.4 Security and Authorization Architecture	49
5.1.5 External Service Integration Architecture	51
5.1.6 Background Processing and Operational Architecture	51
5.1.7 Architecture Limitations	52
5.2 GUI	53
5.2.1 GUI Design Purpose	53
5.2.2 Overall GUI Structure.....	53
5.2.3 Main GUI workflows and Prototype screen description.....	54
5.2.4 Current GUI Limitations.....	64
5.2.5 Future GUI Improvements.....	65
5.3 DB	65
5.3.1 Database model overview	65
5.3.2 Core collections	66
5.3.3 Indexing, TTL, and Locking Support	67
5.3.4 Key highlight: Financial Data lineage	67
5.4 UML.....	69
5.4.1 Use Case Diagram.....	69
5.4.2 Class Diagram.....	72
5.4.3 Sequence Diagram	76
6 Software implementation	81
6.1 Development environment.....	81
6.1.1 Development environment objective	81
6.1.2 Local development topology.....	81
6.1.3 Configuration and dependency management.....	82
6.1.4 Financial data verification.....	82
6.1.5 External service testing	83
6.1.6 Development environment limitations.....	83
6.2 Core features development	84
6.2.1 User registration with Email OTP.....	84
6.2.2 Login with Optional 2FA.....	85
6.2.3 Enable two-factor authentication	87
6.2.4 Group creation and Invitation	89
6.2.5 Invoice creation and Debt generation	91

6.2.6 Payment request creation and Debt settlement plan	95
6.2.7 Transfer payment with OTP and Debt allocation	98
6.2.8 Wallet Top-Up completion	101
6.2.9 Withdrawal with Email OTP	103
6.2.10 Recurring bill auto-generation	107
6.2.11 Personal receipt and Budget tracking	109
6.2.12 Realtime, Push, and Email notification.....	111
6.2.13 AI OCR Invoice Extraction	113
6.3 Important technical problems & solutions	116
6.3.1 Problem: Concurrency during billing execution.....	116
6.3.2 Billing cycle handling across time zones.....	127
6.3.3 Currency rounding when splitting shared expenses.....	128
6.3.4 Problem: Foreign currency invoice entry	129
6.3.5 Exchange rate lock at debt creation time	130
6.4 Test.....	132
6.4.1 Test Strategy	132
6.4.2 Functional and API Test Results.....	132
6.4.3 Financial Workflow and Traceability Testing	135
6.4.4 External Service and Environment-limited Testing.....	136
6.4.5 Usability Feedback from trial users	136
6.4.6 Benchmark testing	137
6.4.7 Performance testing	139
6.5 Results.....	141
6.5.1 Implementation results.....	141
6.5.2 Functional testing results	142
6.5.3 Technical integration results	142
6.5.4 Results compared with the initial objectives	143
6.5.5 Observed limitations from the implementation results.....	145
6.5.6 Summary of results	145
7 Evaluation and conclusion	145
7.1 Evaluation of results	145
Pros of application	146
Cons of application	146

7.2 Conclusion	147
Lessons learnt.....	147
Problems / difficulties	147
Future improvements	147
Conclusion	149
8 Appendix.....	150
Reference	161

Table of Figures

Figure 1 - Global Fintech Market report (Fortune Business Insight, 2025)	10
Figure 2 - Expense Tracker Apps Market Overview 2025 - 2034 (USD Billion) (Market Research Future, 2025)	11
Figure 3 - Scrum Process (pm-partner, 2024).....	22
Figure 4 - KanBan Desk (Entrust Consulting Group, 2021)	23
Figure 5 - Waterfall model (vecteezy.com)	24
Figure 6 - Spiral Model. Software Development and Validation (www.presentationeze.com)	25
Figure 7 - Splitwise Group details UI (https://blog.splitwise.com/).....	30
Figure 8 - Tricount Expense management flow (https://josiegirlblog.com/)	32
Figure 9 - Overall system architecture of SplitPal.....	46
Figure 10 - Authentication and Account security flow.....	55
Figure 11 - Group Management flow	57
Figure 12 - Invoice creation and Expense splitting flow	59
Figure 13 - End to end traceable chain	68
Figure 14 - Authentication and Account Module	69
Figure 15 - Group Management Module	70
Figure 16 - Share Expense and Invoice Module.....	71
Figure 17 - Authentication, Account & Wallet Class Diagram	72
Figure 18 - Group Management Class Diagram	73
Figure 19 - Shared Expense & Debt Settlement Class Diagram	74
Figure 20 - Recurring Expense Class Diagram.....	75
Figure 21 - Personal Expense & Budget Class Diagram	75
Figure 22 - Notification & Communication Class Diagram.....	76
Figure 23 - Login sequences diagram.....	77
Figure 24 - 2FA sequences diagram	78
Figure 25 - Create Invoices sequences diagram	79
Figure 26 - Create Payment request sequences diagram	80

1. Abstract

The rapid development of financial technology has accelerated the emergence of solutions for personal finance and group expense management. However, as shared living models, group travel, and the collective use of digital services become increasingly common, many existing approaches to expense management remain limited because they still depend on manual note-taking, spreadsheets, or applications that only support basic debt splitting. These approaches often lack transparency, make transaction histories difficult to trace, and provide limited support for flexible cost allocation, recurring shared payments, and subscription-based expense scenarios.

To address this problem, the SplitPal project is proposed as a modern group expense management application designed to support invoice recording, multiple cost-splitting methods, debt tracking among members, payment management, internal wallets, and shared subscription plans. The key distinction of the system is that it does not only calculate “who owes whom and how much.” Instead, SplitPal records and manages the full lifecycle of a shared financial obligation, from invoice creation and item-level allocation to original debt generation, payment requests, transfers, debt allocation, and transaction history. This financial data chain allows each balance change and debt settlement to be traced more clearly, which is essential for maintaining transparency and accountability in group financial activities.

This report presents the development process of SplitPal, including domain review, analysis of existing solutions, technology and methodology selection, requirement analysis, system design, implementation, testing, and evaluation. In terms of implementation, the project delivers core functions such as group management, flexible invoice splitting, transaction tracking, shared subscription management, real-time notifications, and AI-assisted invoice data extraction. The AI and OCR features are designed as assistive tools rather than autonomous decision-making components: extracted invoice data is supported by retry and fallback mechanisms and still requires user review before being applied to financial records. The evaluation results suggest that SplitPal can move beyond a simple bill-splitting application and serve as a foundation for a more transparent, flexible, and practical group financial management system for modern shared-living contexts.

2. Introduction

In the context of rapid digital transformation, shared living and shared consumption models such as co-living, group travel, long-term teamwork, and the collective use of digital services are becoming increasingly common, especially among students and young adults. When multiple people participate in shared expenses, the problem is no longer limited to calculating the total amount to be divided. It also involves identifying who participated in which expense, how much each person should pay, where each debt came from, whether it has been settled, and whether the transaction history can be clearly traced.

In practice, many groups still manage shared expenses through manual methods such as personal notes, Excel spreadsheets, group messages, or manual bank transfer reconciliation. These methods may be sufficient for simple expenses, but they become unreliable when financial scenarios become more complex. SplitPal focuses on addressing four main problems:

- Unequal participation: Not every member joins the same invoice or expense item. If the total cost is simply divided equally, the result may be unfair and difficult to explain.
- Recurring expenses: Repeated payments such as rent, service fees, or shared subscriptions can be missed or entered incorrectly if users have to create them manually every month.
- Multi-currency expenses with locked exchange rates: When expenses are recorded in different currencies, the exchange rate needs to be fixed at the time of creation to keep debt amounts stable and easier to reconcile.
- Lack of a clear financial audit trail: Users need to trace each debt from invoice creation to payment and transaction history. Without this, disputes and data inconsistencies are harder to resolve.

Based on these problems, SplitPal is proposed as a modern, flexible, and practical group expense management application. The system allows users to create expense groups, invite members, record invoices, split costs using different methods, track debts among members, manage internal wallets, create payment requests, process transfers, manage shared subscriptions, and store transaction histories in a structured way. In addition to these core financial functions, SplitPal also integrates real-time notifications and AI/OCR-assisted invoice data extraction to reduce manual data entry. However, the AI/OCR features are designed as assistive tools rather than autonomous financial decision-making components, because extracted data still needs to be reviewed by users before being applied to invoice records.

The key difference between SplitPal and basic bill-splitting applications is that the system does not only calculate “who owes whom and how much.” Instead, SplitPal manages the full lifecycle of a shared financial obligation. A shared expense can start from an invoice, be divided into invoice items, generate original debts, be grouped into payment requests, be processed through transfers, be allocated to specific debts through transfer debt allocations, and finally be recorded in the transaction history. This financial data chain allows the system to explain why a debt exists, which invoice it came from, which transaction settled it, and how the wallet balance changed. Therefore, SplitPal acts not only as a calculation tool, but also as a traceable group financial workflow system.

The main objective of this project is to develop a practical group expense management system capable of handling financial operations that are more complex than traditional bill-splitting models. More specifically, the project aims to implement secure user account management through authentication, OTP verification, two-factor authentication, and profile management; support group-based expense management through group creation, member management, and invitations; enable flexible invoice recording and cost splitting through invoice items and split methods; provide transparent debt tracking and settlement through original debts, payment requests, transfers, and transfer debt allocations; support internal financial operations through wallet management, top-up, withdrawal, and transaction history; manage recurring and shared digital service expenses through subscription management; and improve user experience through real-time notifications and AI/OCR-assisted invoice extraction.

Through these objectives, the project does not only focus on building an application with basic functions, but also demonstrates the process of requirement analysis, system design, full-stack implementation, third-party service integration, and practical business problem-

solving in the digital finance domain. This report presents the development process of SplitPal, including the research context, analysis of existing solutions, selection of technologies and development methodologies, requirement analysis, system design, implementation of core features, testing, evaluation of results, and future development directions. Ultimately, the project seeks to demonstrate that a properly designed group expense management system can reduce financial conflict, improve transparency, and enhance the user experience in modern shared-spending contexts.

3. Literature Review

3.1 Review of the Domain

Financial technology (Fintech) refers to the application of digital technologies to improve the delivery, accessibility, and efficiency of financial services. Over the past decade, fintech has evolved from back-office optimization into a broad ecosystem of customer-facing solutions, including digital payments, personal finance applications, lending platforms, and financial infrastructure services. This broader growth context is relevant because it has created both the technical foundation and the market conditions for more specialized financial applications to emerge. Within this expanding ecosystem, collaborative expense management can be viewed as a practical sub-domain that responds to the everyday financial coordination needs of small groups.

According to Fortune Business Insights (2025), the global fintech market was valued at USD 340.10 billion in 2024 and was projected to reach USD 1.13 trillion by 2032. This macro-level growth is reinforced by the BCG and QED Global Fintech Report 2025, which notes that fintech revenues grew 21% year-on-year in 2024, significantly outpacing traditional financial services, while 69% of publicly listed fintechs were profitable (BCG and QED Investors, 2025). Taken together, these figures suggest that fintech is no longer a peripheral innovation space, but a maturing domain that increasingly supports targeted solutions for specific financial coordination problems.

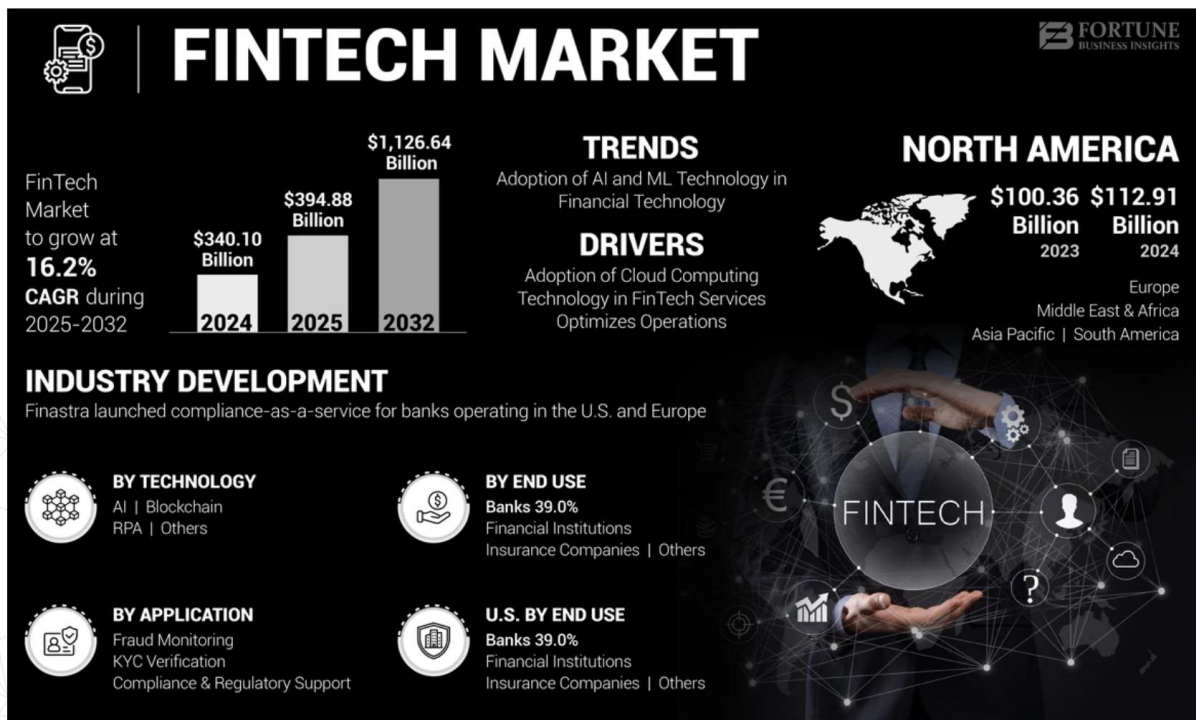


Figure 1 - Global Fintech Market report (Fortune Business Insight, 2025)

Within this broader ecosystem, group expense management has emerged as a practical sub-domain concerned with how individuals coordinate, record, allocate, and settle shared costs in collective settings. Its relevance has grown alongside the spread of shared living arrangements such as dormitories, co-living spaces, and group rentals, as well as the expansion of recurring shared services. Market reports on expense-tracker applications also point to continued growth in this niche, indicating that the need for tools supporting everyday financial coordination is becoming increasingly significant within the wider personal finance landscape (Market Research Future, 2025).

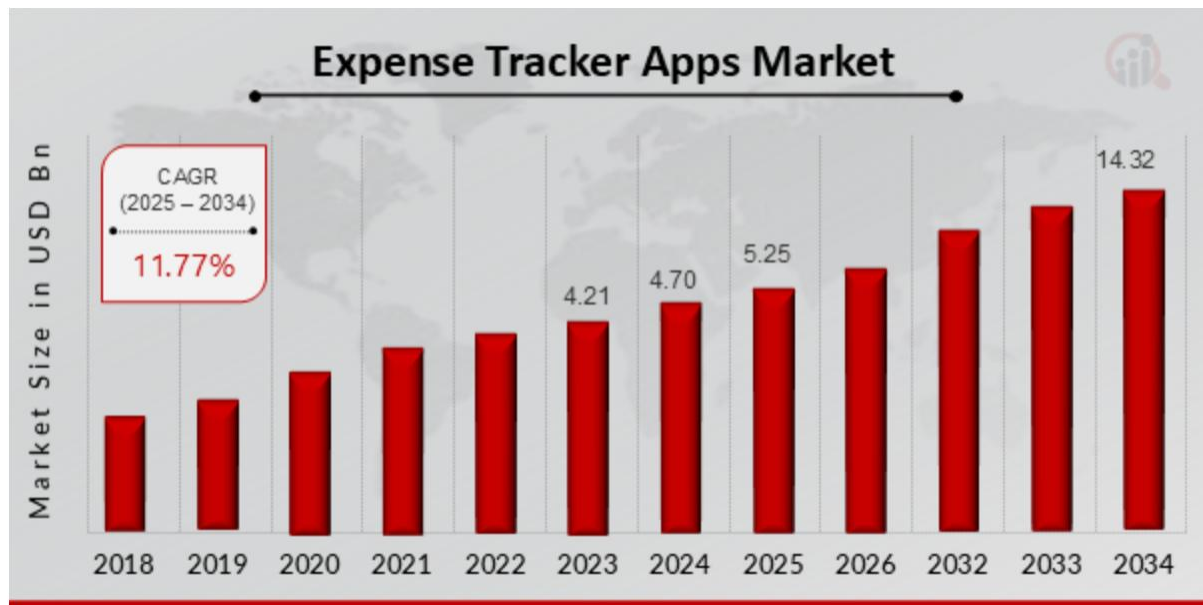


Figure 2 - Expense Tracker Apps Market Overview 2025 - 2034 (USD Billion) (Market Research Future, 2025)

A notable characteristic of this domain is that shared spending is no longer limited to conventional household costs such as rent, utilities, and groceries. Increasingly, group finance also includes recurring digital services such as streaming platforms, cloud storage, design software, and online learning subscriptions. This reflects a broader shift toward shared consumption, in which access to services is coordinated across multiple users rather than individually owned and consumed. Khalek and Chakraborty (2023), in their systematic literature review, argue that shared consumption is shaped by a combination of economic, functional, and social determinants, highlighting its growing importance as a distinct mode of consumption. In the context of group finance, this means that expense management is no longer simply a matter of dividing one-off payments, but increasingly involves selective participation, recurring obligations, and differentiated usage rights over time (Khalek and Chakraborty, 2023).

Historically, many groups have managed shared expenses through informal mechanisms such as handwritten notes, spreadsheets, chat messages, or manual bank transfer reconciliation. While these methods may appear convenient in simple scenarios, they become increasingly ineffective as the number of participants, expense categories, and billing cycles grows. Their limitations are not only operational but also relational: they provide weak transparency, limited traceability, and poor support for unequal or changing participation. Köbis et al. (2020), in their systematic literature review of trust-based commercial sharing, show that ambiguity around roles, rules, and consequences is a major source of negative outcomes in shared exchange settings, and they emphasize transparency as a central principle for more responsible forms of sharing. Applied to group expense management, this suggests that manual coordination methods are structurally weak in situations involving delayed reimbursement, recurring payments, or unclear responsibility, often leading to disputes and additional administrative burden (Köbis et al., 2020).

The relevance of this problem also becomes clearer when viewed in emerging-market contexts, where patterns of digital adoption and collaborative consumption may differ from those commonly observed in North America or Europe. Llanos et al. (2023) argue that consumer adoption of collaborative consumption services in emerging markets is shaped not only by convenience and enjoyment, but also by contextual factors linked to economic and political uncertainty. This implies that the design and expected value of shared financial tools cannot be assumed to be universal across contexts. Instead, group expense solutions need to be sufficiently flexible and transparent to operate under varying trust conditions, financial habits, and digital behaviors (Llanos et al., 2023).

Taken together, these observations suggest that group expense management should be understood not merely as a subset of personal finance, but as a coordination-oriented financial domain involving multiple participants, changing obligations, recurring shared consumption, and the need for transparency over time. Although specialized tools already exist, the domain continues to evolve beyond simple post-expense debt tracking. Modern shared financial behavior increasingly requires flexible allocation logic, better support for recurring and selective participation, and more effective coordination mechanisms across users. These domain characteristics directly motivate the subsequent review of technologies and methodologies for developing a more suitable solution for this problem space.

Therefore, group expense management should be understood not merely as a subset of personal finance, but as a coordination-oriented financial domain involving multiple participants, changing obligations, and recurring shared consumption.

3.2 Technologies for flexible cost splitting and shared subscriptions

3.2.1 Front-end technology comparison: Selection rationale and criteria

For this project, the frontend layer must support complex cost-allocation interfaces, responsive cross-platform rendering, offline data entry, real-time synchronization, and maintainable development within a small-team setting.

Designing and developing intricate application interfaces

React Native leverages the power of the JavaScript/TypeScript ecosystem alongside well-established state management libraries such as Redux, MobX, and the Context API, enabling the development of complex, modular user interface architectures. However, research by Prajapati and Bhattacharjee (2025) highlights that the bridge architecture between the JavaScript thread and the native thread in React Native introduces a significant performance bottleneck, particularly when handling computationally intensive tasks. This often results in frame drops or UI desynchronization, compromising user experience. Consequently, designing complex interfaces - such as ratio-based views or interactive dashboards - requires strict discipline in render optimization and state synchronization. From a maintenance

perspective, while the vast JavaScript ecosystem offers high flexibility, it also poses a risk of instability if rigorous source code quality control practices are not consistently enforced.

Flutter demonstrates a clear advantage in developing highly customizable user interfaces, thanks to its comprehensive widget system and reactive rendering engine. According to (Jagiello, 2019), Flutter outperforms React Native in terms of UI performance, particularly in frame rate benchmarks and transition smoothness. This superiority stems from Flutter's elimination of the intermediary bridge, allowing it to render directly through the Skia engine. However, Flutter's reliance on the Dart programming language - which remains relatively uncommon - introduces a steep learning curve, slowing initial development for teams transitioning from the JavaScript ecosystem. Moreover, the limited diversity of third-party libraries can make the development of complex user interfaces more challenging, sometimes requiring deep-level native code interventions, thereby increasing maintenance costs.

Kotlin Multiplatform (KMM) enables the sharing of business logic between Android and iOS, while the user interface must still be developed separately for each platform. This approach optimizes performance and preserves the native user experience, but also increases development workload due to the need to maintain two parallel UI layers. A study by Aripirala, Airan, and Peddinti (2024) indicates that although KMM provides efficient data logic reuse, the UI development and testing costs are significantly higher compared to Flutter and React Native, primarily due to the lack of unified cross-platform design tools. Consequently, KMM is particularly well-suited for projects that demand high native performance and strict control over interface behavior, but it is less ideal for small development teams or projects requiring frequently changing or highly complex user interfaces.

Offline-first and real-time data synchronisation UX

React Native benefits from a mature ecosystem of libraries supporting the offline-first paradigm, including Realm, WatermelonDB, and Redux Offline. These libraries enable applications to maintain local data persistence when network connectivity is unavailable, and to synchronise changes seamlessly once connectivity is restored. This strength contributes to improved user experience and reduced risk of data loss. Nevertheless, React Native places a considerable burden on developers to manually implement advanced synchronisation mechanisms and conflict resolution strategies, which can be complex and error-prone (Riaz, 2021).

Flutter provides strong native support for realtime and offline-first experiences through an extensive set of official and third-party plugins. Hive and Drift (formerly Moor) offer lightweight local storage solutions, while Firebase and Supabase ensure efficient synchronisation once the device reconnects. As shown in the comparative analysis by Riaz (2021), Flutter's integration with Firebase simplifies real-time data flows and substantially reduces sync latency compared to traditional approaches. However, the framework lacks a unified mechanism for conflict resolution, leaving developers to implement custom logic to

preserve data integrity. From a UX standpoint, this limitation may result in inconsistent update behaviour across devices in multi-user environments.

Kotlin Multiplatform allows shared data and caching layers across Android and iOS, enabling consistent business logic and data flow between platforms. This approach simplifies domain-level synchronisation, improving reliability and reducing code duplication. However, as reported by JetBrains (2025), the KMM ecosystem lacks mature, ready-to-use libraries for offline-first architecture, forcing developers to implement custom synchronisation and conflict-handling mechanisms. This not only increases development complexity but also raises UX concerns: inconsistent synchronisation behaviour may lead to delayed updates or user confusion in distributed environments.

Ecosystem & Long-term Maintainability

React Native leverages the extensive JavaScript and Node.js ecosystem, offering developers rapid prototyping and access to a rich library environment. However, this dependency on external packages introduces long-term maintainability risks. An empirical study of the npm ecosystem found that 55.88% of continuous integration build time was wasted on updates to unused dependencies, highlighting a substantial amount of dependency-induced technical debt (Weeraddana, Alfadel & McIntosh, 2024). This finding underscores that while React Native accelerates development through ecosystem maturity, its sustainability depends heavily on disciplined dependency management and rigorous code governance to avoid cumulative maintenance overhead.

Flutter has emerged as one of the most balanced cross-platform frameworks, combining high performance with maintainable architecture. According to Riaz (2025), Flutter demonstrates the best trade-off between performance and maintainability among major cross-platform tools such as React Native and Kotlin Multiplatform. Its unified widget-based architecture allows a single codebase to serve multiple platforms, reducing maintenance complexity and ensuring consistent UI behavior across devices. Nonetheless, its reliance on the less widespread Dart language and strong dependence on Google's ecosystem may raise long-term sustainability concerns, particularly for teams seeking greater technological independence.

Kotlin Multiplatform (KMM) facilitates business logic sharing across Android and iOS, significantly reducing code duplication and enhancing consistency in data and domain layers. A recent study published on SSRN found that KMM improves maintainability by enabling cross-platform reuse of business logic while preserving native performance characteristics. This architectural model allows organizations to achieve code efficiency without sacrificing platform-specific optimization. Nevertheless, because the UI layer must be implemented separately for each platform, development and maintenance costs remain higher than those of unified frameworks such as Flutter. Consequently, KMM is best suited for larger teams or enterprise-level projects prioritizing native integration over rapid development speed (Ananya Punia et al., 2024).

Taken together, these comparisons indicate that Flutter is the most suitable frontend technology for this project. While React Native offers ecosystem maturity and Kotlin Multiplatform provides strong native integration, both introduce trade-offs that are less aligned with the project’s constraints. React Native demands stricter optimization to maintain performance consistency in complex interfaces, whereas Kotlin Multiplatform increases development overhead through separate platform-specific UI implementation. Flutter, in contrast, offers a more effective balance of customizable interface design, cross-platform rendering consistency, support for offline and real-time interaction patterns, and manageable long-term maintenance.

Evaluation Criteria	React Native	Flutter	Kotlin Multiplatform (KMM)
UI Architecture & Complexity Handling – Strengths	Leverages the extensive JavaScript/TypeScript ecosystem with mature state management tools (Redux, MobX, Context API). Supports modular and reusable component-based UI development.	Offers a robust widget system and reactive rendering engine for highly customized, smooth-performing interfaces. Direct rendering via Skia engine ensures superior visual performance.	Enables shared business logic across platforms while maintaining fully native UI, ensuring optimal performance and user experience fidelity.
UI Architecture & Complexity Handling – Limitations	Bridge between JavaScript and native threads introduces performance bottlenecks in computation-heavy UIs. Requires strict optimization and disciplined state management. Dependent on third-party JS packages, increasing maintenance risks.	Dart language adoption poses a learning curve; limited ecosystem may require native code for advanced UI, increasing complexity and maintenance.	Separate UI implementation for Android and iOS doubles workload and testing effort. Lacks unified cross-platform design tooling.
Offline-first & Real-time Data Synchronization – Strengths	Mature ecosystem supporting offline-first patterns (Realm, WatermelonDB, Redux Offline). Reliable local data persistence and seamless synchronization after reconnection.	Rich plugin ecosystem (Hive, Drift, Firebase, Supabase) simplifies real-time sync and reduces latency. Native Firebase integration enhances user experience.	Shared caching and business logic layers promote consistent data handling and reduced duplication across platforms. Simplifies synchronization logic.
Offline-first & Real-time Data Synchronization – Limitations	Requires manual implementation of complex sync and conflict resolution mechanisms, increasing development risk.	Lacks a unified conflict resolution framework, potentially causing inconsistent updates in multi-user environments.	Immature ecosystem for offline-first architecture; custom implementations increase complexity and may result in delayed or inconsistent updates.

Ecosystem Maturity & Long-term Maintainability – Strengths	Broad JavaScript and Node.js ecosystem enables rapid prototyping and abundant third-party support. Fast iteration cycle supported by strong community resources.	Provides balanced trade-off between performance and maintainability. Unified widget architecture ensures consistency across platforms and reduces long-term complexity.	Facilitates efficient business logic reuse while maintaining native performance. Suitable for enterprise-scale projects requiring strong platform integration.
Ecosystem Maturity & Long-term Maintainability – Limitations	Heavy reliance on external dependencies can lead to technical debt accumulation. Requires strong governance to ensure long-term stability.	Dependence on Google's ecosystem and limited Dart talent pool may pose long-term sustainability risks.	Higher development and maintenance cost due to separate UI layers. Slower delivery compared to unified cross-platform frameworks.

3.2.2 Backend technology comparison: Selection rationale and key challenge

Performance Scalability and Maintainability

Within the context of the performance–maintainability trade-off, Node.js exemplifies how scalability can sometimes come at the cost of structural simplicity. Its asynchronous, non-blocking I/O architecture enables exceptional scalability for I/O-bound workloads and real-time applications. Yet, Jha (2025) observes that this same model increases architectural fragility when applied to CPU-bound logic, often necessitating the introduction of worker threads which complicates the codebase. Furthermore, while the adoption of TypeScript addresses the maintainability risks of JavaScript’s dynamic typing by enforcing static analysis, it introduces a "transpilation tax." As noted by Boghian et al. (2019), migrating to TypeScript significantly reduces runtime errors and improves code readability, but it adds configuration overhead and build-step complexity. Node.js thus represents a paradigm where achieving both high scalability and long-term maintainability requires a rigorous combination of architectural discipline and strict type-system enforcement.

Django illustrates the opposite side of this trade-off. As Jha (2025) highlights, Django’s synchronous WSGI design and Python’s Global Interpreter Lock (GIL) restrict concurrency, limiting performance under high request volumes. However, these constraints contribute to the framework’s maintainability: Django’s monolithic architecture, built-in conventions, and clear separation of concerns foster consistency, readability, and long-term stability. In practice, Django sacrifices raw scalability to achieve predictable behaviour and maintainable code structures, aligning well with systems where reliability and long-term evolution are prioritised over throughput.

Go offers a unique lens on the same performance–maintainability dilemma. Designed for high concurrency through lightweight goroutines, Go achieves impressive scalability and

near-native performance. However, Costa et al. (2020) found that roughly 24% of Go projects use the unsafe package to bypass type safety, trading code reliability for micro-level performance gains. This behaviour demonstrates how the pursuit of maximal scalability and speed can directly undermine maintainability. Go thus stands as a cautionary case: while its concurrency model provides a clean path to scalable architectures, the temptation to over-optimize can erode code safety and increase long-term technical debt.

Security & Built-in protection

Node.js operates on a minimalist core, delegating security implementations to its vast ecosystem of third-party packages. While the adoption of TypeScript enhances code correctness by preventing type-related vulnerabilities at compile-time, it does not mitigate the risks inherent in the dependency chain. Zimmermann et al. (2019), in a comprehensive study of the npm ecosystem, identified that the heavy reliance on external dependencies creates a fragile transitive trust model, where a single compromised package can impact thousands of downstream applications. Consequently, securing a Node.js/TypeScript application requires developers to actively manage dependency audits and manually configure middleware (such as Helmet or CORS), shifting the burden of security configuration from the framework to the engineering team.

In contrast, Django is architected with a "batteries-included" philosophy that prioritises security by default. The framework automatically handles critical protections against common vulnerabilities such as Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), and SQL Injection without requiring explicit developer intervention. Django's rigid internal structure significantly reduces the application's attack surface compared to more flexible frameworks by enforcing established security patterns. However, this rigidity can become a limitation when implementing non-standard authentication flows, where overriding Django's defaults requires careful handling to avoid introducing vulnerabilities.

Go approaches security through the lens of memory safety and compile-time strictness rather than high-level web guards. Being a compiled, strongly-typed language, Go naturally eliminates entire classes of vulnerabilities related to memory management, such as buffer overflows. As noted by Rob Pike (2012) in his foundational paper on Go's design, the language was built to address safety issues in systems programming at Google. However, regarding web-specific threats, Go's standard library is minimalist; it lacks the automatic CSRF or XSS mitigation found in Django. Developers must often implement these protections manually or rely on specific middleware, demanding a higher level of security awareness during the coding phase.

Developer velocity and Type safety

Node.js initially offers high developer velocity due to JavaScript's dynamic nature and JSON ubiquity. However, to combat the runtime errors common in dynamic languages, modern development has shifted towards TypeScript. Bogner and Merkel (2022) conducted a systematic comparison of JavaScript and TypeScript projects and found that while TypeScript improves code quality and readability, it introduces a "transpilation tax" -

requiring additional setup time for types, interfaces, and build configurations. This creates a trade-off where the immediate velocity of writing code is slightly reduced in exchange for better long-term stability and fewer runtime crashes.

Django balances this trade-off by leveraging Python's readability and a vast library of pre-built components (DRY principle), which accelerates the development of complex features. Nevertheless, Python's dynamic typing remains a double-edged sword. As highlighted in the large-scale study by Ray et al. (2014), dynamically typed languages like Python are statistically more prone to defect injection compared to statically typed languages. While developers can prototype features extremely fast in Django, the lack of strict type enforcement can lead to obscure bugs in large-scale systems that are difficult to trace without extensive unit testing.

Go prioritises long-term maintainability over initial coding speed through strict static typing. The language's syntax is deliberately verbose, requiring explicit error handling (e.g., if err != nil) and lacking "magic" abstractions. Rob Pike (2012) explains that this design choice was intentional to make code readable and predictable across large teams. While this verbosity reduces the initial lines-of-code per hour compared to a dynamic Node.js or Django setup, it significantly lowers the technical debt and time spent on refactoring in later stages. Go's model suggests that true developer velocity is sustained by the ease of reading and fixing code, rather than just the speed of writing it.

Evaluation Criteria	Node.js (ExpressJS + TypeScript)	Django (Python)	Go (Golang)
Performance Scalability vs. Maintainability – Strengths	Asynchronous, non-blocking I/O architecture enables exceptional scalability for I/O-bound workloads and real-time applications.	Monolithic architecture, built-in conventions, and clear separation of concerns foster consistency, readability, and long-term stability.	Designed for high concurrency through lightweight goroutines, achieving impressive scalability and near-native performance.
Performance Scalability vs. Maintainability – Limitations	Architectural fragility when applied to CPU-bound logic. TypeScript integration introduces a "transpilation tax" and configuration overhead.	Synchronous WSGI design and Python's Global Interpreter Lock (GIL) restrict concurrency, limiting performance under high request volumes.	Pursuit of maximal scalability can lead to bypassing type safety (e.g., using unsafe package), undermining maintainability.
Security & Built-in Protection – Strengths	Flexible configuration allows for highly customisable security strategies via middleware.	"Batteries-included" philosophy handles critical protections (XSS, CSRF, SQL Injection) by default, significantly reducing the attack surface.	Compiled, strongly-typed nature eliminates entire classes of memory management vulnerabilities (e.g., buffer overflows).
Security & Built-in Protection – Limitations	Heavy reliance on external dependencies creates a fragile transitive trust model with supply chain risks. Requires manual configuration of security headers.	Rigidity can be a limitation when implementing non-standard authentication flows, requiring careful overriding of defaults.	Standard library lacks automatic web guards (CSRF/XSS), requiring manual implementation or middleware and higher developer security awareness.

Developer Velocity & Type Safety – Strengths	High initial velocity due to JavaScript’s dynamic nature and JSON ubiquity; TypeScript adoption improves code quality and readability.	Vast library of pre-built components (DRY principle) accelerates development of complex features and administration tools.	Strict static typing and explicit error handling significantly lower technical debt and refactoring time in later stages.
Developer Velocity & Type Safety – Limitations	TypeScript setup introduces additional complexity (types, interfaces, build config), creating a trade-off where immediate writing speed is reduced.	Dynamic typing makes the system statistically more prone to defect injection compared to statically typed languages.	Verbose syntax and lack of "magic" abstractions reduce initial lines-of-code per hour compared to dynamic frameworks.

3.2.3 Database architecture comparison

Schema flexibility and Data evolution

The ability of a database to adapt to changing application requirements - specifically regarding schema modifications - is a critical factor in the development lifecycle.

Relational Databases (RDBMS) enforce a "Schema-on-Write" policy, requiring a predefined structure before any data can be inserted. Curino et al. (2008), in their analysis of schema evolution, highlight that while this rigidity guarantees data quality and structural integrity, it introduces significant friction during rapid development. Modifying the schema (e.g., adding a new column for a feature update) often requires executing expensive **ALTER TABLE** commands, which can lock the database and cause downtime. For a project like SplitPal, this means the financial data model must be carefully designed upfront, as frequent changes can become operationally costly.

NoSQL Databases (specifically Document stores) champion a "Schema-on-Read" or schemaless approach. Sadalage and Fowler (2012), in their foundational work on NoSQL architectures, explain that this flexibility allows developers to store objects with differing attributes in the same collection without downtime for migration. This implies that new features can be deployed immediately by simply writing new fields to the database. However, this flexibility shifts the burden of schema management to the application layer; the code must be robust enough to handle missing fields or inconsistent data structures across old and new records, potentially leading to logic bugs if not managed strictly.

Column-based Databases are generally the least flexible regarding schema evolution due to their physical storage layout. Since data is compressed and stored by column to optimise read performance, adding or modifying columns often necessitates rewriting large segments of the storage files. Abadi et al. (2008) note that column-stores achieve their superior performance through aggressive compression algorithms (like Run-Length Encoding) that are tightly coupled with the data type and structure. Consequently, changing the schema breaks these compression blocks, making schema evolution a computationally heavy process that is best reserved for stable, mature analytical datasets rather than evolving operational data.

Data consistency and transactional integrity (ACID vs. BASE)

The selection of a database model fundamentally dictates the system’s ability to guarantee data accuracy under concurrent access.

Relational Databases (RDBMS) represent the gold standard for strict consistency through adherence to ACID properties (Atomicity, Consistency, Isolation, Durability). As Cattell (2011) highlights in his survey of data stores, RDBMSs like PostgreSQL enforce rigid schemas and referential integrity, ensuring that complex financial transactions are either fully committed or entirely rolled back. This model is indispensable for core banking features where data anomalies are unacceptable. However, this strictness imposes a "coordination overhead," making it difficult to scale write operations across distributed clusters without significant performance degradation.

NoSQL Databases emerged to address the scalability limitations of RDBMS by relaxing consistency requirements in favour of Availability and Partition tolerance (the CAP theorem). Brewer (2012) argues that for many modern web applications, the BASE model (Basically Available, Soft state, Eventual consistency) is superior, allowing systems to accept writes even during network partitions. While this offers high throughput for features like user sessions or activity logs, the lack of immediate consistency poses significant risks for financial ledgers, where a "read-after-write" delay could lead to incorrect balance displays. Column-based Databases occupy a unique position regarding consistency. While some modern columnar stores support ACID transactions, their architecture is primarily optimised for batch loading rather than individual row updates. Stonebraker et al. (2005), in their seminal work on C-Store, explain that column-oriented systems often use a "read-optimised" storage layout that makes single-record updates computationally expensive. Therefore, while they can maintain data integrity, they are ill-suited for the high-volatility transactional environments (OLTP) that RDBMSs handle effortlessly, being better reserved for stable, immutable datasets used for reporting.

Workload performance: OLTP vs. OLAP architectures

The performance of a database is heavily dependent on whether the workload involves processing individual records (OLTP) or analysing vast datasets (OLAP).

Relational Databases are architected as "row-stores," storing data sequentially by row. This design is highly efficient for On-Line Transaction Processing (OLTP), where the primary access pattern involves retrieving or updating all attributes of a specific entity (e.g., fetching a user's full profile). However, Abadi et al. (2008) demonstrate that this row-oriented layout becomes a bottleneck for analytical queries. To calculate a sum of a single column (e.g., "Total Transaction Value"), the database must read the entire row for every record, loading unnecessary data into memory and significantly degrading performance as the dataset grows. NoSQL Databases (specifically Document or Key-Value stores) excel in handling unstructured or semi-structured data with high read/write throughput for simple access patterns. They avoid complex joins by denormalising data, which accelerates retrieval speeds for specific application views. However, as noted in the study by Moniruzzaman and Hossain (2013), NoSQL systems often lack the sophisticated query optimisers found in RDBMS. Consequently, complex ad-hoc queries or aggregations that were not anticipated during the data modelling phase can be inefficient, requiring application-level processing rather than database-level optimisation.

Column-based Databases are specifically designed to solve the analytical limitations of row-stores. By storing data vertically (column by column) rather than horizontally, they allow queries to read only the relevant attributes, ignoring unrelated data. Abadi et al. (2008) empirically proved that for On-Line Analytical Processing (OLAP) workloads, column-stores outperform row-stores by an order of magnitude due to better compression rates and reduced I/O. This makes them the ideal choice for features requiring heavy data aggregation, such as generating monthly spending reports or analysing financial trends in the SplitPal application, albeit at the cost of slow single-row insertion speeds.

Evaluation Criteria	Relational Database (RDBMS)	NoSQL Database (Document)	Column-based Database
Schema Flexibility & Data Evolution – Strengths	rigid "Schema-on-Write" policy guarantees data quality and structural integrity.	"Schema-on-Read" flexibility allows storing objects with differing attributes, enabling immediate deployment of new features.	Optimised storage layout ensures high performance for stable, mature datasets.
Schema Flexibility & Data Evolution – Limitations	Schema modifications (e.g., ALTER TABLE) are expensive and can cause downtime, creating friction during rapid development.	Shifts schema management burden to the application layer; requires robust code to handle inconsistent data structures.	Least flexible; modifying columns often necessitates rewriting large segments of storage files due to compression dependancy.
Data Consistency & Transactional Integrity – Strengths	Gold standard for strict consistency (ACID); ensures financial transactions are fully committed or rolled back.	BASE model (Basically Available, Soft state, Eventual consistency) offers high availability even during network partitions.	Can maintain integrity suitable for analytical reporting; some modern systems support ACID transactions.
Data Consistency & Transactional Integrity – Limitations	Strictness imposes "coordination overhead," limiting the ability to scale write operations across distributed clusters.	Lack of immediate consistency poses risks for financial ledgers (e.g., "read-after-write" delays leading to incorrect balances).	Architecture is optimised for batch loading; single-record updates are computationally expensive.
Workload Performance: OLTP vs. OLAP – Strengths	Row-store design is highly efficient for OLTP (retrieving/updating specific entities).	High read/write throughput for simple access patterns; denormalisation accelerates retrieval speeds.	Column-store design outperforms row-stores by an order of magnitude for OLAP workloads due to better compression and reduced I/O.
Workload Performance: OLTP vs. OLAP – Limitations	Row-oriented layout is a bottleneck for analytical queries, as it loads unnecessary data into memory.	Lacks sophisticated query optimisers; complex ad-hoc aggregations can be inefficient.	Slow single-row insertion speeds make it ill-suited for high-volatility transactional environments.

3.3 Review of methodologies

Scrum is one of the most widely adopted Agile frameworks, built around short iterations (sprints) that promote rapid feedback and continuous improvement. Its key strengths include increased transparency in development, enhanced team communication, and accelerated time-to-market. According to the 17th State of Agile Report, as many as 87% of organizations

implementing Agile use Scrum or its variants, making it the most widely practiced methodology (State of Agile, 2023). An article in the Harvard Business Review further emphasizes that Scrum enables organizations to adapt more effectively to change and create sustainable value through short, transparent cycles (Rigby et al., 2016).

However, Scrum also presents certain limitations. It places high demands on team discipline, with daily stand-ups, sprint reviews, and retrospectives often creating pressure for smaller or less experienced teams. Moreover, Scrum becomes less effective when requirements change too frequently within the course of a sprint.

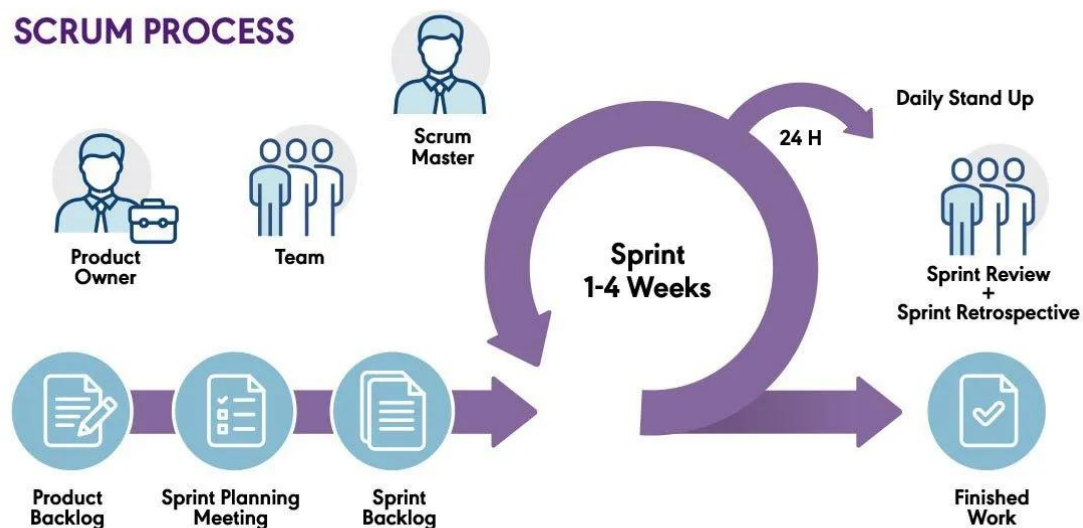


Figure 3 - Scrum Process (pm-partner, 2024)

Kanban focuses on visualizing workflow and improving flow efficiency by limiting work in progress (WIP), thereby enabling continuous delivery and greater transparency across the pipeline. In financial services, agile operating models (including Kanban-style flow and cross-functional squads) are associated with higher speed-to-market, stronger customer centricity, and better employee engagement; banks that adopt agile are 1.5× more likely to outperform financially (Nilsson et al., 2023)

Sector-level evidence also shows that financial services is among the most mature in enterprise agility: the share of truly agile companies is 48% in financial services - one of the highest across industries - indicating strong fit for flow-based ways of working like Kanban. However, Kanban lacks time-boxing by design, so without explicit WIP limits and regular cadence reviews, work can sprawl and timelines slip. It is also less suited to large programs that require fixed stage gates/milestones unless it is combined with lightweight planning and governance rituals. (Grebe et al. 2024)



Figure 4 - KanBan Desk (Entrust Consulting Group, 2021)

The Waterfall model is a sequential and linear software development methodology, first introduced by Winston W. Royce in 1970 as a structured approach progressing through distinct phases: requirements → design → implementation → testing → deployment → maintenance. Its primary strength lies in its well-defined structure, clear milestones, and comprehensive documentation, making it highly suitable for projects with stable and clearly defined requirements. As highlighted in comparative studies of software development life cycles (SDLCs), Waterfall is particularly effective when projects demand rigorous documentation, control, and predictability (Balaji and Murugaiyan, 2012).

Nevertheless, Waterfall carries significant limitations. Due to its sequential nature, it provides little flexibility to adapt to requirement changes, and the late testing phase increases the risk of identifying defects only at the end of the cycle, which in turn raises correction costs. This makes the methodology less suitable during ideation or MVP stages, where rapid feedback and adaptability are essential.

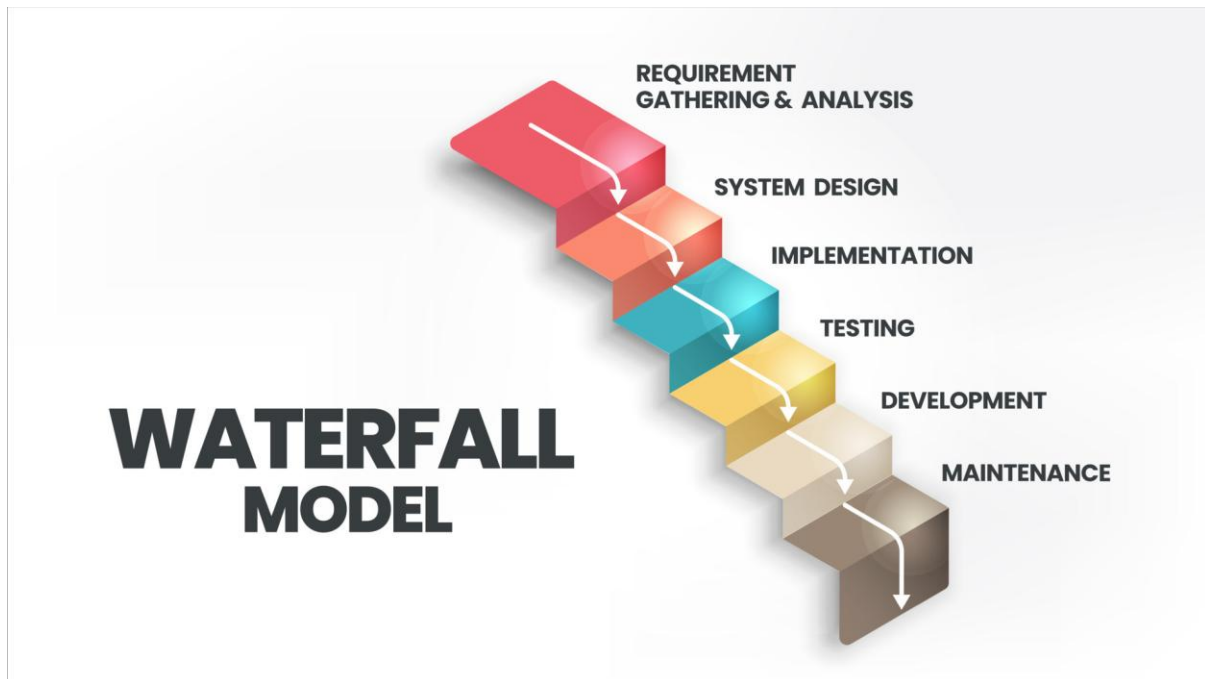


Figure 5 - Waterfall model (vecteezy.com)

The Spiral model is an iterative and risk-driven software development methodology, introduced by Boehm (1988) as an enhancement over purely linear approaches. It combines elements of iterative prototyping with systematic risk analysis, progressing through four recurring phases: planning, risk assessment, engineering, and evaluation. Its main strength lies in its explicit focus on risk management, allowing teams to address uncertainties early and build prototypes to validate critical design and functional assumptions. This makes Spiral particularly suitable for projects characterized by complex requirements, regulatory constraints, or high-risk environments such as financial technology applications (Boehm, 1988).

Despite these strengths, Spiral also has significant limitations. It is resource-intensive, requiring substantial investment in risk analysis, documentation, and multiple iterations of prototyping. Moreover, its complexity demands teams with high technical maturity and strong risk management capabilities. As comparative studies on software development methodologies point out, Spiral can be challenging for smaller teams with limited resources, as the overhead associated with repeated cycles may outweigh its benefits in early-stage projects (Balaji and Murugaiyan, 2012).

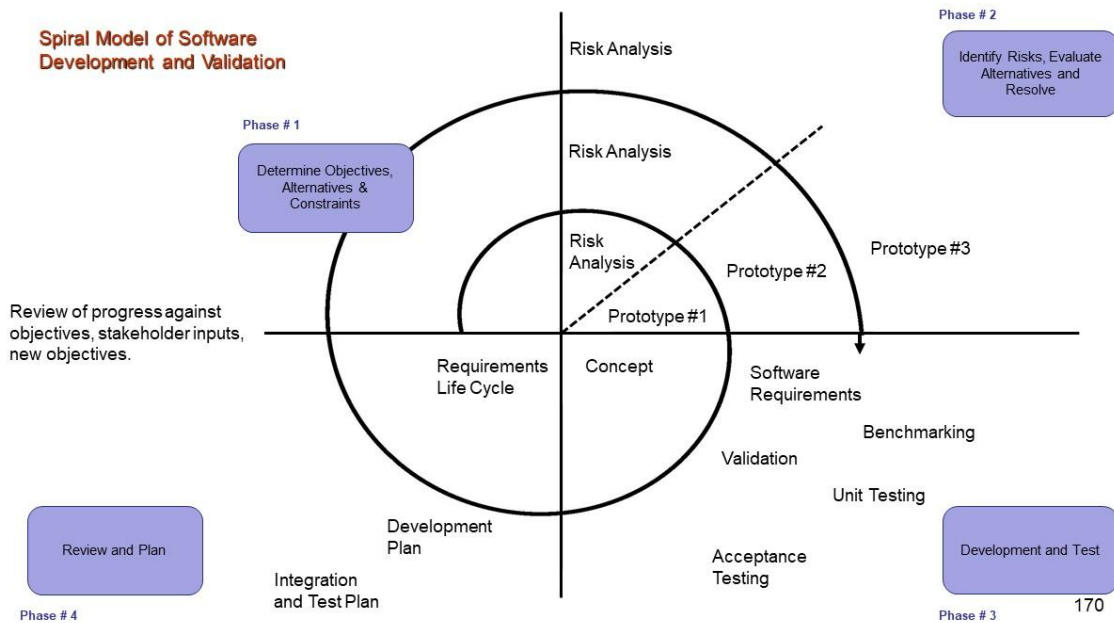


Figure 6 - Spiral Model. Software Development and Validation (www.presentationeze.com)

3.4 Choosing solutions

Based on the characteristics of SplitPal, the technology selection should not be based only on the popularity of each framework, but on the nature of the system itself: a mobile-first group expense management application with multiple financial workflows, real-time interactions, and strict requirements for data consistency. SplitPal is not simply a basic bill-splitting application; it also includes multi-item invoices, different splitting rules, payment requests, internal wallet operations, recurring subscriptions, receipt upload, chat, notifications, and settlement flows between users. Therefore, the selected solution must support user experience, backend business orchestration, and flexible data management while still maintaining sufficient control over critical financial operations.

For the user interface layer, Flutter was selected because SplitPal requires a consistent mobile-first experience across complex financial workflows. First, SplitPal is not a simple CRUD-based application, but a mobile application where users need to complete multi-step flows such as creating groups, adding invoices, splitting items by different rules, viewing debts, confirming payment requests, managing subscriptions, and tracking notifications. These flows contain many states, input points, and decisions that directly affect debts between group members. Therefore, the frontend must provide a clear and consistent interface that reduces the risk of user mistakes when entering amounts, selecting participants, or confirming payments. Flutter is suitable in this context because it supports the development of stateful, interactive, and consistent mobile interfaces across both Android and iOS.

Second, Flutter is appropriate because SplitPal's frontend must integrate with many native mobile capabilities. The application does not only display data from the backend, but also handles features such as receipt image upload, QR scanning, deep linking for joining groups, push notifications, secure token storage, and real-time socket communication. These requirements make the frontend the direct interaction layer between the user, the mobile device, and the financial workflows of the system. By using Flutter, these mobile capabilities can be managed within a unified codebase instead of maintaining separate experience logic for each platform. This is especially important for SplitPal because mobile interaction is not a secondary channel, but the primary way users access and operate the product.

Third, Flutter helps maintain development speed and interface consistency as the product scope continues to expand. SplitPal is no longer limited to basic bill-splitting features; it has expanded into receipts, subscriptions, chat, notifications, and wallet operations. As the number of modules increases, a major frontend risk is fragmented user experience across screens or inconsistent behavior between Android and iOS. With a single codebase, Flutter allows the development team to implement features faster while maintaining a unified design language across the whole system. Therefore, Flutter is a suitable choice for the user interface layer because it addresses three main needs of SplitPal: mobile-first workflows, native capability integration, and consistency as the product grows.

For the server-side layer, Node.js, Express, and TypeScript were selected because SplitPal requires business orchestration and real-time event handling rather than simple CRUD processing. First, the backend must handle REST APIs, WebSocket communication, payment callbacks, notification dispatching, recurring billing, exchange rate lookup, and scheduled jobs at the same time. This is an I/O-bound backend where data is continuously exchanged between the client, database, payment service, notification service, and scheduler. Node.js fits this characteristic because its asynchronous model supports systems with many connections and concurrent events. For SplitPal, this is more important than generic performance claims, because the core value of the backend lies in its ability to coordinate multiple business modules reliably.

Second, Express and TypeScript are suitable because SplitPal contains many interdependent business modules that require strict control over data contracts. A change in an invoice may affect debts, payment requests, transfers, transaction history, and notifications. If request/response shapes or domain objects are not well controlled, the system can easily produce integration errors between services. Express helps organize the backend into a clear route-controller-service structure, while TypeScript supports data types, interfaces, and contracts between business modules. In a group finance system, type safety does not directly solve monetary precision, but it reduces the risk of structural data errors when handling complex business flows.

Third, Node.js, Express, and TypeScript provide a balance between development speed and control over money-related workflows. SplitPal includes sensitive operations such as creating payment requests, updating balances, processing transfer settlements, top-up, withdrawal, and subscription charging. These flows do not only require working APIs, but also validation,

transaction boundaries, locking, retry strategies, and controlled rounding to avoid race conditions or inconsistent states. Express keeps the architecture simple enough for fast development while the product scope is still evolving, whereas TypeScript makes the logic between modules safer and easier to maintain. Therefore, this backend stack is a reasonable choice because SplitPal needs flexibility for rapid development while still requiring strict control over core financial operations.

For the data layer, MongoDB, Mongoose, and Redis were selected because SplitPal requires both flexible data modelling and controlled consistency. First, MongoDB is suitable because SplitPal's data is aggregate-based and semi-structured, which fits a document model better than a rigid relational model at the current stage. The system does not only store users and transactions, but also groups, group members, invoices, invoice items, split rules, original debts, transfers, receipts, messages, notifications, subscriptions, and billing history. Many of these entities have flexible structures, change depending on business features, and can be viewed as data clusters around a specific domain. MongoDB allows these data clusters to be represented in a document-oriented way, making the system more adaptable as features such as receipts, notifications, chat, and subscriptions continue to evolve.

Second, MongoDB can still be appropriate for SplitPal if it is used together with Mongoose, transaction/session support, and service-level controls. SplitPal has flows that require high consistency, such as payment requests, transfer settlement, balance updates, and subscription charging. However, not every part of the system needs to be designed like a core banking system from the beginning. Some modules, such as chat, notifications, receipts, and billing history, are more content- and event-oriented than strictly financial transactions. Therefore, MongoDB is a reasonable choice if the system clearly identifies which flows require strict control and applies validation, transaction boundaries, concurrency control, and locking where necessary. Mongoose adds schema discipline, validation, indexes, and clearer model organization, reducing the risk caused by excessive flexibility in a document database.

Third, Redis is added because SplitPal needs not only correct data persistence but also fast response and safe coordination of concurrent tasks. For flows such as caching exchange rates, speeding up user/group-based queries, or preventing multiple processes from operating on the same payment request at the same time, Redis should not be treated as an optional accessory but as an important part of the data architecture. Redis can support short-lived cache, reduce load on the main database, and provide short-lived locks for sensitive operations. As a result, the combination of MongoDB, Mongoose, and Redis creates a balanced solution: MongoDB provides flexible persistence, Mongoose adds schema discipline, and Redis supports caching and locking to improve responsiveness and reduce the risk of state conflicts. Therefore, this solution is suitable for a system with a diverse and fast-changing domain while still having several financial workflows that require strict control.

For the development methodology, the Waterfall model was selected as the main approach for this project. Although SplitPal is a modern financial application with many interactive modules, the project itself was developed within an academic final-year project context, where the submission deadline, report structure, required deliverables, and assessment criteria

were already clearly defined from the beginning. Therefore, a sequential development process was more suitable than a highly flexible Agile process.

The Waterfall model fits SplitPal because the project required strong documentation before implementation. Before coding the system, the team needed to clarify the problem domain, analyse similar applications, define functional and non-functional requirements, design the system architecture, prepare the database schema, and produce UML diagrams. This sequence helped reduce ambiguity before development started. For a group finance application, this is important because core data objects such as users, groups, invoices, debts, payment requests, transfers, wallet transactions, and subscriptions are closely connected. If the requirements and database structure are not planned carefully, later changes may cause inconsistencies in financial workflows.

Another reason for choosing Waterfall is that SplitPal contains several money-related processes that require careful design before implementation. Features such as invoice splitting, debt generation, payment request creation, wallet transfer, subscription renewal, withdrawal, and transaction logging cannot be developed only by trial and error. These workflows need clear input data, expected outputs, validation rules, and transaction boundaries. By following Waterfall, the team could first design the financial logic and data flow, then implement the backend and frontend based on a more stable structure.

The main trade-off of using Waterfall is reduced flexibility. If new requirements appear late in the project, they may require changes across the requirement, design, database, backend, and frontend layers. This can be costly, especially in SplitPal because one change in invoice logic may also affect debt calculation, payment requests, notifications, and transaction history. To reduce this limitation, the project scope was controlled around the main required modules, while additional features such as AI receipt extraction, advanced analytics, and future financial optimization were treated as extensions rather than core dependencies.

Overall, Waterfall was chosen because it matches the academic nature of the project and supports a clear progression from research, requirement analysis, design, implementation, testing, and evaluation. For SplitPal, this approach helped the team maintain traceability between the identified market gaps, the selected solution, the designed system architecture, and the implemented features. Although Waterfall is less adaptive than Agile, it provided a suitable structure for delivering a complete, well-documented, and assessable final-year software project.

4 Requirement Analysis

In the context of the growing need for group expense management, many applications have been developed to help users record shared expenses, calculate debts, and simplify the settlement process between members. Among them, Splitwise and Tricount are two applications that are highly relevant to the development direction of SplitPal.

Analysing these applications helps identify the functions that have already been well implemented in the market, while also highlighting the remaining gaps in group financial management. In particular, most existing applications mainly focus on splitting expenses after they have already occurred. In contrast, SplitPal aims to develop a more structured

group financial management system, which includes balance management, purpose-based fund allocation, subscription management, and proactive alert support.

4.1 Splitwise Application

Splitwise is one of the most popular group expense management applications in the international market. The application is designed to help users record expenses within a group, identify who paid for an expense, who participated in that expense, and how much each member needs to pay back. Splitwise is commonly used in situations such as group travel, shared housing, splitting bills between friends, or managing repeated expenses in a small group.

In terms of strengths, Splitwise handles basic expense splitting quite well. Users can create groups, add members, enter expenses, choose the payer, and select different splitting methods such as equal split, exact amount split, or percentage-based split. After the expenses are recorded, the system automatically calculates the balances between members and suggests an optimized way to settle debts in order to reduce the number of transactions needed. This is an important strength because it helps users avoid manual calculations, reduce mistakes, and improve transparency in personal debts.

In addition, Splitwise is suitable for groups that have many small expenses occurring regularly. Instead of transferring money immediately every time a cost occurs, users can record all expenses in the system and settle the total amount after a period of time. This approach makes debt management more flexible, especially for friend groups or shared-living groups with many small daily expenses.

However, when viewed as a long-term group financial management system, Splitwise still has several limitations. First, the application mainly works as a tool for recording expenses after they have already occurred. This means that the system only answers the question of “who owes whom and how much”, but it does not fully answer questions such as “does the group have enough money to spend?” or “which source of money should this expense be taken from?”. Splitwise does not provide a mechanism to manage the actual balance of a group, check spending capacity before recording an expense, or connect an expense to a specific fund source.

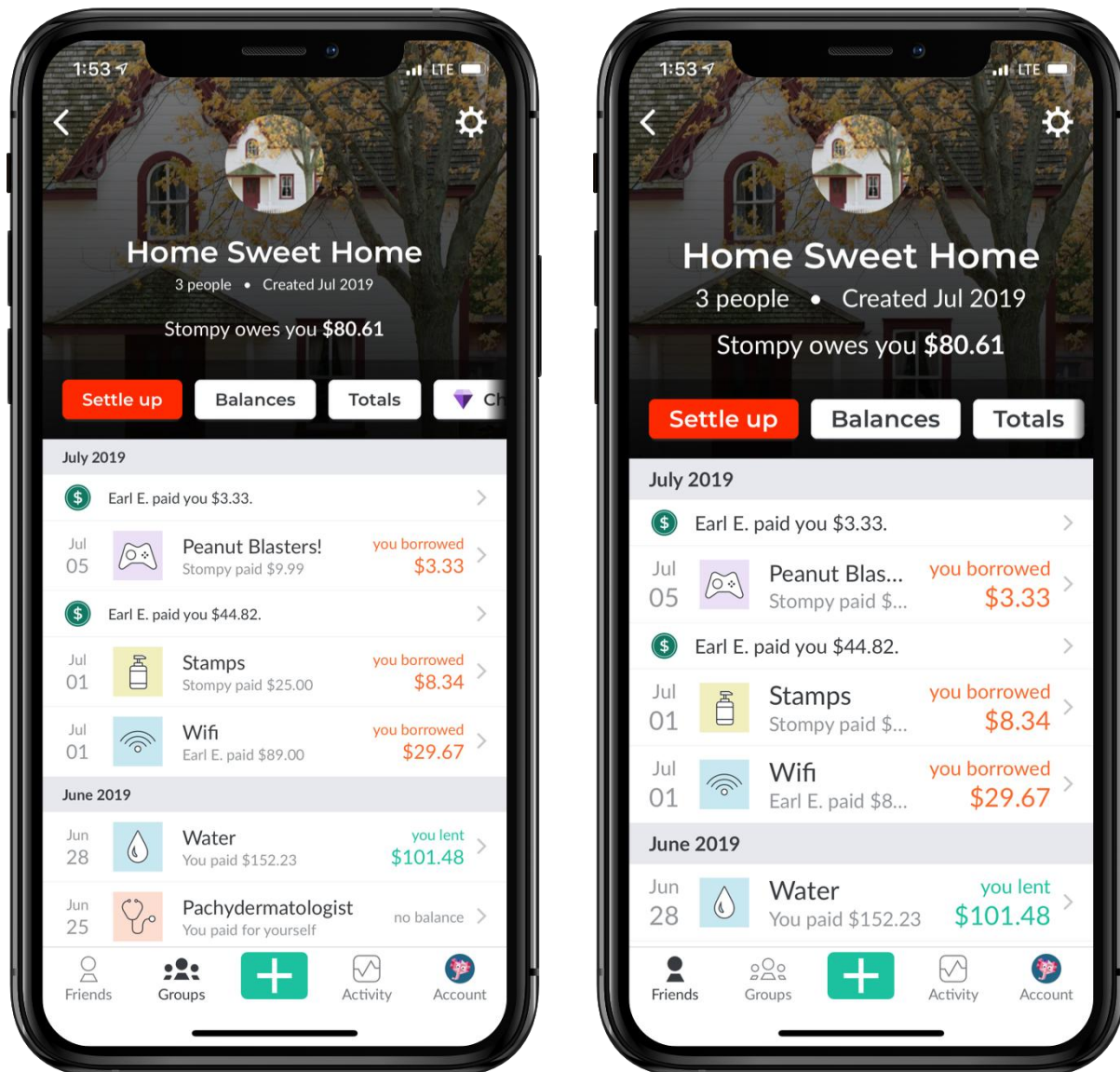


Figure 7 - Splitwise Group details UI (<https://blog.splitwise.com/>)

As shown in figures, Splitwise mainly structures the user flow around adding an expense, selecting the payer and calculating how the cost should be shared among members. This supports the observation that Splitwise performs effectively as a post-expense accounting tool. However, the interface does not demonstrate mechanisms for checking actual group balance, allocating funds before spending, or linking a recurring digital service to specific usage slots.

Another limitation is that Splitwise does not strongly support purpose-based fund allocation before spending. In real shared-living situations, groups often need to separate money for different purposes, such as rent, utilities, internet, shared savings, or recurring subscription services. However, Splitwise mainly records expenses after they happen, instead of allowing the group to prepare and allocate money in advance for each spending purpose.

Besides that, Splitwise does not fully solve the problem of subscription management based on usage slots. Nowadays, many groups share digital services such as Netflix, Spotify, Google

Drive, Canva, or learning software. These services often have limits on the number of users or available slots. However, Splitwise usually treats a subscription as a normal expense. It does not clearly show who is using which slot, which member has access rights, or how the payment responsibility should be allocated based on slot usage.

Based on the analysis above, Splitwise can be seen as a strong application for splitting expenses and optimizing personal debts. However, it is not sufficient to become a complete group financial management system. This is the gap that SplitPal can address by expanding the scope from simply “recording debts” to “managing group finances with better control”.

4.2 Tricount Application

Tricount is a group expense management application that focuses on simplicity and ease of use. It is often used in short-term situations such as travelling, organizing events, joining group activities, or splitting costs among friends. Compared to Splitwise, Tricount provides a more minimal user experience, allowing users to quickly create a group, add expenses, and view the splitting results without setting up too much information at the beginning.

In terms of strengths, Tricount has a clear interface and a simple user flow. Users can create a “tricount” for a trip or an event, add the related members, enter expenses, and choose who participated in each expense. After that, the system automatically calculates the balance and shows which member needs to pay whom. This makes Tricount suitable for users who do not need a complex financial management system, but only need a quick, lightweight, and easy to use tool.

Another important advantage of Tricount is its suitability for short-term groups. For example, during a trip, members may continuously have different expenses such as food, transportation, entrance tickets, or accommodation. Tricount allows all these expenses to be recorded in one shared space and then automatically summarizes the amount that needs to be settled at the end of the trip. As a result, users can avoid manual tracking through paper notes, spreadsheets, or group chat messages.

However, because Tricount focuses strongly on simplicity, it also has several limitations when applied to long-term group financial management. First, the application does not provide a mechanism to manage the actual balance of a group. Users can record expenses and calculate debts, but the system does not show how much money the group currently has, which amount has already been allocated, which amount is still available, or whether an upcoming expense may exceed the group’s financial capacity.

Second, Tricount does not support purpose - based fund allocation. In shared-living groups, financial management is not only about splitting money after buying something or paying a bill. The group often needs to plan ahead for recurring expenses, savings, or shared services. Without a mechanism for allocating money in advance, users still have to rely on agreements outside the system, spreadsheets, or messages to manage these financial plans.

Third, Tricount does not provide advanced functions related to subscription lifecycle management. Subscription services usually have renewal cycles, limited user slots, different access rights among members, and the need for recurring payment reminders. If a

subscription is only recorded as a normal expense, the system cannot effectively support tracking who is using the service, when the subscription needs to be renewed, and who needs to contribute for the next period.

In addition, Tricount also has limitations in terms of alerts and data analysis. The application helps users understand how much money needs to be paid, but it does not strongly focus on proactive alerts such as payment due reminders, budget limit warnings, low balance alerts, or spending trend analysis over time. This makes Tricount suitable for handling simple expenses, but not strong enough to support sustainable group financial management.

Based on the analysis above, Tricount can be seen as an effective tool for quick and short-term expense splitting. However, for long-term shared-living groups or groups that regularly deal with many types of recurring expenses, Tricount still does not fully meet the needs for balance management, budget allocation, subscription management, and proactive financial control.

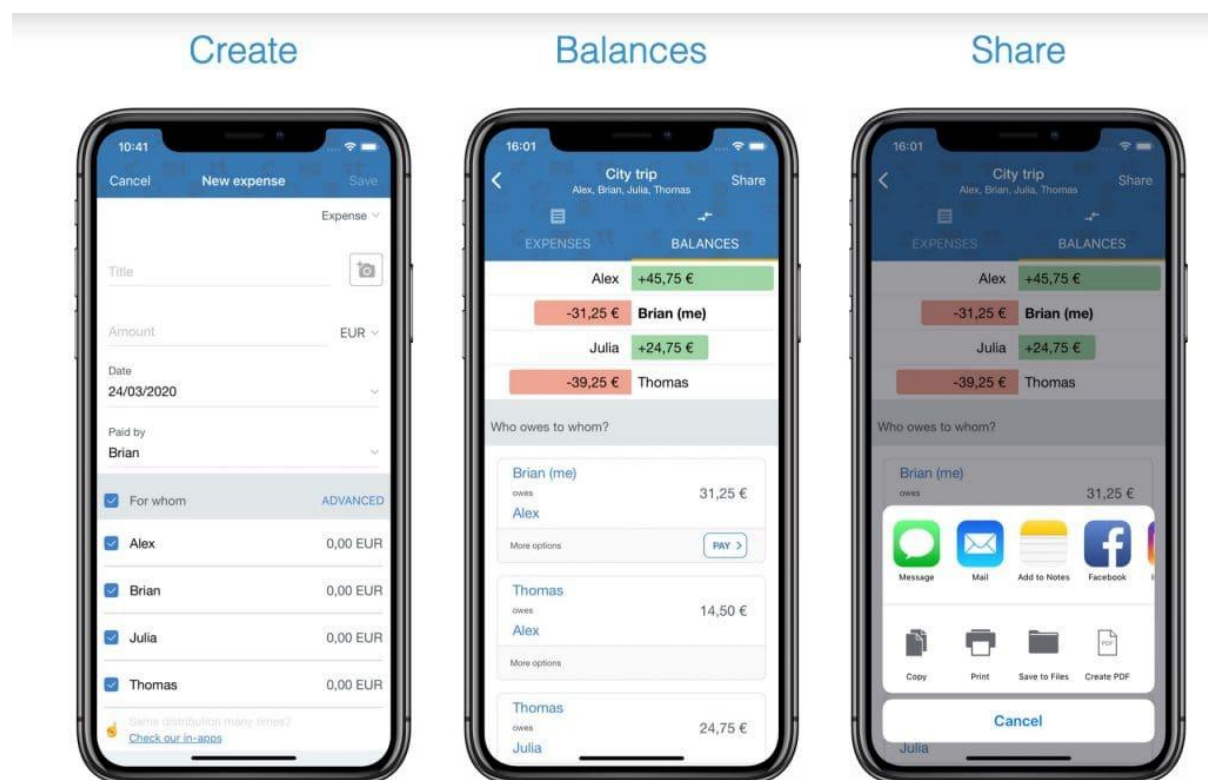


Figure 8 - Tricount Expense management flow (<https://josiegirlblog.com/>)

As shown in figures, Tricount presents shared expenses and balances in a simple and easy-to-understand format. This design supports quick expense recording and settlement, especially for trips or short-term groups. However, the interface mainly represents expenses and balances after costs have been added, rather than supporting proactive financial planning, budget allocation, subscription lifecycle management, or balance-based spending control.

4.3 Market gaps identified from similar applications

Application	Main Purpose	Strengths	Limitations	Meaning for SplitPal
Splitwise	Group expense management and debt optimization between members	Supports group creation, expense recording, payer selection, different splitting methods, and optimization of the number of settlement transactions. It is suitable for friend groups, shared-living groups, or travel groups with many expenses.	Mainly handles expenses after they have already occurred. It does not manage actual group balance, check spending capacity before payment, allocate money by purpose, or support subscription management based on usage slots.	Shows that expense splitting and debt calculation are important needs, but also reveals gaps in balance management, budget control, and financial planning before spending.
Tricount	Simple and quick recording and splitting of group expenses	Has an easy-to-use interface and is suitable for short-term situations such as travelling, events, or group activities. Users can quickly create a group, enter expenses, and view the settlement results clearly.	Because it is designed to be simple, it lacks advanced financial management features. It does not provide actual balance management, purpose-based fund allocation, subscription lifecycle management, renewal reminders, or deeper spending analysis.	Shows that a simple user experience is an important advantage. SplitPal should add more advanced financial features while still keeping the user flow clear and easy to understand.

From the analysis of Splitwise and Tricount, it can be seen that current applications have already solved the basic problem of expense splitting quite well. Both applications help users record expenses, identify participating members, calculate debts, and reduce the amount of manual calculation. These are fundamental functions that any group expense management application should provide.

However, a common limitation of these applications is that they mainly approach the problem in a reactive way after spending has already happened. The system records an expense once it has occurred and then calculates the debts between members. This approach is suitable for short-term groups, but it is not enough for long-term shared-living groups, where users need to plan finances, control available money, and manage many types of recurring expenses. The first gap is the lack of actual balance management. Existing applications can show which member owes whom, but they do not clearly show how much money the group currently has, which amount has already been allocated, and which amount is still available for use. As a result, the system only reflects payment responsibilities, but not the real financial capacity of the group.

The second gap is the lack of purpose-based fund allocation. In practice, a shared-living group may need to separate money for different purposes, such as monthly living expenses, recurring bills, subscriptions, savings, or emergency funds. Without a mechanism for allocating money in advance, financial management still depends heavily on manual agreements outside the system.

The third gap is the lack of subscription management based on usage slots. Shared digital services are becoming more common, but current applications usually record them as normal expenses. This does not fully reflect the nature of subscriptions, where each member may use a specific slot, have individual access rights, and carry a corresponding payment responsibility.

The fourth gap is the lack of proactive alerts. Users often only notice problems after debts have occurred, payments are late, or spending has exceeded the available budget. Features such as low-balance alerts, subscription renewal reminders, budget limit warnings, or upcoming payment notifications have not been fully used in existing applications.

Finally, current applications still depend heavily on manual data entry. Users have to enter invoice information, amounts, participating members, and expense descriptions by themselves. This increases the time needed to use the system, creates a higher risk of errors, and limits the ability to use data for analysing spending behaviour.

Market Gap	Problem identified	SplitPal Improvement
Current applications mainly record expenses after they have already occurred	Users know who owes whom, but they do not know whether the group has enough financial capacity before spending	SplitPal adds an account model with actual balance, allowing the system to reflect the group's spending capacity instead of only recording debts
There is no mechanism to control the total allocated amount	A group may record many expenses or spending plans without linking them to real available funds	SplitPal applies the rule that the total allocated amount cannot exceed the available balance, which helps support better financial control
Money is not managed by purpose before spending	Group money can easily be treated as one general fund, making it difficult to separate money for daily living, savings, or subscriptions	SplitPal allows money to be allocated into different purposes such as budget, subscription, and saving, helping the group prepare financially before spending
Subscriptions are often treated as normal expenses	Existing applications do not properly reflect shared digital services with limited user slots	SplitPal develops a subscription slot model, where each slot can be assigned to a specific member, linking service access with payment responsibility
Proactive alerts are limited	Users often only detect problems after debts, late payments, or budget overruns have already happened	SplitPal integrates in-app notifications and push notifications to remind users of subscription renewals, low balance, budget overruns, or upcoming payment responsibilities
Applications depend heavily on manual input	Users must enter invoices and expenses manually, which takes time and can lead to mistakes	SplitPal aims to integrate OCR to extract data from invoices and use LLM support to help analyse spending habits
Financial data can become fragmented	It is difficult to fully trace the flow from the original expense to debt, payment, and completed transaction	SplitPal builds financial data lineage through the chain of Invoice, InvoiceItem, OriginalDebt, PaymentRequest, Transfer, TransferDebtAllocation, and Transaction to ensure clear financial traceability

4.4 Conclusion

From the analysis of similar applications, it can be seen that Splitwise and Tricount both solve the basic need for expense splitting quite well. Splitwise is strong in recording debts

and optimizing settlements between members, while Tricount stands out because of its simplicity and suitability for short-term groups. However, both applications still mainly focus on handling expenses after they have already occurred.

For long-term shared-living groups or groups that regularly share many digital services, financial management is not only about splitting bills. Users need a system that can support actual balance management, purpose-based fund allocation, budget control, subscription tracking, payment reminders, and full traceability across the lifecycle of an expense.

Therefore, SplitPal is positioned not only as an expense-splitting application, but also as a structured group financial management system. The difference of SplitPal lies in expanding the scope from “expense sharing” to “group financial management”, through components such as account balance, purpose-based allocation, subscription slot management, proactive notifications, OCR/LLM integration, and financial data lineage.

In other words, SplitPal does not directly replace applications such as Splitwise or Tricount at the basic expense-splitting level. Instead, it develops further to solve more complex financial problems in long-term shared-living contexts, such as students living together, young professionals renting a house together, small families, or groups that share many digital services.

4.5 Stakeholder Requirements

Regular Users

Regular users are individuals who use SplitPal to manage shared expenses, wallet balances, personal receipts, subscriptions, and group payments. They benefit from clearer debt tracking, easier payment management, and reduced confusion in shared spending situations.

Regular users expect SplitPal to help them understand their personal financial position within each group without having to calculate everything manually. The system should allow them to clearly see who they owe, who owes them, where each expense comes from, and the current status of each payment.

The user experience should be simple, understandable, and trustworthy, especially when users perform actions related to wallets, payments, withdrawals, or transaction confirmation. In addition, users expect the system to reduce disputes in shared spending situations by providing transparent, traceable, and easy-to-review financial information.

Group Owners and Group Administrators

Group owners and group administrators are responsible for managing shared expense groups. They benefit from structured control over group membership, invoice settlement, and payment coordination.

Group owners and administrators expect the system to help them manage groups in a more organized way, especially when there are multiple members, multiple expenses, and different payment statuses. They need to understand the overall financial situation of the group,

including outstanding debts, payment progress, and recurring expenses, so that they can coordinate group payments more effectively.

The system should reduce the amount of manual reminding, limit mistakes in expense splitting, and provide a clear mechanism for handling unpaid amounts. From a business perspective, this group expects SplitPal to work as a reliable group finance management tool that improves transparency and accountability among members.

Group Members

Group members participate in shared expenses but have fewer administrative responsibilities than group owners or administrators. They benefit from transparent calculation of what they owe and what has already been paid.

Group members expect to track their payment obligations clearly and fairly. They need to understand why they are asked to pay a certain amount, which invoice the amount comes from, how the expense was split, and whether the payment has already been completed.

The system should reduce the feeling of being passive in shared finance situations by providing understandable information, timely notifications, and transparent transaction history. The experience for this group should focus on clarity, fairness, and self-verification rather than simply receiving payment requests from others.

System Administrators / Maintainers

System administrators or maintainers are responsible for configuring, deploying, monitoring, and maintaining the SplitPal platform. They benefit from a modular backend architecture, environment-based configuration, and clear deployment files.

System administrators expect SplitPal to operate stably, be easy to configure, and remain maintainable across different environments. They need a clear system structure to support deployment, error tracking, payment flow verification, incident handling, and future extension.

Since SplitPal involves financial data, maintainers also need important operations such as payment processing, balance updates, notification delivery, and background jobs to be traceable and recoverable when errors occur. From an operational perspective, the system should reduce the risks of data loss, transaction inconsistency, and difficulty in debugging or upgrading the platform.

4.6 Business Requirement

Module 1: Authentication and Account security

No	Feature	Priority	Description
1	User registration and email Verification	High	Users can register with email and password. The system sends an OTP to verify the account before activation.
2	User Login and JWT sessionmanagement	High	Users can log in with valid credentials and receive a JWT access token for authenticated requests.

3	Google login	High	Users can log in using Google through Firebase Authentication. The backend verifies the Firebase ID token before creating or authenticating the user.
4	Password recovery	High	Users can request password reset, verify OTP, and set a new password through a reset token.
5	Change password	High	Authenticated users can change their password after confirming their current password and verifying an OTP.
6	Two-Factor authentication	High	Users can enable TOTP-based 2FA, scan a QR code, verify setup, receive backup codes, check 2FA status, and disable 2FA when needed.
7	User profile management	Medium	Users can view and update basic profile information such as display name and avatar.
8	Theme and Onboarding preference	Low	The application stores theme preference and onboarding status to improve the personal user experience.

Module 2: Group and Member management

No	Feature	Priority	Description
9	Group Management	High	Users can create groups, view joined groups, view group details, and update group information based on their role.
10	Group Invitation Management	High	Group owners or admins can invite members by email. Invited users can accept invitations through invite tokens.
11	Join Group by Code or Deep link	High	Users can join a group using a join code, invitation token, deep link, or QR-based mobile flow. If the user is not authenticated when opening the link, the app can defer the navigation and continue the join flow after login..
12	Group Role Management	High	The system supports OWNER, ADMIN, and USER roles. Owners can grant admin permissions and transfer ownership.
13	Member Removal and Leave Group Control	High	Owners or admins can remove members according to permission rules. Users can leave a group only when they do not have unresolved net debt.
14	Group Balance Calculation	High	The system calculates each member's owed amount, lent amount, and net balance based on remaining original debts.

Module 3: Invoice, Debt, and Payment request management

No	Feature	Priority	Description
15	Invoice Management	High	Group members can create, view, update, delete, submit, and view details of invoices before they are locked.
16	Invoice Item Split Management	High	Invoice items can be split by equal share, percentage, custom amount, or weight. The system validates percentage totals, custom totals, positive share weights, assignee completeness, and group membership before saving or confirming the invoice.
17	Original Debt Tracking	High	After invoice creation or confirmation, the system creates original debt records between debtors and creditors for traceable debt management.
18	Payment Request Management	High	Group owners or admins can create payment requests from submitted invoices. Related invoices are locked after the payment request is issued.
19	Debt Optimization and Transfer Allocation	High	The system generates optimized transfers and allocates them to original debts to reduce manual settlement complexity.
20	Transfer Payment with OTP	High	Users can pay pending transfers using wallet balance after verifying an OTP sent by email.

21	Payment Request Cancellation and Refund	High	Payment requests can be cancelled according to business rules. If partial payments already exist, the system handles refund and debt restoration.
22	Payment Request Expiration and Reminder	High	The system automatically checks expired payment requests and sends reminders for pending transfers.
23	Multi-Currency Invoice Support	Medium	Invoices can use currencies different from the group base currency. The system stores exchange rates and converted debt values.
24	Invoice Adjustment	Medium	Users can create adjustment invoices linked to previous invoices when correction or compensation is needed.

Module 4: Wallet, Payment, and Transaction management

No	Feature	Priority	Description
25	Wallet Balance Management	High	Each user has a personal wallet balance used for top-up, withdrawal, transfer payment, subscription billing, and refunds.
26	VNPay Top-up	High	Users can create top-up orders through VNPay. The backend verifies VNPay return/IPN data before updating wallet balance.
27	VNPay Transfer Payment	High	Users can pay certain transfer requests through VNPay, and the system updates transfer status after successful gateway verification.
28	Withdrawal Management	High	Users can request withdrawal, receive an OTP, verify the OTP, and have the withdrawal recorded in the transaction history.
29	Transaction History	High	Users can view wallet-related transactions such as top-ups, withdrawals, sent transfers, received transfers, refunds, and subscription fees.

Module 5: Subscription and Recurring bill management

No	Feature	Priority	Description
30	Subscription Management	High	Authorized group members can create subscriptions with name, description, amount, currency, billing cycle, and owner information.
31	Subscription Invitation	High	Subscription owners can invite group members to join a subscription. Invitees can accept or decline the invitation.
32	Subscription Member Billing	High	When a member accepts a subscription, the first billing cycle is charged from the member's wallet and transferred to the subscription owner.
33	Automatic Subscription Renewal	High	The scheduler processes due subscription members, charges their wallets, updates next billing dates, and records billing results.
34	Subscription Failure Handling	High	If a member has insufficient balance, the system records failed billing attempts and may mark the member as left after repeated failures.
35	Recurring Bill Template Management	High	Groups can create recurring bill templates for repeated expenses with daily, weekly, or monthly cycles. Authorized users can also update, archive, pause, resume, and trigger immediate generation of template-based invoices.
36	Recurring Draft Invoice Confirmation	High	The system generates draft invoices from active bill templates. Owners or admins can review and edit the draft invoice before confirming it, and official debt records are created only after confirmation.
37	Subscription Billing History	Medium	Users can view subscription billing history, including success status, failed members, total collected amount, and member-level billing results.

Module 6: Receipt, Budget, and Forecast management

No	Feature	Priority	Description
----	---------	----------	-------------

38	Receipt Diary Management	Medium	Users can record personal receipts with image, total amount, note, date, and spending tags.
39	Receipt Tag Management	Medium	Users can create and manage personal spending tags with name, color, icon, and monthly budget.
40	Budget Summary	Medium	The system summarizes spending by tag using receipts, completed transfers, and subscription billing records.
41	Cashflow Forecast	Medium	The system forecasts future wallet balance based on current balance, confirmed subscription charges, committed outgoing transfers, and expected incoming transfers. It provides both safe balance and expected balance views across the forecast horizon.
42	Risk Alerts	Medium	The forecast module identifies financial risks such as negative balance, clustered outflows, subscription-at-risk conditions, high retry billing members, and dependency on expected incoming transfers.

Module 7: AI and OCR support

No	Feature	Priority	Description
43	AI Text Invoice Extraction	Medium	Users can enter invoice text, and the system extracts suggested invoice information using Gemini AI when configured or rule-based fallback logic.
44	OCR Invoice Extraction	Medium	Users can upload invoice images for AI-based extraction. The system validates image type and file size before processing.
45	AI Debt Reminder Generation	Low	The system can generate debt reminder messages in different writing styles to help users remind debtors more conveniently.

Module 8: Notification and Real - time communication

No	Feature	Priority	Description
46	Notification Management	High	Users can receive, view, mark as read, and delete notifications related to groups, transfers, subscriptions, billing, and payment requests.
47	Push and Local Notifications	High	The system supports in-app real-time notifications through Socket.IO, push notifications through Firebase Cloud Messaging when configured, and Flutter local notifications for replaying missed unread notifications on the client
48	Group Chat	Medium	Group members can send and receive real-time messages and view message history through Socket.IO and backend chat APIs.

Module 9: Dashboard and Utility services

No	Feature	Priority	Description
49	Personal Dashboard	Medium	Users can view wallet balance, recent cashflow, pending debts, open payment requests, upcoming subscription information, forecast summary, and recent transactions.
50	Group Dashboard	Medium	Users can view group-level payment request status, collected amount, pending amount, pending transfers, and recent transfer activity.
51	Currency Converter	Low	Users can convert between supported currencies using exchange rates cached by the backend service.

Module 10: System Operation, Validation, and Reliability

No	Feature	Priority	Description
52	Scheduler and Background Processing	High	The system automatically processes subscription billing, payment request reminders, payment request expiration, and recurring bill generation.

53	Error Handling and Input Validation	High	The system validates user input, split rules, group membership, role permissions, upload constraints, amount values, and business conditions before processing requests.
54	Rate Limiting and Abuse Prevention	High	The backend applies rate limits to sensitive actions such as OTP verification, OTP resend, receipt operations, and AI-related requests.
55	Concurrency Control	High	The system uses locks and database transactions to reduce conflicts in financial operations such as payment request creation, transfer payment, refunds, and billing.

4.7 Solution Requirement

After defining the business requirements in Section 4.6, this section explains how those requirements are translated into executable system modules. The purpose of this section is not to repeat the functional requirements, but to show the implementation direction of each requirement group, including the related application module, backend responsibility, main data objects, and expected system outcome.

This mapping helps ensure requirement traceability. Each business requirement is connected to a specific part of the SplitPal solution so that the design, implementation, and testing stages can be aligned with the original business needs.

Requirement Area	Business Requirements	Implementation Module	Main Data / Technical Components	Solution Response
Authentication and Account Security	User registration, email verification, login, Google login, password recovery, change password, two-factor authentication, profile management	Authentication and Account Module	User, OTP flow, JWT, bcryptjs, Firebase Authentication, TOTP, secure token storage	This module protects access to the system and ensures that only verified users can perform financial operations. JWT is used for authenticated API access, while OTP and optional 2FA provide additional protection for sensitive account and payment-related actions.
Group Management	Create group, view group, update group, delete group, invite members, join group, manage member roles	Group Management Module	Group, GroupMember, Invite, join code, role-based access control	This module creates the shared financial space where expenses, members, invoices, debts, and subscriptions are managed. Role-based permissions separate the responsibilities of owner, admin, and normal member, reducing the risk of unauthorized group changes.
Invoice and Expense Splitting	Create invoice, add invoice items, assign participants, apply different split methods, support multi-currency expenses	Invoice and Debt Module	Invoice, InvoiceItem, OriginalDebt, exchange-rate service, split calculation logic	This module converts a shared expense into structured debt records. Instead of storing only a final balance, the system keeps the invoice, item-level details, participants, split logic, and generated debts. This supports flexible cost allocation and makes the reason behind each debt traceable.

Debt Tracking and Financial Traceability	View outstanding debts, track who owes whom, review debt source, support transparent payment history	Debt and Audit Trail Module	OriginalDebt, PaymentRequest, Transfer, TransferDebtAllocation, Transaction	This module allows users to understand not only how much they owe, but also why they owe that amount. The financial lineage from invoice to debt, payment request, transfer, allocation, and final transaction improves transparency and reduces disputes between group members.
Payment Request and Settlement	Create payment request, pay transfer, confirm settlement, manage payment status, prevent duplicate settlement	Payment and Settlement Module	PaymentRequest, Transfer, TransferDebtAllocation, Transaction, wallet balance, OTP verification, MongoDB transaction, lock mechanism	This module handles the transition from recorded debt to actual settlement. Payment requests group unpaid debts into actionable requests, while transfers and allocation records show which debts are resolved by each payment. OTP, transaction boundaries, and locks are used to reduce financial inconsistency.
Wallet and External Payment	Top up wallet, process VNPay payment, view wallet balance, withdraw money, store transaction history	Wallet and Payment Gateway Module	TopUp, Withdrawal, Transaction, VNPay return/IPN callback, secure hash verification	This module manages internal wallet operations and external payment integration. VNPay is used for wallet top-up and payment redirection, while transaction logs record balance changes before and after each operation to support auditability.
Subscription Management	Create shared subscription, assign members to slots, manage billing cycle, renew subscription, track billing history	Subscription and Recurring Billing Module	Subscription, SubscriptionMember, SubscriptionInvitation, BillingHistory, scheduled jobs	This module addresses the gap where shared digital services are usually treated as normal expenses. By modelling subscription slots and member participation, SplitPal links service access with payment responsibility and supports recurring billing cycles.
Receipt and AI-Assisted Invoice Extraction	Upload receipt, extract invoice data, reduce manual input, review extracted information	Receipt and AI Module	Receipt, ReceiptTag, uploaded image, multer, Gemini AI, fallback extraction logic	This module reduces the burden of manual invoice entry. Uploaded receipt images can be processed by Gemini AI to suggest invoice fields such as title, date, items, amount, and currency. However, AI output is treated as assistive data and should be reviewed by the user before becoming financial records.
Notifications and Reminders	Receive payment reminders, subscription renewal reminders, group invitations, status	Notification Module	Notification, Firebase Cloud Messaging, Socket.IO, local notification handling	This module keeps users informed about financial and group activities. Realtime and push notifications help users react to payment requests, renewals, invites, and important system events

	updates, push notifications			without manually checking every module.
Realtime Communication	Group chat, realtime notification delivery, instant update of relevant events	Communication Module	Message, Socket.IO, user rooms, group rooms	This module supports quick coordination inside groups. Socket.IO allows the backend to push new messages and notifications to relevant users, improving responsiveness in shared financial workflows.
Dashboard and Reporting	View financial overview, wallet status, cashflow, upcoming payments, group summary, transaction history	Dashboard and Reporting Module	Dashboard API, Transaction, PaymentRequest, Transfer, Subscription, chart data	This module summarizes financial information from different parts of the system into a user-friendly overview. It helps users understand their current financial status without opening each module separately.
Cross-cutting Reliability and Security	Rate limiting, abuse prevention, data consistency, concurrency control, access control	Middleware and Infrastructure Layer	authMiddleware, role-check middleware, Redis, MongoDB transactions, lock collection, rate limiter	These controls support multiple modules rather than one single feature. Authentication middleware protects APIs, role checks enforce permissions, rate limiting reduces OTP and login abuse, while locks and transactions reduce race conditions in financial operations.

Overall, the solution requirement mapping shows that SplitPal is implemented as a set of connected modules rather than isolated features. Authentication secures access to the platform; group management defines the shared financial context; invoices and subscriptions generate financial obligations; payment requests, transfers, wallet operations, and transaction history complete the settlement process; notifications and realtime communication support coordination; and cross-cutting controls improve reliability and security.

This structure also reflects the main product direction identified from the market gap analysis. SplitPal does not only record expenses after they occur, but also provides stronger support for actual balance management, purpose-based financial control, subscription slot handling, proactive alerts, and financial traceability. As a result, each requirement is connected to an implementation module that contributes to the overall goal of building a more transparent and practical group expense management system.

4.8 Non-Functional Requirement

The non-functional requirements of SplitPal focus on five main aspects: performance, reliability, security, auditability, compatibility, and maintainability. Since the system handles shared expenses, wallet transactions, payment requests, and recurring financial activities, these requirements are important to ensure that the application can operate safely and consistently.

In terms of performance, the backend is built on an Express.js REST API and uses MongoDB as the main database. Important query fields such as user ID, group ID, payment request ID,

notification status, and transaction creation time are supported by Mongoose indexes to improve data retrieval. Redis is also used when configured to support caching and rate limiting, especially for repeated requests or sensitive operations such as login and OTP verification. In addition, scheduled background jobs are used for subscription billing, recurring bill generation, and payment reminders, which helps reduce the need for users to manually trigger these processes.

Reliability and data consistency are especially important because SplitPal manages financial records between multiple users. Instead of storing only final balances, the system separates financial data into invoices, invoice items, original debts, payment requests, transfers, transfer-debt allocations, and transactions. This structure makes each financial operation easier to trace and reduces the risk of losing important context behind a debt or payment. For sensitive workflows such as invoice creation, transfer payment, wallet balance updates, and debt settlement, MongoDB transactions are used where implemented. The system also applies lock mechanisms in certain operations, such as payment request generation, to reduce the risk of duplicated or conflicting updates.

Security is another core requirement of the system. Protected routes use JWT authentication, while user passwords are hashed with bcrypt. OTP verification is applied to sensitive flows such as registration, password reset, payment confirmation, and withdrawal requests. The system also supports optional two-factor authentication using TOTP. To reduce abuse, login and OTP-related endpoints include rate-limiting logic through Redis or an in-memory fallback. On the frontend, authentication tokens are stored using secure storage and are automatically attached to API requests through Dio interceptors. However, before production deployment, the system should still be reviewed through dependency auditing, stricter request validation, penetration testing, and safer production secret management.

Auditability is a key requirement because users need to understand not only how much they owe, but also why they owe that amount. SplitPal supports this by keeping a clear financial history across the main expense and payment records. A debt can be traced from the original invoice and invoice items, through the generated payment request, to the transfer used for settlement and the final wallet transaction. This design improves transparency for group finance management and can help users resolve disputes when there is confusion about a payment or shared expense. It also provides a stronger foundation for future reporting and financial summaries.

In terms of compatibility, the frontend is developed with Flutter, allowing the application to support mobile-oriented interfaces and potentially web-based access from the same codebase. The application includes pages for authentication, dashboard, group management, invoices, wallet operations, subscriptions, receipts, notifications, chat, and forecasting features. This makes the system suitable for users who may need to access their financial information across different devices. Nevertheless, final compatibility testing on Android, iOS, web, and desktop platforms should still be completed before claiming full cross-platform support.

Maintainability is supported by the overall project structure. The backend follows a route-controller-service-model pattern, while the Flutter frontend is organized around feature-based modules. This makes the codebase easier to navigate and allows future features to be added with less disruption to existing modules. However, some frontend providers and backend service files still contain broad responsibilities. Future refactoring should therefore focus on

separating repositories, use cases, validation logic, and smaller service modules to improve testability and long-term maintainability.

4.9 Legal, Social, Ethical and Professional Issues

SplitPal is not only an expense-splitting application. It handles personal information, group financial records, wallet balances, payment requests, transfers, withdrawals, receipt images, and AI-assisted invoice extraction. Therefore, the requirement analysis must consider legal, social, ethical, and professional issues from the beginning. These issues directly affect how the system stores data, protects users, controls access, verifies financial actions, and explains transaction history.

4.9.1 Personal Data and Privacy

SplitPal collects and processes several types of user data, including email, display name, authentication credentials, group membership, notification tokens, receipt images, wallet history, debts, payment requests, transfers, and withdrawal information. Because these records may reveal personal identity and financial behaviour, the system should only collect data that is necessary for authentication, group management, expense tracking, payment processing, and notification delivery.

The system also needs to separate personal financial data from group financial data. For example, group members may need to see shared invoices and debts, but they should not automatically access another user's full wallet history, withdrawal details, or private receipt records. When a user leaves a group, SplitPal should keep the financial records needed for auditability, but unnecessary personal visibility should be reduced.

4.9.2 Payment, Refund and Withdrawal Responsibility

SplitPal includes wallet top-up, payment request, transfer, refund, and withdrawal workflows. These features require stronger control than normal CRUD operations because incorrect updates can directly affect balances and debts. Therefore, the system must treat each money-related operation as a controlled financial state change, not just a simple database update.

Each payment, refund, transfer, and withdrawal should have a clear status such as pending, completed, failed, cancelled, or refunded. Balance changes should only be made after backend validation, not from client-side actions alone. For withdrawal and sensitive transfer actions, OTP verification should be required before the transaction is finalized. This reduces the risk of unauthorized or accidental financial changes.

4.9.3 AI-Assisted Invoice Extraction

SplitPal uses AI/OCR to extract invoice information from uploaded receipt images. This improves user experience by reducing manual input, but it also creates an ethical risk because AI may read the receipt incorrectly. For example, it may extract the wrong total amount, date, currency, item name, or participant suggestion.

Therefore, AI output must be treated as a suggestion, not as a final financial record. The system should show the extracted result to the user for review before creating the invoice. Low-confidence or incomplete extraction should be clearly marked, and users must be able to edit the result manually. This keeps financial responsibility with the user and prevents AI errors from automatically creating wrong debts. The current report already positions OCR as a support function with confidence and error information rather than an automatic database-writing process.

4.9.4 Authentication and JWT Security

SplitPal uses authentication mechanisms such as JWT, OTP, and optional 2FA. Since the system handles wallet and transaction-related data, authentication must protect not only login access but also all sensitive APIs. Any request related to groups, invoices, debts, payment requests, transfers, withdrawals, and transaction history should require a valid authenticated session.

JWT should be managed carefully to reduce the impact of token leakage. Tokens should be stored securely on the client side, and invalid or expired tokens should be removed when the backend returns an unauthorized response. For users with 2FA enabled, the system should not grant full access immediately after password verification. Access should only be completed after the second authentication step is passed.

4.9.5 Role-Based Access Control

SplitPal supports different roles inside a group, such as owner, admin, and normal member. This role structure is important because group finance is shared, but not every user should have the same authority. The owner should control high-level group settings, admins may manage selected operational tasks, and normal members should only interact with records that are relevant to their participation.

Role-based access must be enforced on the backend, not only hidden in the frontend interface. A user who is not a member of a group must not be able to access that group's invoices, debts, payment requests, or transaction records even if they know the resource ID. This prevents unauthorized access and protects the financial privacy of group members.

4.9.6 Financial Audit Log and Accountability

Auditability is a core requirement for SplitPal because the system needs to explain how each debt and balance change was created. The report's financial data lineage connects invoices, original debts, payment requests, transfers, transfer debt allocations, and transactions into a traceable chain. This allows the system to show where a debt came from, which transfer settled it, and how the wallet balance changed.

For this reason, every important financial action should create an audit record. The record should include the actor, action, amount, currency, timestamp, previous status, new status, and related reference ID. Transaction history should not be directly overwritten. If a correction is needed, the system should create a new adjustment, refund, or cancellation record instead of editing the original transaction silently.

4.9.7 Professional Responsibility

From a professional perspective, SplitPal must be designed as a trustworthy financial workflow system, not only a feature-complete prototype. The development team should test not only successful cases, but also risky cases such as duplicate payment callbacks, repeated withdrawal requests, invalid OTP attempts, unauthorized group access, concurrent payment request creation, and AI extraction errors.

Sensitive credentials such as JWT secrets, payment gateway keys, AI service keys, SMTP credentials, and Firebase credentials should be stored through environment variables instead of hard-coded source code. Logs should support debugging and audit checking, but they should not expose passwords, OTP codes, full tokens, or unnecessary financial details. This helps the system remain maintainable, secure, and responsible as the product grows.

5 Software design

5.1 Architecture

5.1.1 Overall Architecture

SplitPal is designed using a client-server architecture, where the Flutter mobile application acts as the client layer, the Node.js backend handles business logic and system coordination, and the database layer manages persistent data, caching, and short-lived locking. External services are integrated to support payment, notification, authentication, and exchange rate lookup.

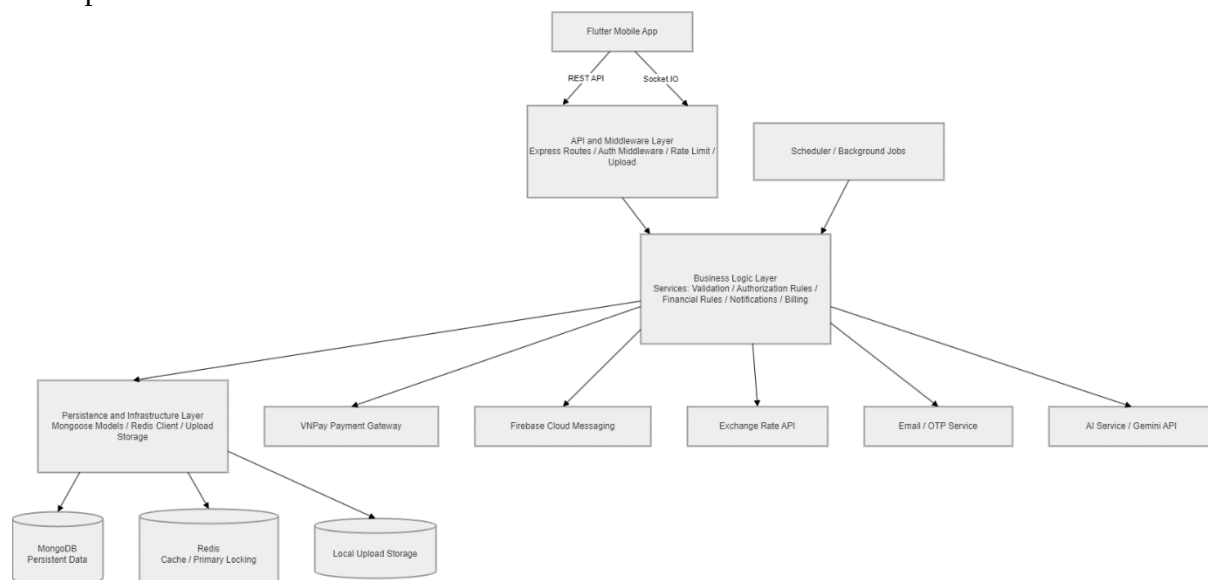


Figure 9 - Overall system architecture of SplitPal

This architecture is suitable for SplitPal because the mobile application should not directly access or modify financial data. Instead, all critical operations such as invoice splitting, debt generation, payment request creation, and balance updates are processed through the backend, where validation, authorization, and consistency controls can be enforced.

Layer	Technology	Main responsibility	Example features
Client Layer	Flutter, Dio, Socket.IO Client, Firebase Messaging	Provides the mobile user interface and handles user interaction, local session state, deep links, and device-level integrations.	Invoice creation, QR scanning, receipt upload, notification display, join-group deep link flow.
API & Realtime Layer	Node.js, Express, TypeScript, Socket.IO	Exposes REST endpoints, authenticates requests, applies middleware, and manages realtime communication between clients and the server.	Authentication routes, group APIs, chat events, online notification delivery, upload endpoints.
Business Logic Layer	TypeScript service modules	Centralizes business rules, validation, authorization decisions, and workflow coordination for financial operations.	Split calculation, payment request issuance, debt settlement, subscription billing, balance update, invoice locking.
Data Layer	MongoDB, Mongoose	Stores persistent business data and enforces schema-level structure for domain entities.	Users, groups, invoices, debts, transfers, subscriptions, messages, receipts, notifications.
Cache & Control Layer	Redis	Supports short-lived caching and operational control for concurrency-sensitive or frequently accessed data.	Exchange rate cache, invoice/payment-request cache, payment request lock, OTP storage, rate limiting, login attempt guard.
External Service Layer	VNPay, Firebase Cloud Messaging, SMTP Email Service, Exchange Rate API, Gemini API	Provides third-party capabilities that extend the core system beyond internal processing.	Payment gateway, push notifications, email/OTP delivery, currency conversion, AI-assisted receipt extraction.
Background Job Layer	Node.js Scheduler	Executes periodic or deferred system tasks that should not depend on direct user interaction.	Subscription renewal processing, billing retries, scheduled maintenance tasks.

The system follows a layered architecture that separates mobile interaction, API handling, business processing, data persistence, caching, external integration, and background execution. This separation improves maintainability and extensibility because each layer has a clear and limited responsibility. For example, the Flutter client focuses on user experience and interaction flow, while the backend centralizes financial rules and validation so that the client cannot directly manipulate debts, transfers, or balance records. MongoDB persists the core domain data, Redis improves responsiveness and concurrency safety, and the external service layer provides payment, notification, email, exchange-rate, and AI capabilities required by the current implementation

5.1.2 Architectural rationale and Key decisions

Key decision	Trade off	Rationale	System benefit
Centralize financial logic and state-changing operations in the backend	The backend becomes more complex because it must handle financial calculations and state changes, instead of allowing the mobile app to calculate final split	SplitPal handles sensitive workflows such as bill splitting, debt creation, invoice locking, payment request issuance, settlement, balance updates, subscription billing, and transaction history. If these	Debts, balances, transfers, and payment requests are controlled centrally. This reduces calculation errors, limits unauthorized manipulation, and keeps

	results, debts, or balances by itself.	calculations or state changes are distributed to the client side, it becomes difficult to maintain consistency across multiple users, devices, and app versions.	financial states consistent across the whole system.
Use REST API, Socket.IO, and Firebase Cloud Messaging together instead of relying on a single communication mechanism	The communication architecture becomes more complex because the system must clearly separate request-response operations, real-time socket events, and background push notifications.	SplitPal contains structured business operations such as authentication, group creation, invoice creation, and payment request issuance. At the same time, it also requires real-time interactions such as group chat, in-app notifications, and status updates. When the app is closed or running in the background, important events still need to be delivered through Firebase Cloud Messaging.	REST API supports structured workflows that require complete request-response handling. Socket.IO enables real-time chat and in-app notification delivery. Firebase Cloud Messaging ensures that important events can still reach users even when the app is not actively open.
Use a document-oriented data model with MongoDB + Mongoose, combined with Redis for caching and locking	Compared with a traditional RDBMS approach, this choice requires more control at the service, transaction, and locking layers to protect sensitive financial workflows.	SplitPal's data is not limited to fixed financial records. It also includes receipts, notifications, chat messages, billing history, and aggregates such as invoices with items, debts, and transfers. MongoDB provides flexibility for an evolving domain, while Mongoose adds schema validation, indexing, and clearer model organization. Redis is added to support short-lived caching and locks for sensitive operations.	The system remains flexible for semi-structured modules while still controlling important operations such as payment request issuance, balance-affecting actions, and exchange-rate handling. Redis also improves performance through caching and reduces the risk of duplicate processing in concurrent workflows.
Route external service integrations through the backend instead of allowing the mobile app to call third-party services directly	The backend takes on more responsibility for integration, authentication, error handling, retry logic, and response normalization, which increases server-side complexity.	SplitPal integrates with several external services such as VNPay, Firebase Cloud Messaging, Exchange Rate API, SMTP Email Service, and Gemini API. If the mobile app directly calls these services, secrets are harder to protect, errors are harder to control, logs become fragmented, and external responses may not match SplitPal's domain logic.	The backend becomes the central control point for external integrations. This improves security, auditability, error handling, and ensures that third-party responses are transformed into a consistent format before being returned to the client or written into the system.
Adopt a client-server architecture instead of a client-heavy architecture	The system requires a backend layer to coordinate data access and business logic, which increases server-side responsibility and deployment complexity.	The system requires a backend layer to coordinate data access and business logic, which increases server-side responsibility and deployment complexity.	The system requires a backend layer to coordinate data access and business logic, which increases server-side responsibility and deployment complexity.

5.1.3 API, Realtime and Notification communication design

SplitPal uses different communication mechanisms for different types of system interaction. REST API is used for structured request-response operations such as authentication, group management, invoice creation, payment request generation, wallet operations, receipt upload, and subscription management. In these flows, the Flutter client sends HTTP requests through

Dio, while the backend validates JWT tokens through middleware, executes the required business logic in controller and service layers, and returns JSON responses to the client.

For realtime interaction, Socket.IO is used to support group chat, online notification delivery, and immediate status updates. This mechanism is more suitable than repeated API polling because chat messages and notification events need to be delivered as soon as they occur. In addition, Firebase Cloud Messaging is used for push notifications when the application is closed, running in the background, or not actively connected to the socket server.

Communication mechanism	Main purpose	Example in SplitPal
REST API	Structured request-response workflows	Login, create group, create invoice, create payment request, wallet top-up
Socket.IO	Realtime connected-user events	Group chat, in-app notification, live status update
Firebase Cloud Messaging	Push notification outside active app session	Payment reminder, subscription reminder, important system alert
External callback	Third-party result notification	VNPay return/IPN callback

5.1.4 Security and Authorization Architecture

5.1.4.1. Security design principle

Security in SplitPal is designed around one main architectural decision: the backend must be the final authority for identity, authorization, and financial state changes.

This decision is necessary because SplitPal handles not only user accounts, but also group financial records, wallet balances, debts, payment requests, withdrawals, and VNPay callbacks. These are sensitive operations that should not be trusted directly from the Flutter client.

5.1.4.2. JWT-based authentication

SplitPal uses JWT authentication because the mobile client needs to make repeated authenticated API requests without requiring the user to log in again for every action. After a successful login, the backend issues a JWT that represents the authenticated user. On the Flutter side, this token is stored through TokenManager using FlutterSecureStorage.

For later requests, the Dio networking layer automatically attaches the token to the request header using the Authorization: Bearer <token> format.

This implementation reduces duplicated authentication logic across Flutter screens. Instead of each screen manually handling authentication, the token attachment is centralized in the networking layer.

The trade-off of JWT is that the system must carefully handle expired or invalid tokens. SplitPal addresses this by verifying the Bearer token in authMiddleware before protected routes can continue.

If the backend returns 401 Unauthorized, the Flutter client clears local authentication data. This prevents an expired or invalid session from being silently reused.

5.1.4.3. Authentication flow

Login or registration starts from the Flutter application.

The backend authentication service verifies the submitted credentials.

If the credentials are valid, the backend checks whether 2FA is required.

If 2FA is not enabled, the backend issues a JWT directly.

If 2FA is enabled, the backend first issues a temporary token and only issues a full JWT after TOTP or backup code verification.

The Flutter client stores the JWT securely through TokenManager.

The Dio interceptor attaches the JWT to protected API requests.

The backend authMiddleware verifies the JWT before allowing the request to reach protected business logic.

5.1.4.4. Optional two-factor authentication

SplitPal supports optional two-factor authentication because the system includes wallet and payment-related features.

When 2FA is enabled, password verification alone does not immediately create a full authenticated session.

Instead, the backend issues a temporary token and requires the user to verify a TOTP code or backup code before completing login.

Backup codes are stored as hashed values rather than plain text.

The trade-off is that the login flow becomes more complex. However, this gives users stronger protection if they use SplitPal as a financial management tool rather than only a simple expense-splitting application.

5.1.4.5. Rate limiting

Rate limiting is used to reduce abuse against sensitive endpoints.

This is especially important for login attempts, OTP resend requests, OTP verification, transfer OTP, and withdrawal OTP flows.

Without rate limiting, attackers could repeatedly try passwords or OTP codes.

SplitPal uses Redis-based rate limiting when Redis is available. If Redis is not connected, the system falls back to an in-memory store.

This is a practical design choice for an academic prototype because it allows the system to run in both local development and deployed environments.

However, the limitation is that in-memory rate limiting is not reliable across multiple server instances. Therefore, Redis should be required in a production deployment.

5.1.5 External Service Integration Architecture

External services are integrated through backend-controlled service modules rather than being called directly from the Flutter client. This design protects sensitive credentials, keeps third-party response handling consistent, and allows the backend to translate external service responses into SplitPal’s own domain logic. For example, VNPay callbacks must be verified before any wallet balance or transaction status is updated. Similarly, Gemini AI extraction results should be treated as assistive data and reviewed by the user before they are converted into financial records.

External service	Role in SplitPal	Backend responsibility	Risk control
VNPay	Wallet top-up and payment callback	Create payment URL, verify return/IPN, update transaction	Secure hash verification
Firebase Cloud Messaging	Push notifications	Send notification payload to devices	Store notification in database if push delivery fails
SMTP/Nodemailer	OTP and email delivery	Send verification and notification emails	Rate limit OTP and log delivery failure
Gemini AI	Invoice OCR/extraction	Extract text and suggested invoice fields	User review and fallback extraction
Exchange Rate API	Currency conversion	Fetch and cache exchange rates	Cache rates and handle unavailable API

5.1.6 Background Processing and Operational Architecture

Background processing is required because several SplitPal workflows should not depend on direct user interaction. Subscription renewals, recurring bill generation, billing retries, payment request reminders, and expiration handling need to run periodically even when users are not actively using the application. Therefore, the backend includes scheduled processing to automate these recurring financial operations and reduce the amount of manual coordination required from users.

Background process	Purpose	Related data
--------------------	---------	--------------

Subscription renewal processing	Automatically check and process renewal cycles	subscriptions, billing histories, transactions
Recurring bill generation	Generate draft bills from templates	bill templates, invoices
Payment request reminders	Notify users about pending payments	payment requests, notifications
Expiration handling	Update overdue/expired payment requests	payment requests, transfers
Billing retry	Retry failed subscription-related operations	billing histories, transactions

5.1.7 Architecture Limitations

Although the architecture provides a clear separation between the Flutter client, the Express backend, the MongoDB persistence layer, the Redis cache/control layer, external integrations, and scheduled background processing, several limitations remain. First, the backend acts as the main authority for authentication, financial validation, debt generation, payment processing, notification triggering, and third-party service coordination. This centralization improves consistency and traceability, but it also concentrates complexity in a relatively small number of service modules, making the backend harder to maintain, test, and evolve over time. Second, the system depends on a broad set of supporting technologies, including MongoDB, Redis, VNPay, Firebase Cloud Messaging, Gemini AI, SMTP email delivery, and exchange-rate APIs. While this increases functional scope, it also raises the cost of configuration, local setup, debugging, and operational troubleshooting.

A further limitation is that some operational workflows are still implemented in a relatively lightweight way for an academic project. Background tasks such as subscription renewals, recurring bill generation, reminders, and expiration handling currently rely on an in-process scheduler rather than a dedicated job queue. This is acceptable for a prototype, but it is less resilient under failure, retry, and scaling scenarios than a queue-based background processing architecture. In a production environment, the system would also require stronger monitoring and observability. Redis connection loss, failed payment callbacks, notification delivery failures, scheduler interruptions, and external API degradation could otherwise occur without sufficiently visible alerts. For this reason, centralized logging, health monitoring, metrics collection, and alerting should be considered necessary architectural improvements rather than optional enhancements.

Finally, the current architecture still has a testing gap. The codebase demonstrates strong business rules for financial flows such as invoice submission, debt creation, transfer allocation, refunds, and recurring billing, but these flows require more comprehensive integration and end-to-end tests to verify correctness across multiple collections, transactions, and external-service boundaries. Without this level of automated verification, regression risk remains significant as the system grows. Therefore, although the current architecture is suitable for demonstrating a broad and technically meaningful full-stack solution, it still requires stronger queueing support, centralized logging, monitoring, integration-test coverage, and security hardening before it can be considered production-ready.

5.2 GUI

5.2.1 GUI Design Purpose

The GUI of SplitPal is designed to support group financial activities in a clear, controlled, and traceable way. Since the system handles multi-currency invoices, multiple splitting methods, payment requests, wallet transfers, VNPay top-ups, OTP-based withdrawals, personal subscriptions, and transaction history, the interface must help users understand the financial impact of each action before confirming it.

Instead of placing many functions on a single screen, SplitPal organises the interface around task-based workflows. Key actions such as creating a group, inviting members, creating an AI/OCR-supported invoice, checking debts, sending payment requests, transferring money, managing subscriptions, and reviewing transactions are separated into dedicated screens. This reduces confusion, especially in workflows that affect wallet balances or debt status.

Transparency is also a core design goal. Users need to know not only how much they owe, but also where the debt comes from, who paid first, whether the debt has been settled, which transfer completed the payment, and how each transaction changed their balance. Therefore, screens such as debt breakdown, payment request detail, and transaction history display the data source, processing status, final result, and balance changes where relevant.

5.2.2 Overall GUI Structure

SplitPal's frontend is developed with Flutter and follows a feature-based structure. Each major module is separated into its own folder, including state management providers, presentation pages/widgets, and related models. The main interface is organised through HomeShellPage, which acts as the primary container and provides five main navigation tabs: Home, Subs, Groups, Activity, and Settings.

GUI Area	Main Functions	Role in the System
Authentication Screens	Register, email/Google login, OTP verification, TOTP-based 2FA, password recovery	Protect account access and financial operations
Home Dashboard	Wallet balance, cashflow forecast, currency converter, receipt diary, active groups, upcoming subscriptions, open payment requests	Provide a quick overview of personal and group financial status
Group Management	Create groups, view group details, manage members, invite by email/join code, assign roles	Organise shared expenses by group and enforce role-based access
Invoice & Bill Screens	Create invoices manually or with AI/OCR, select currency, add items, choose participants, apply split methods, manage bill templates and draft invoices	Record expenses and generate original debt records
Payment & Transfer Screens	View payment request details, debt breakdown, related invoices, transfers, wallet/VNPay payment, cancellation	Manage the debt lifecycle from issued to paid or cancelled
Wallet Screens	VNPay top-up, NCB withdrawal with OTP/2FA, wallet balance, paginated transaction history	Manage personal cash flow inside the system
Subscription Screens	Create recurring subscriptions, invite members, view billing history, manage split percentage and status	Handle recurring personal or shared expenses

Receipt & Budget Screens	Create personal receipts, monthly calendar view, tags, budget envelopes, budget summary	Track daily personal spending
Notification & Chat	Read/unread notifications, entity-based navigation, real-time group chat via Socket.IO	Support event tracking and group communication
Profile & Settings	Update profile, change password, enable/disable 2FA, push notification settings, dark theme, onboarding settings	Support personalisation, security, and user control

This structure keeps the interface aligned with the system's business logic. Users do not need to remember every feature manually; instead, they can access functions through the main navigation bar or quick-access sections on the dashboard based on their current task.

5.2.3 Main GUI workflows and Prototype screen description

5.2.3.1 Authentication and Account security flow

The authentication flow is designed to balance usability and security. Users can sign in using email/password or Google OAuth. Newly registered users must verify their email through OTP before the account is activated. If two-factor authentication is enabled, users are redirected to the TOTP verification screen after a successful login attempt.

- Registration: AuthPage (Sign Up) → VerifyOTPPage → OnboardingPage → HomeShellPage.
- Normal login: AuthPage (Sign In) → OnboardingPage for first-time users only → HomeShellPage.
- Login with 2FA: AuthPage (Sign In) → Verify2FAPage → HomeShellPage.
- Forgot password: AuthPage → ForgotPasswordPage → Enter email → Verify OTP → Set new password.

Separating OTP and 2FA into dedicated screens, such as VerifyOTPPage, Verify2FAPage, and Setup2FAPage, makes the login process longer. However, this is appropriate for a system that supports wallet balance, top-up, withdrawal, and real money transfers. The separation helps users understand each security layer and reduces the risk of unauthorised access.



Figure 10 - Authentication and Account security flow

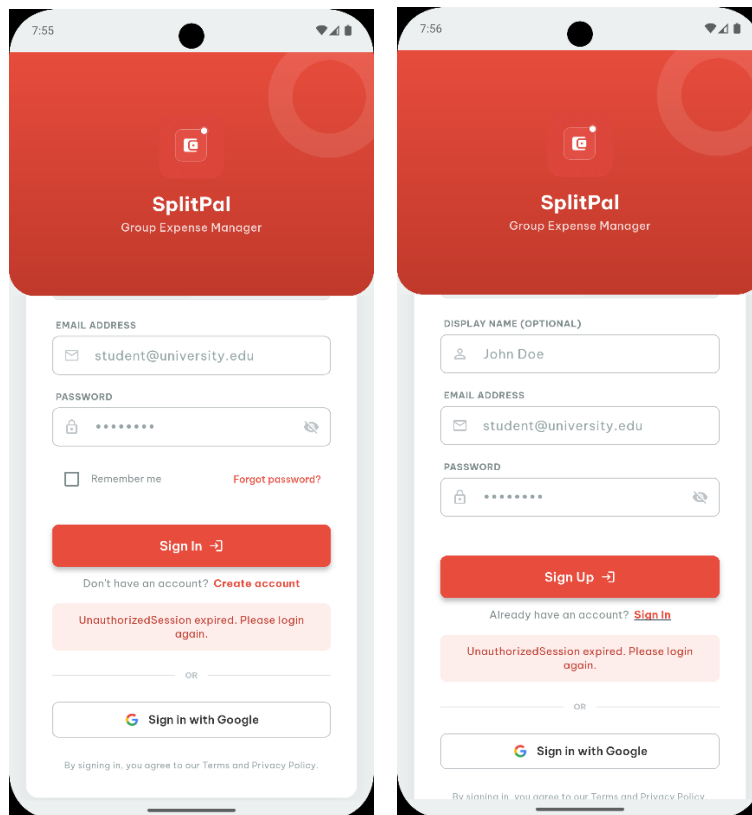


Figure 11 - Sign In/ Sign Up UI

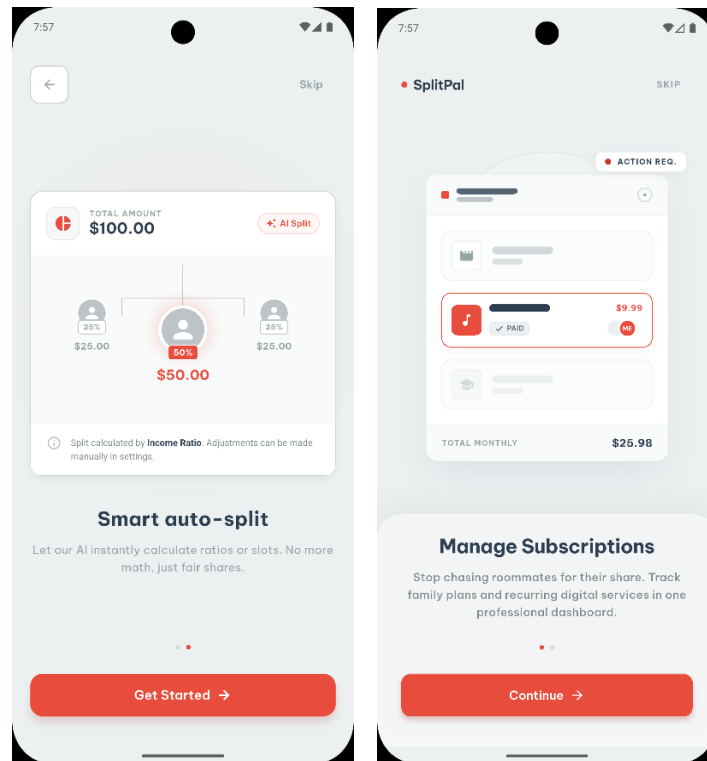


Figure 12 - Onboarding Page UI

5.2.3.2 Group Management flow

Groups are the main workspace for shared expense management. The group interface follows a tab-based structure with three tabs: Overview, Invoices, and Payments. This allows users to quickly access members, invoices, debts, and payment requests within the same group context.

Role-based access is reflected directly in the interface. Only users with OWNER or ADMIN roles can see actions such as creating invoices, inviting members, creating payment requests, or updating member roles.

- Basic flow: HomeShellPage (Groups tab) → GroupsPage → GroupDetailPage.
- From the Overview tab, users can view members and roles, check debt summaries, create invoices, invite members by email or join code, create payment requests, and open group chat.
- From the Invoices tab, users can view submitted or locked invoices, create new invoices, manage bill templates, and edit draft invoices.
- From the Payments tab, users can view payment requests, inspect debt breakdowns, and perform transfers.
- Invite member: GroupDetailPage → InviteMembersBottomSheet → Enter email or share join code.
- Join group: AcceptInvitePage from invitation link or JoinGroupPage using join code.

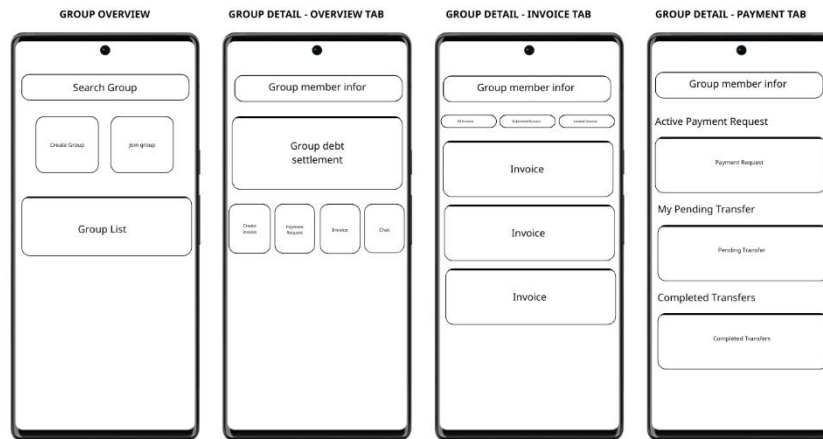


Figure 13 - Group Management flow

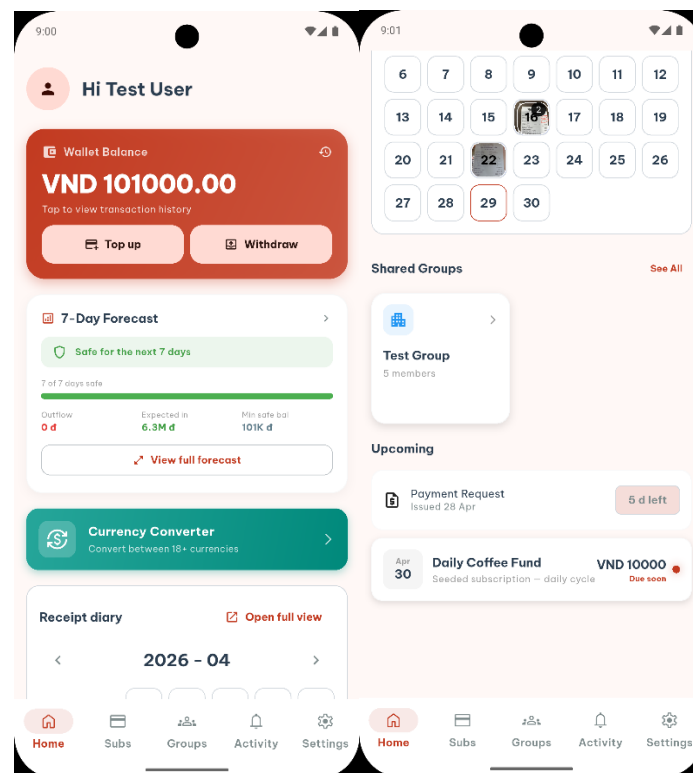


Figure 14 - Homepage UI

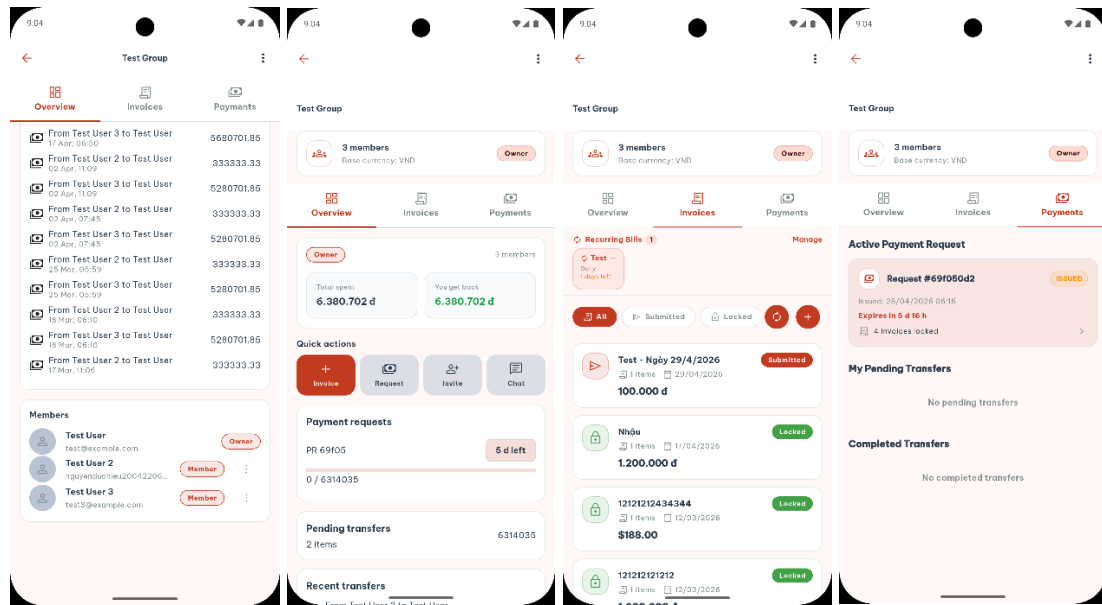


Figure 15 - Group detailed tabs UI

5.2.3.3 Invoice creation and Expense splitting flow

This is one of the most important GUI flows because it creates the original financial records of the system. In SplitPal, an invoice can contain multiple items, multiple participants, three splitting methods - EQUAL, PERCENTAGE, and CUSTOM - and 19 supported currencies with the exchange rate locked at creation time.

The system also integrates AI/OCR to reduce manual input. Gemini Vision is used for receipt images, while Gemini Text is used for text-based invoice extraction. The extracted data can be used to auto-fill invoice fields, including Vietnamese member name matching.

- Manual flow: GroupDetailPage → CreateInvoicePage → Enter title and note → Select currency → Add items → Assign members → Choose split type → Review total → Save.
- AI/OCR-supported flow: GroupDetailPage → CreateInvoicePage → Camera button → InvoiceOcrPicker → Capture or select receipt image → Gemini Vision extraction → Auto-fill form → Review and edit → Save.
- Recurring invoice flow: GroupDetailPage (Invoices tab) → CreateBillTemplatePage → Configure template → Scheduler creates draft invoice → EditDraftInvoicePage → Confirm → Generate debts.

Supporting multiple split methods and currencies increases the number of fields in the interface. However, AI/OCR auto-fill reduces manual effort and makes invoice creation faster, especially when the invoice contains many items.

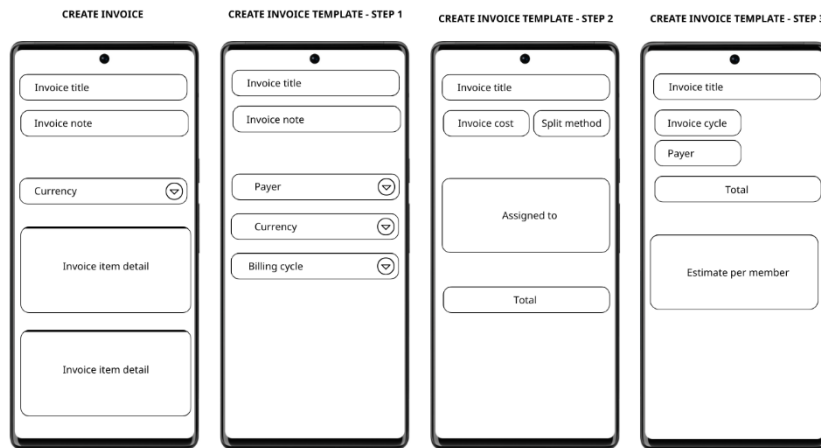


Figure 16 - Invoice creation and Expense splitting flow

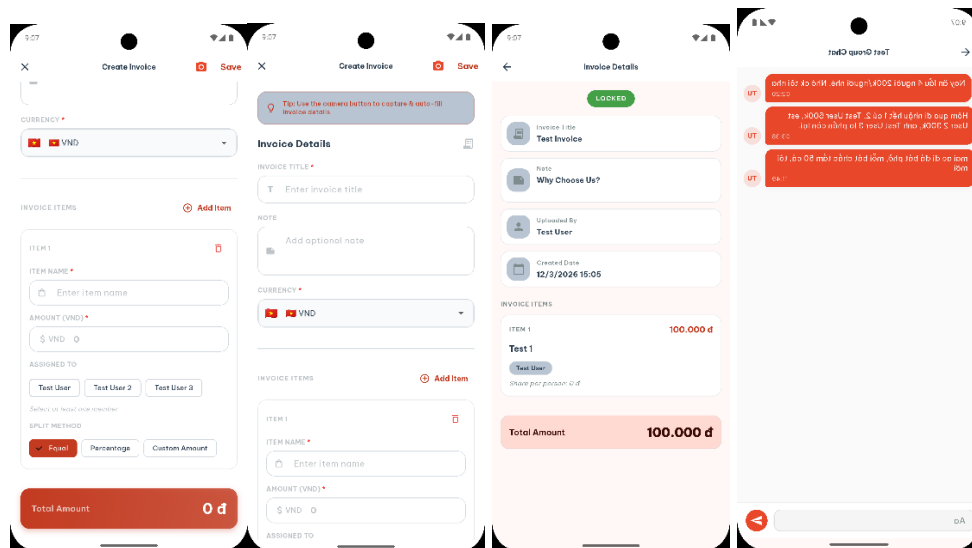


Figure 17 - Invoice Creation

5.2.3.4 Wallet and Transaction History Flow

The wallet interface directly affects the user's balance, so it requires clear confirmation and traceability. The top-up flow integrates with VNPay by opening an external browser. When users return to the app, the system automatically polls for the updated balance.

The withdrawal flow requires two-step verification. Users submit a withdrawal request, provide a 2FA code if enabled, and then confirm the operation with an email OTP.

- Top-up flow: Dashboard Wallet Card → Top Up → WalletOperationsPage → Enter amount → Generate VNPay link → Open external browser → Complete payment → Return to app → Auto-poll balance update → Refresh wallet balance.

- Withdrawal flow: Dashboard Wallet Card → Withdraw → WalletOperationsPage → View NCB bank details → Enter amount and 2FA code if enabled → Request withdrawal OTP → Enter email OTP → Verify → Update balance.
- Transaction history flow: Dashboard Wallet Card → TransactionHistoryPage → Paginated transaction list.

Each transaction displays the transaction type, amount, description, timestamp, and balanceAfter. This supports traceability and helps users verify how their wallet balance changed over time.

The main trade-off is that top-up and withdrawal require multiple steps, including VNPay redirect, OTP email, and TOTP verification. However, this is necessary because wallet actions directly affect user funds.

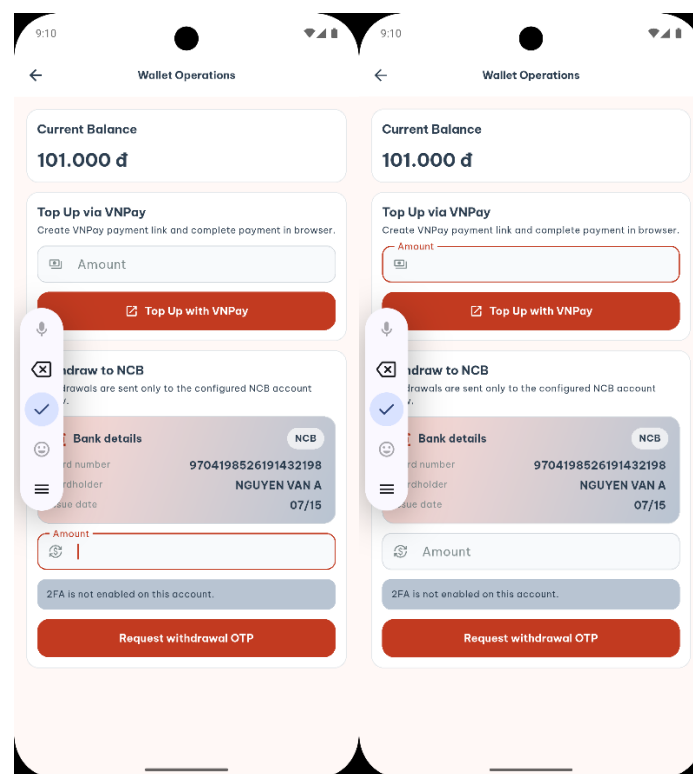


Figure 18 - Top Up & Withdrawal balance ui

5.2.3.5 Notification and Status Tracking Flow

SplitPal supports real-time in-app notifications through Socket.IO and push notifications through Firebase Cloud Messaging. Notifications are visually categorised by icon and colour, such as expense, invoice, payment, invite, group joined, and balance updated.

When a user taps a notification, the system marks it as read and navigates to the related screen based on the notification type.

- Basic flow: System Event → Push Notification via FCM + In-App Notification via Socket.IO → NotificationsPage → Tap notification → Mark as read → Navigate to related detail screen.

- Navigation mapping examples: balance-related notification → TransactionHistoryPage; invite notification → AcceptInvitePage; group-related notification → GroupDetailPage.

The Activity tab also displays an unread badge count, allowing users to notice pending events without opening the notification page manually.

Batch actions include Mark all as read and Delete read notifications.

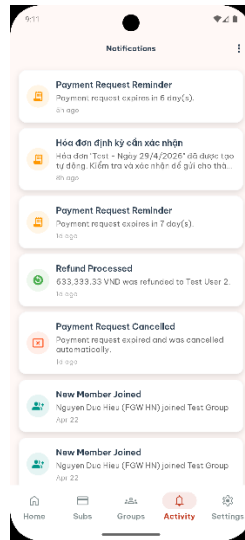


Figure 19 - In app Notification

5.2.3.6 Dashboard Overview

The dashboard is the first screen after login and summarises the most important financial information in one scrollable interface. It refreshes automatically when users navigate back to it by using the RouteAware pattern. A dedicated dashboard API is used to fetch aggregated data in one request, reducing the need for multiple separate API calls.

Component	Description
Header	Greeting message with user avatar
Wallet Balance Card	Displays wallet balance, Top Up / Withdraw quick actions, and opens Transaction History when tapped
Forecast Risk Card	Shows 7/14/30-day cashflow forecast, balance trend, and low-balance risk
Currency Converter Card	Provides quick access to 18+ currency conversions
Receipt Diary	Monthly calendar view with receipt thumbnails, receipt count badge, and today highlight
Shared Groups	Horizontal group cards showing group name and member count
Upcoming Section	Shows open payment requests and upcoming subscriptions

The dashboard is designed as a high-level financial summary rather than a detailed management screen. It helps users quickly identify what requires attention, such as low wallet balance, pending payment requests, or upcoming subscription billing.

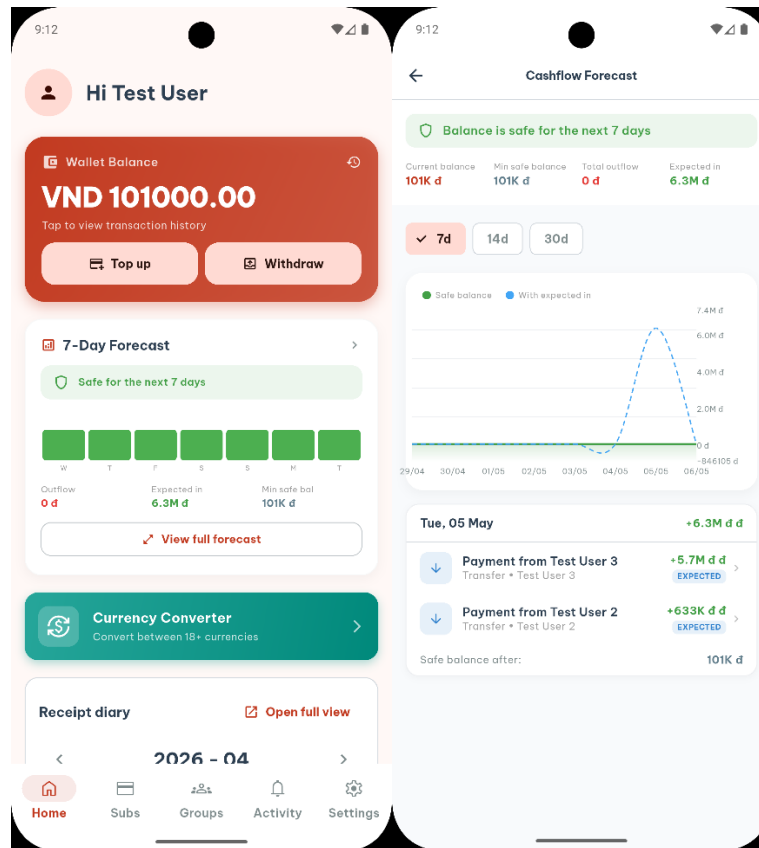


Figure 20 - Dashboard and Forecast UI

5.2.3.7 Profile and Account Settings Flow

The Profile screen allows users to manage their account, security, and application preferences. The interface is organised into grouped setting cards, where each item is presented as a row with an icon, title, and trailing action.

- Basic flow: HomeShellPage (Settings tab) → ProfilePage.
- Account settings include Edit Profile → EditProfilePage, Change Password → ChangePasswordPage, and Two-Factor Auth → Setup2FAPage or Disable 2FA dialog with TOTP verification.
- Application preferences include Push notification toggle, Dark theme toggle, and Show onboarding on login toggle.
- Support and logout flows include Contact Us → Dialog with subject and message, and Logout → Confirmation dialog → Remove FCM token → Clear session → Return to AuthPage.

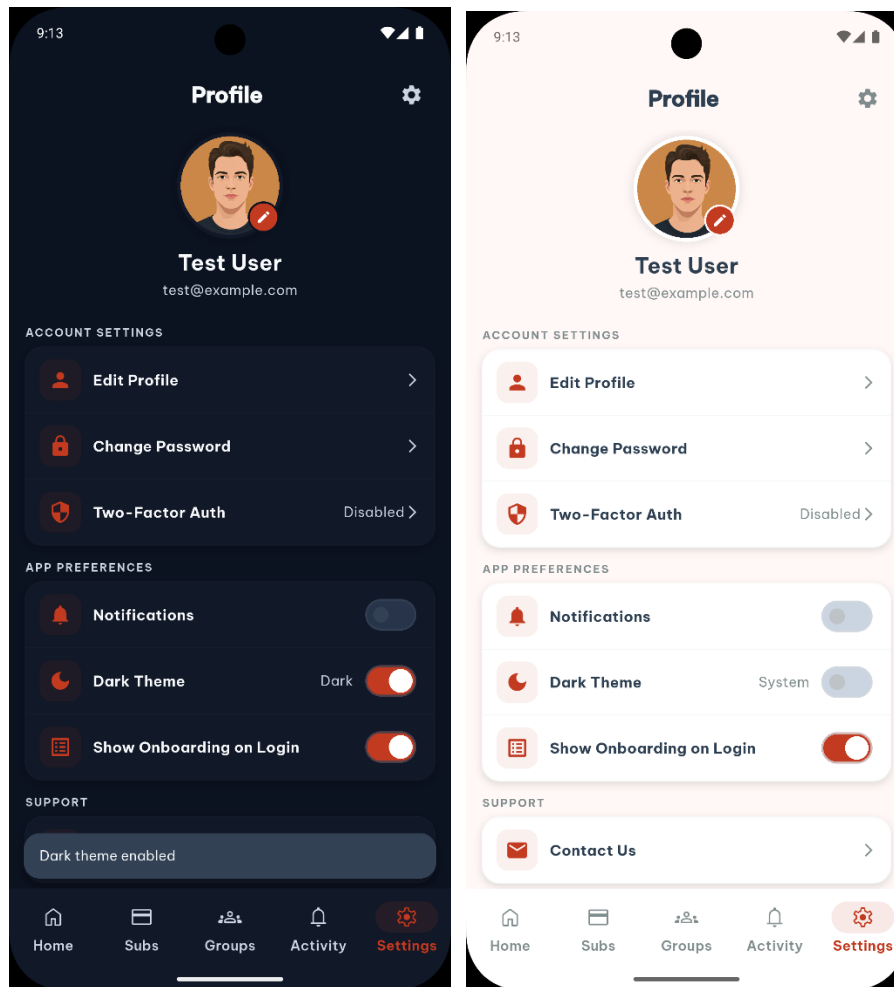


Figure 21 - Setting & Profile UI

5.2.4 GUI Design Decisions and Trade-offs

Design Decision	Rationale for SplitPal	Trade-off	System Impact
Organising GroupDetailPage into Overview / Invoices / Payments tabs	A group contains different but related data, including members, invoices, debts, payment requests, and chat	Users must switch tabs instead of seeing everything at once	Reduces information overload and keeps actions within the group context
AI/OCR-supported invoice creation with Gemini Vision	Manually entering many items, assignees, and split rules is time-consuming	AI extraction may be inaccurate and requires manual review	Speeds up invoice creation, but users must verify before saving
Multi-step wallet operations with VNPay redirect, email OTP, and TOTP	Wallet actions directly affect user balance and require strong protection	Top-up and withdrawal become longer workflows	Reduces accidental or fraudulent operations
Clear status display for payment requests and transfers	Users need to understand whether a request is issued, partially paid, paid, or cancelled	Backend must keep financial status consistent and updated	Improves transparency and allows users to track payment progress
Transaction history showing balanceAfter	Users need to trace every wallet balance change	Requires complete transaction logs with balanceBefore and balanceAfter	Increases trust and supports dispute checking

Design Decision	Rationale for SplitPal	Trade-off	System Impact
Dashboard with multiple financial widgets	Users need a quick overview without opening each module separately	The dashboard may become crowded	Requires careful data prioritisation and a dedicated dashboard API
Role-based UI visibility for OWNER, ADMIN, and USER	Users should only see actions they are allowed to perform	Frontend and backend permission logic must remain consistent	Improves UX while backend still enforces access control
Notification-to-screen navigation	Users can respond quickly to events such as invites, payments, or group updates	Notification types must be mapped correctly to related entities and screens	Reduces time spent searching for relevant information

These decisions show that SplitPal’s GUI is not designed only for visual presentation. Each interface decision is connected to a business requirement, a financial risk, or a real usability concern.

5.2.4 Current GUI Limitations

Although the current GUI supports the main workflows of SplitPal, several limitations remain.

First, financial flows such as multi-currency invoice creation, payment request generation, and withdrawal may still feel lengthy for new users. In particular, `CreateInvoicePage` contains many input fields, including title, note, currency, multiple items, assignees, and split type for each item. AI/OCR reduces manual input, but the overall flow can still be complex at first use.

Second, the dashboard currently combines many widgets in a single scrollable screen, including wallet balance, forecast, currency converter, receipt diary, shared groups, and upcoming items. When users join many groups or manage many subscriptions, the Shared Groups and Upcoming sections may become long. The current version does not yet support widget personalisation, reordering, or hiding.

Third, the current invoice flow does not include a final review screen before saving. After entering items, assignees, and split rules, users save the invoice directly. For complex invoices, this may cause users to notice mistakes only after debt records have already been created.

Fourth, some financial terms in the interface, such as debt allocation, payment request, transfer, settlement, and balanceAfter, have not yet been validated through real user testing. If users do not clearly understand the difference between a payment request and a transfer, they may perform the wrong action.

Fifth, the VNPay top-up flow depends on opening an external browser through `launchUrl` with `LaunchMode.externalApplication`. On some devices, switching between the app and browser may interrupt the user experience. The automatic polling mechanism after returning to the app reduces this issue, but users may still need to refresh manually if the network is slow or the VNPay callback is delayed.

5.2.5 Future GUI Improvements

In future versions, the GUI of SplitPal can be improved in four main directions.

First, the system should add a final invoice review screen before submission. After users complete all items, assignees, and split rules, the interface should display a summary showing who owes whom, how much each person owes, which item each debt comes from, and the total amount for each member. Users should confirm this summary before the system creates debt records.

Second, the dashboard should support personalisation based on user behaviour. For example, users who frequently check debts should see debt summaries more prominently, while users who often use the wallet should see wallet balance and transaction history first. Allowing users to reorder or hide dashboard widgets would also reduce information overload.

Third, financial actions should include smarter contextual warnings. For example, the interface could warn users when an invoice amount is unusually high compared with previous records, when a withdrawal exceeds a certain threshold, or when a new payment request overlaps with an existing pending request.

Fourth, the onboarding experience should provide more practical guidance. The current OnboardingPage appears after the first login, but it does not yet explain complex flows such as AI/OCR invoice creation, split method selection, or the relationship between payment requests and transfers. Tooltips or coach marks for important actions such as the OCR camera button, split type selector, and Create Payment Request button would help new users understand the system faster.

5.3 DB

5.3.1 Database model overview

SplitPal's data model is organized around two main dimensions: group-based collaboration boundaries and the lifecycle of financial obligations. At the collaboration level, users, groups, and group_members define who participates in the system, which groups each user belongs to, and which users have permission to initiate or approve important actions. At the financial level, data is structured as a traceable chain from expense recording to payment completion, starting from invoices and continuing through original_debts, payment_requests, transfers, transfer_debt_allocations, and transactions.

Although SplitPal uses MongoDB, the data model does not embed all related information into one large document. This is because many entities have independent lifecycles and different query requirements. For example, invoice_items are created as part of an invoice, original_debts represent debt obligations after split calculation, payment_requests are created later when debts are finalized for payment, transfers are generated as settlement obligations, and transactions are recorded only when wallet balances actually change. Embedding all of these records inside one invoice document would make updates, partial payments, refunds, and historical tracing more difficult.

Separating these entities into their own collections increases the number of collections, but it gives the system stronger control over financial workflows. It allows the backend to update only the affected records during sensitive operations, design indexes around specific query patterns, use database transactions more clearly, and preserve an audit trail for each stage of the payment lifecycle. This design is especially suitable for SplitPal because the system needs to explain not only the final balance, but also how that balance was produced.

5.3.2 Core collections

To understand SplitPal’s database design, the reader does not need to go through the full schema of every module. Instead, the core collections below are sufficient to explain how the system operates, especially in relation to group collaboration, expense recording, debt generation, payment requests, settlement, and wallet auditing.

Collection	Design Role	Purpose
users	Stores account identity and user state	Represents each participant in the system. It also maintains wallet balance, currency preferences, 2FA settings, and push notification configuration.
groups	Defines the collaboration boundary and financial context	Each group represents a separate shared-expense space, with baseCurrency and timezone acting as the standard context for invoices and settlement.
group_members	Represents the relationship between users and groups	Defines which users belong to which groups and what role they hold within each group, such as OWNER, ADMIN, or USER.
invoices	Stores expense records at the invoice level	Records an expense submitted to a group, including its status such as SUBMITTED or LOCKED, creator, currency, and the linked payment request if one has been issued.
invoice_items	Moves splitting logic down to the item level	Allows a single invoice to be split using different rules such as EQUAL, PERCENTAGE, CUSTOM, or WEIGHT, instead of applying only one split rule to the total invoice amount.
original_debts	Acts as the source of truth for original debt obligations	Defines who owes whom, how much remains unpaid, and stores the locked exchange-rate context when an invoice uses a foreign currency.
payment_requests	Encapsulates the “debt finalization for payment” stage	Groups open invoices within a group into an official payment request, with statuses such as ISSUED, PARTIALLY_PAID, PAID, or CANCELLED.
transfers	Represents payment obligations to be executed	Stores each payment line between a payer and a receiver, including processing status, OTP confirmation, and payment-related information.
transfer_debt_allocations	Links transfers back to original debt records	Specifies which original_debts are covered by a particular transfer and how much of each debt is settled by that transfer.
transactions	Provides an audit trail for wallet balance changes	Records every important wallet mutation, such as top-up, withdrawal, transfer, refund, or adjustment, so that balance changes can be traced and verified.

5.3.3 Indexing, TTL, and Locking Support

In addition to storing business data, the database model also supports performance and operational safety. SplitPal contains several workflows that require frequent lookup by user, group, status, and time. For example, `group_members` must be queried quickly by `userId` to retrieve the groups a user belongs to, and by `groupId` with `userId` to verify membership and role permissions. Similarly, `original_debts` are frequently queried by `groupId`, `debtorId`, `creditorId`, and `remainingAmount` to calculate balances and settlement obligations. Transactions are queried by `userId` and `createdAt` to build wallet history, while notifications and receipts depend on `userId`, read status, and date-based queries.

The system also uses lifecycle-based data control through TTL indexing. The locks collection is the clearest example. Each lock record contains an `expiresAt` field, and MongoDB can automatically remove expired lock documents through a TTL index. This prevents old lock records from remaining forever after failures and reduces the risk of deadlock in financial operations.

Locking is important because several SplitPal operations can be triggered concurrently. For example, two admins might try to create a payment request for the same group at nearly the same time. To reduce this risk, the system uses a Redis-first locking strategy. When Redis is available, a short-lived distributed lock is acquired before the sensitive operation continues. If Redis is unavailable, the system falls back to MongoDB locks using the locks collection. This fallback combines `expiresAt` with a unique compound constraint on the lock identity, such as name and `groupId`, so only one valid lock can exist for the same protected resource at a time.

This design shows that the database is not only used for storing application records. It also supports query optimization, lifecycle cleanup, and concurrency control for financial workflows.

5.3.4 Key highlight: Financial Data lineage

The most important aspect of SplitPal's database design is that its financial collections do not exist as isolated records. Instead, they are connected into an end-to-end traceable chain:

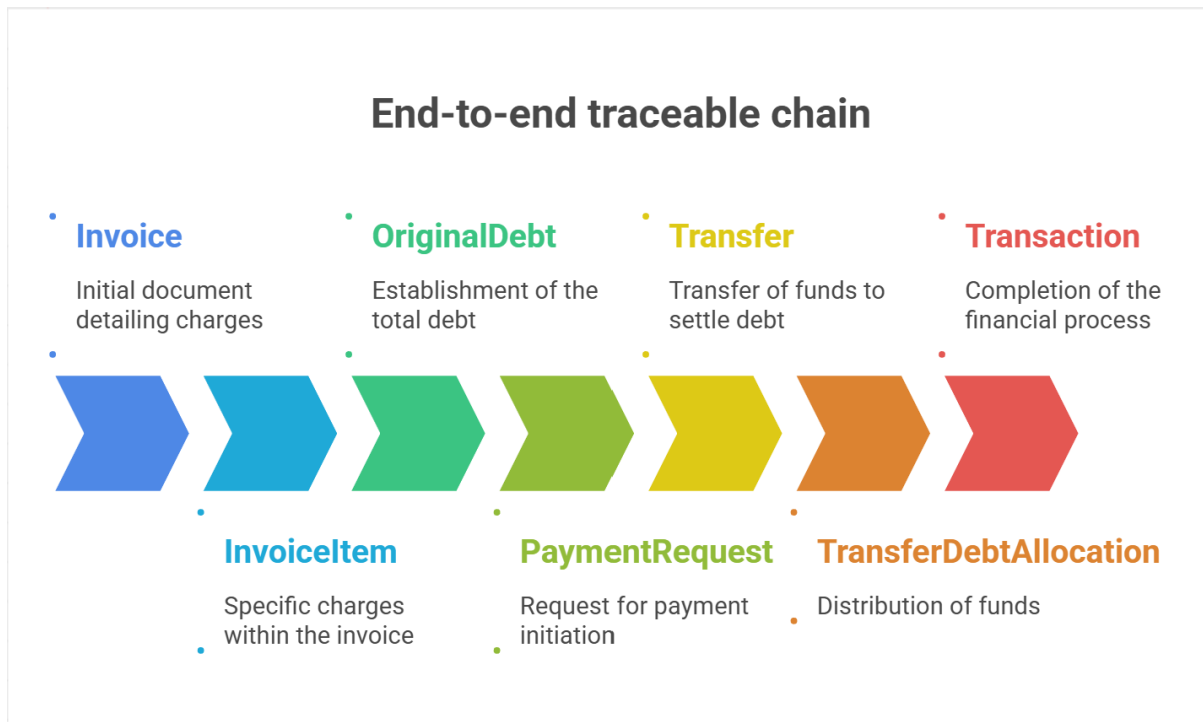


Figure 22 - End to end traceable chain

Within this chain, **OriginalDebt** is the key collection because it stores the original financial obligation after the system completes the split calculation from invoice items. Without **OriginalDebt**, the backend would need to recalculate debts from invoice data every time it performs a balance query or payment operation. That would make remaining balance control, adjustment handling, multi-currency processing, and payment status management much more difficult. In short, **Invoice** records the expense, while **OriginalDebt** answers the financial question: who owes whom, how much was owed, and how much remains unpaid.

This design is especially important for multi-currency invoices. When an invoice uses a currency different from the group base currency, the system stores `exchangeRateUsed` and `rateLockedAt` inside the debt context. This means historical debt values are not recalculated using future exchange rates. Once a debt is created, its converted amount remains tied to the rate at the time of creation. This protects historical accuracy and avoids disputes caused by exchange-rate fluctuations after the invoice has already been submitted.

TransferDebtAllocation plays a similar role in the later payment stage. A **Transfer** only shows that a payer is expected to send money to a receiver. However, it does not explain which exact debts are being settled by that payment. **TransferDebtAllocation** solves this by mapping each transfer to one or more **OriginalDebt** records and recording the allocated amount for each debt. As a result, the system can support partial settlement, refund handling, cancellation recovery, and detailed audit checking.

Recurring bill generation also benefits from this database design. When an invoice is generated from a bill template, `billingPeriod` identifies the cycle that the generated invoice

belongs to. This prevents the scheduler from accidentally creating duplicate draft invoices for the same template and same billing period. In other words, billingPeriod supports idempotency in recurring background workflows.

Overall, this database structure makes financial traceability clear in both directions. Starting from an invoice, the system can trace forward to the related debts, payment request, transfers, allocations, and wallet transactions. Conversely, starting from a transaction or transfer, the system can trace backward to the exact debt and invoice that created the obligation. For example, if Bob pays Alice 320,000 VND, the database can show that 200,000 VND settled one original debt from the first invoice and 120,000 VND settled another debt from the second invoice. This end-to-end lineage is the strongest part of SplitPal's database design..

5.4 UML

5.4.1 Use Case Diagram

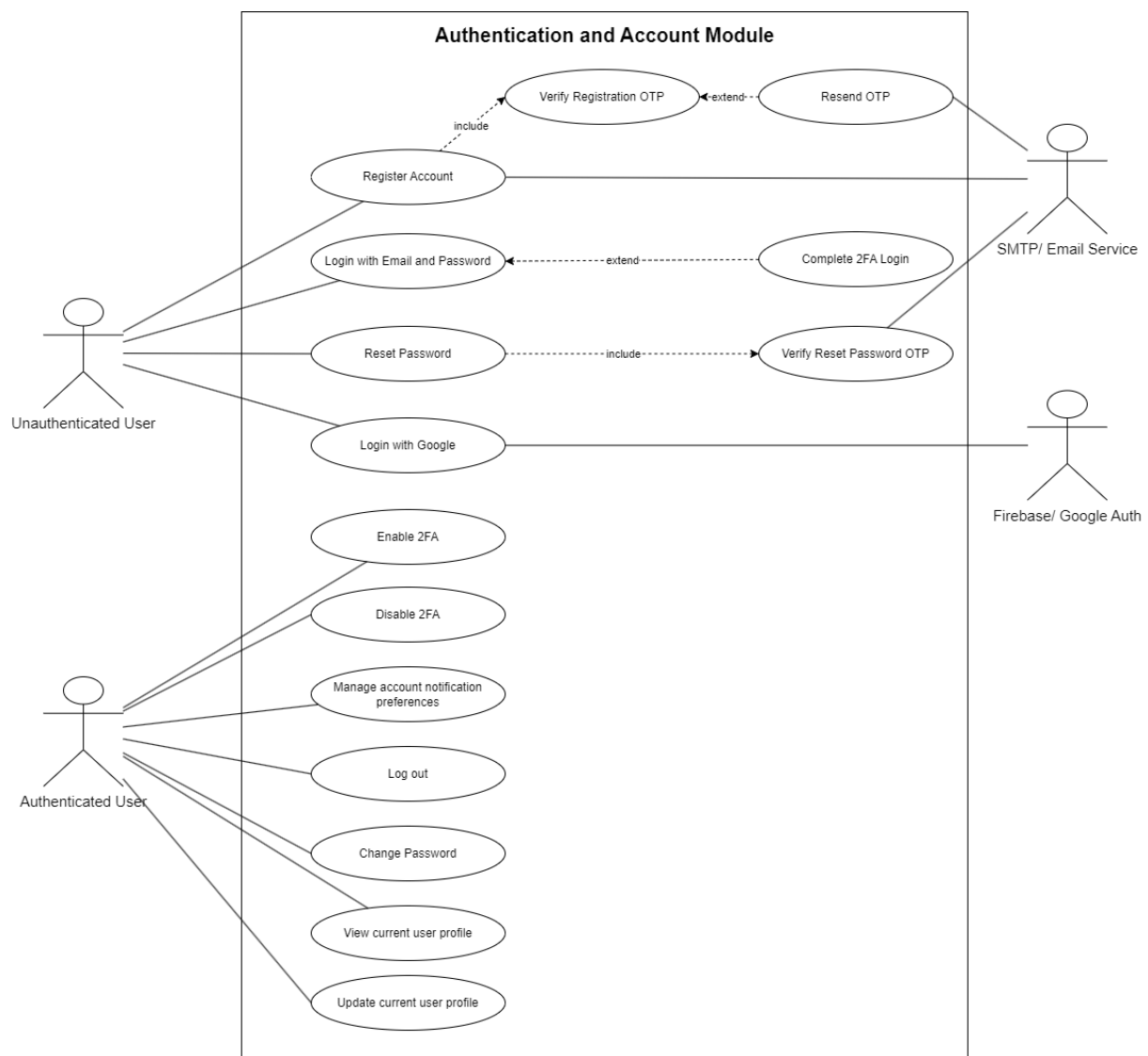


Figure 23 - Authentication and Account Module

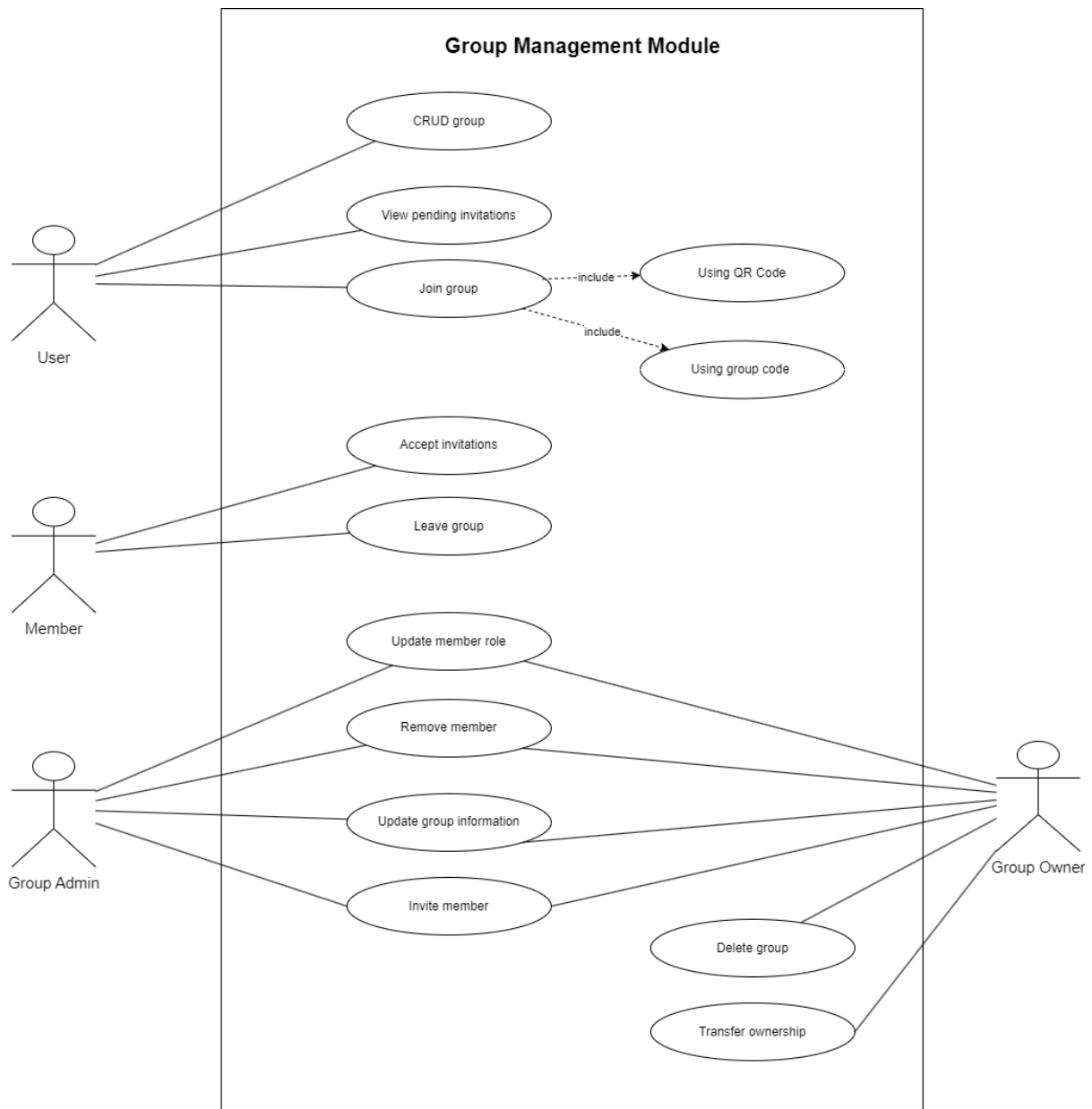


Figure 24 - Group Management Module

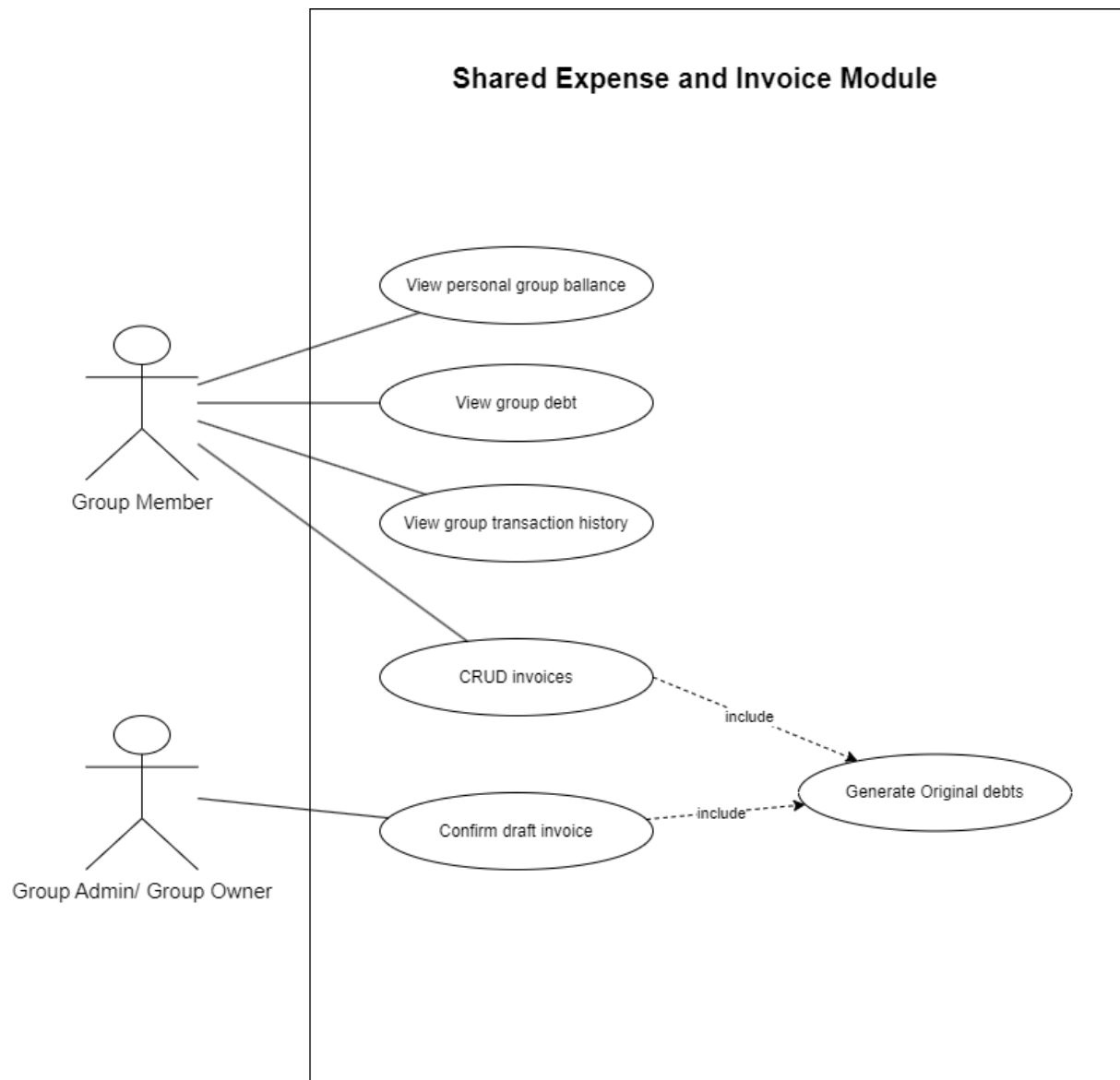


Figure 25 - Share Expense and Invoice Module

5.4.2 Class Diagram

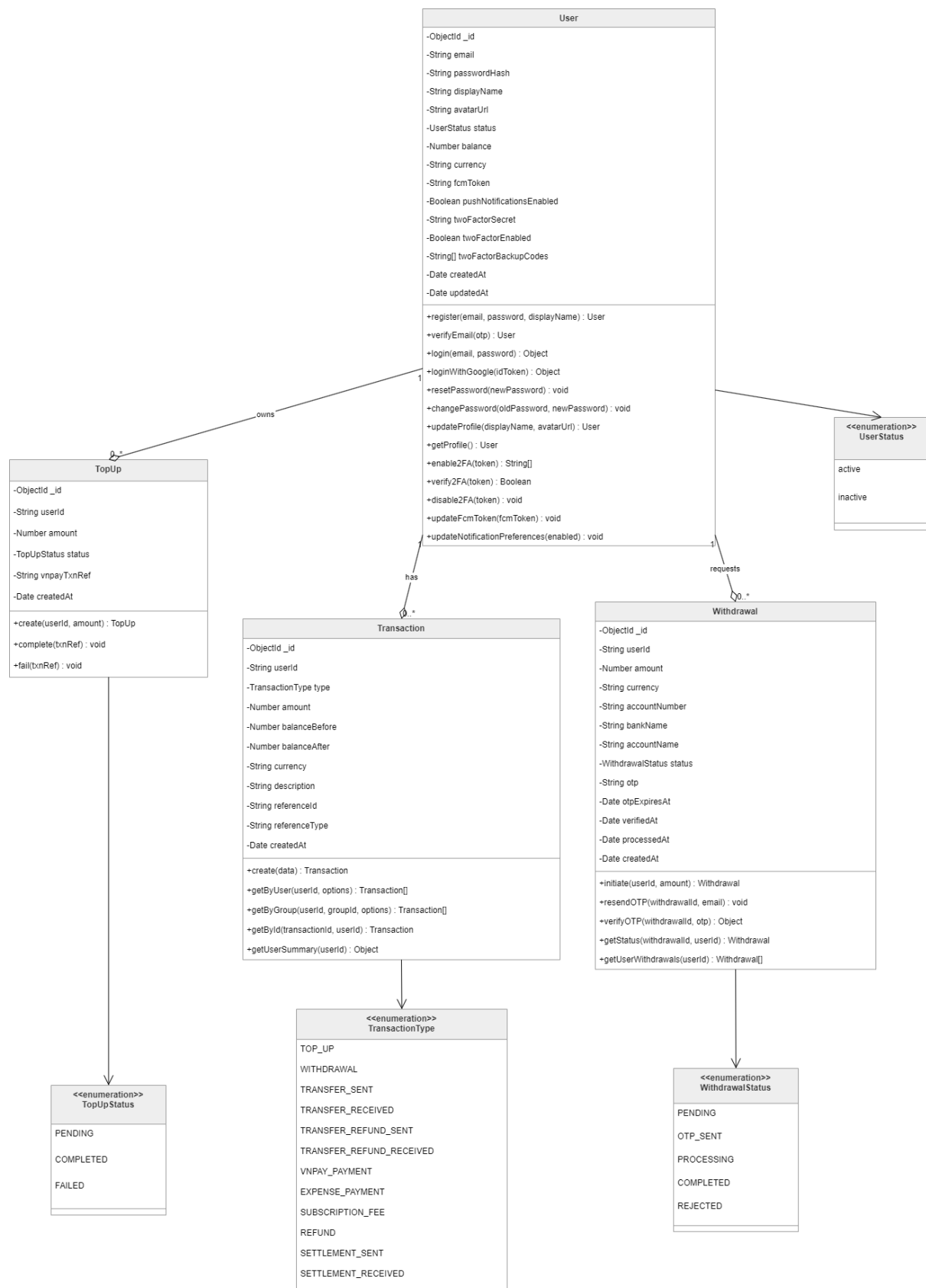


Figure 26 - Authentication, Account & Wallet Class Diagram

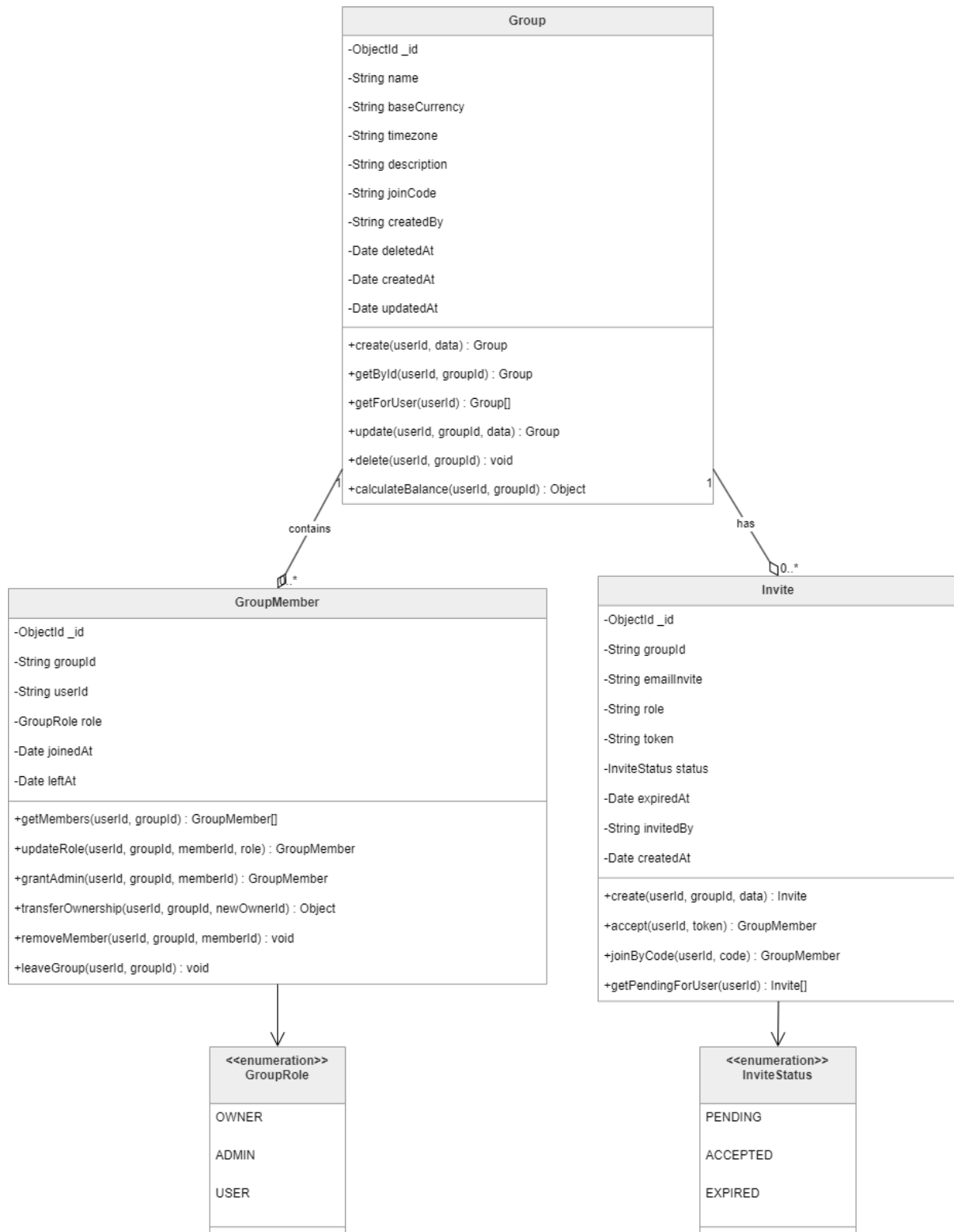


Figure 27 - Group Management Class Diagram



Figure 28 - Shared Expense & Debt Settlement Class Diagram



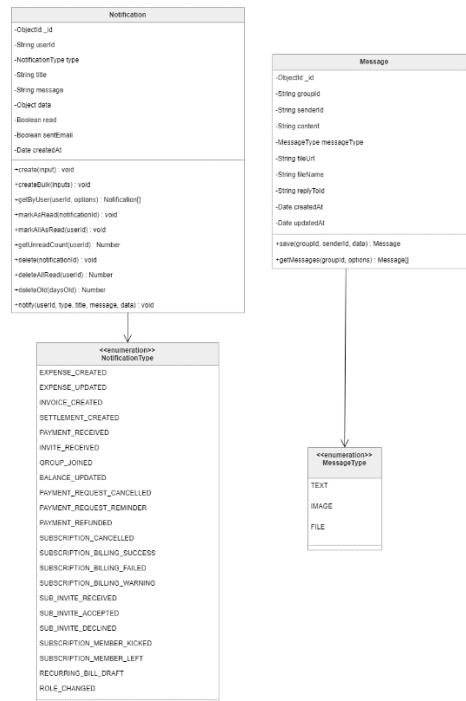


Figure 31 - Notification & Communication Class Diagram

5.4.3 Sequence Diagram

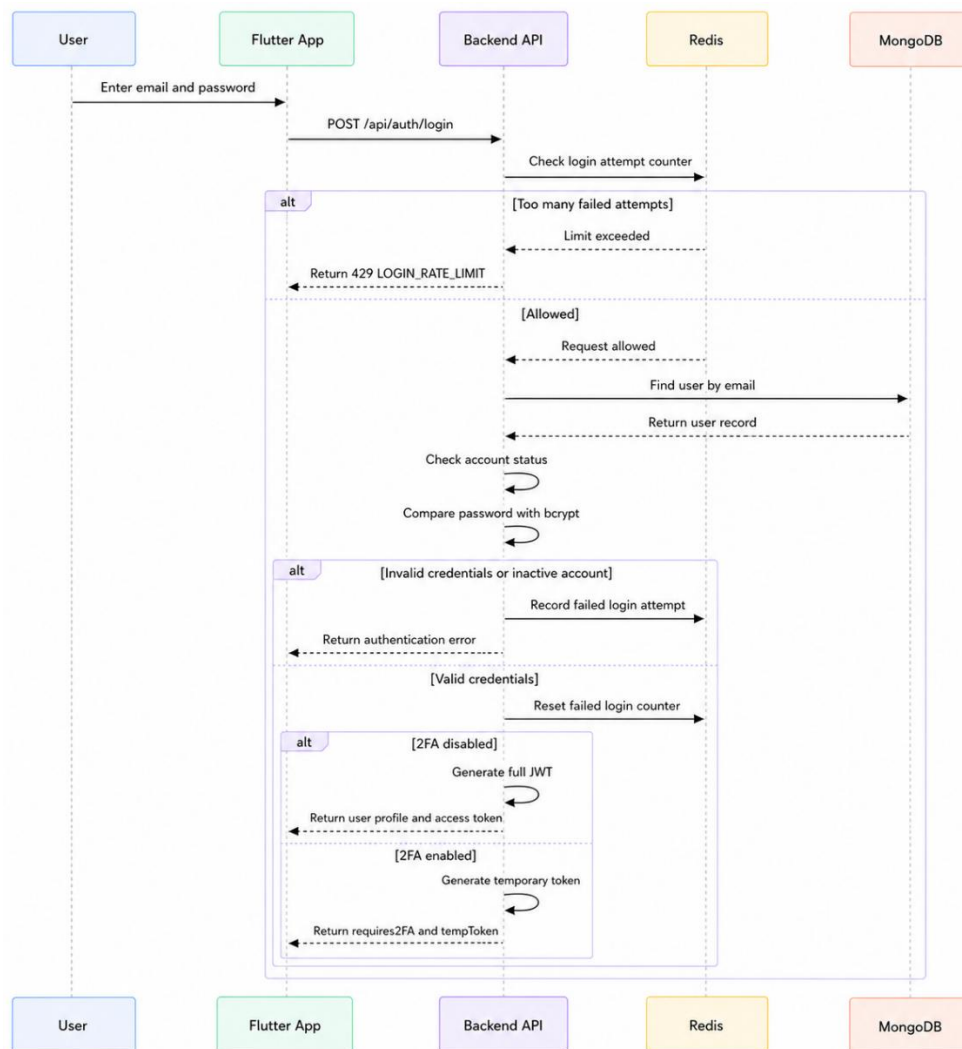


Figure 32 - Login sequences diagram

This sequence describes the initial login flow. The backend first checks the Redis-based login guard to prevent repeated failed attempts. If the credentials are valid, the system either returns a full access token directly or returns a temporary token when two-factor authentication is enabled.

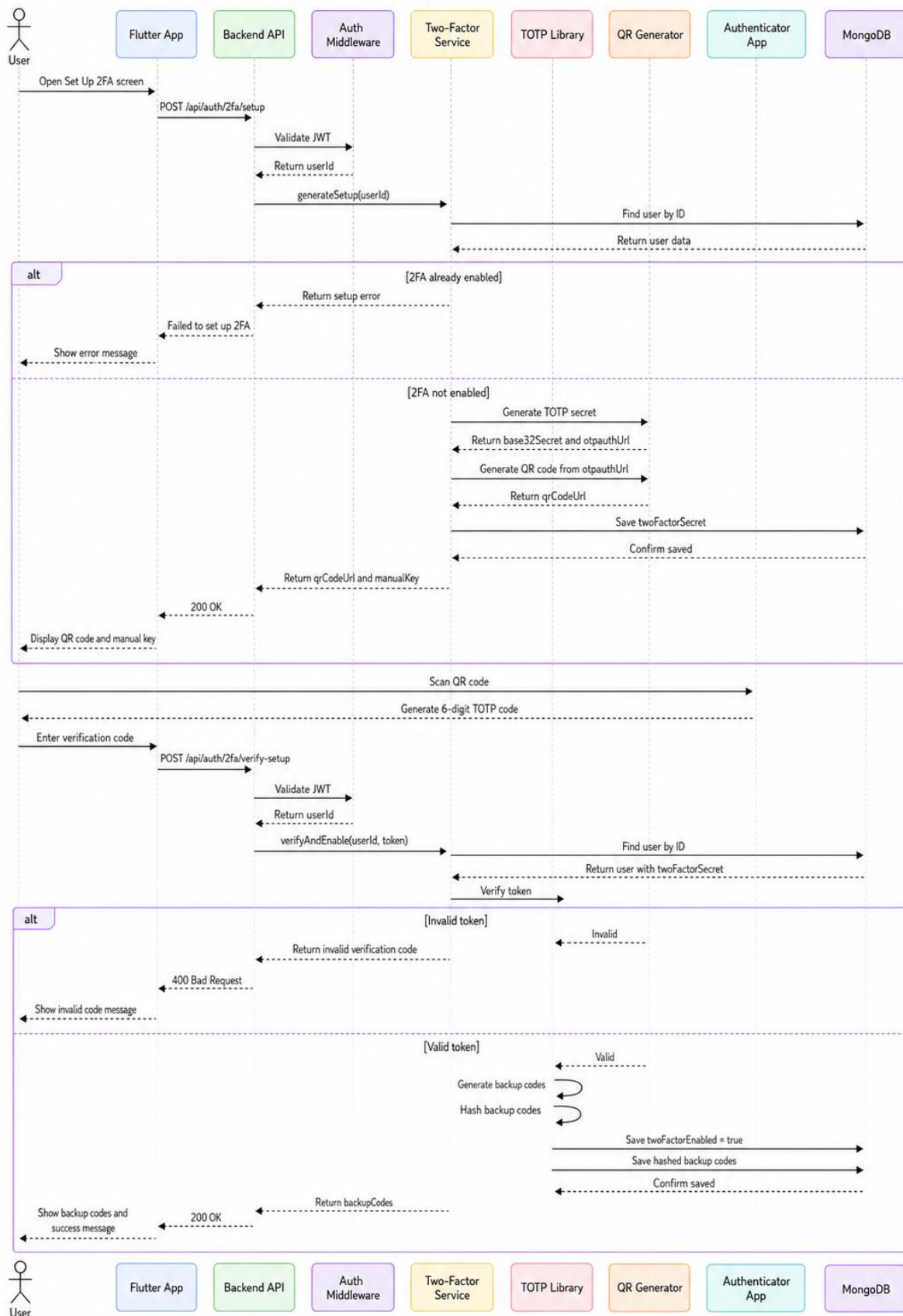


Figure 33 - 2FA sequences diagram

This sequence is triggered only when the login response indicates that two-factor authentication is required. The client submits the temporary token together with the

verification code, and the backend validates both before issuing the final JWT. This ensures that account access is granted only after the second authentication step is successfully completed.

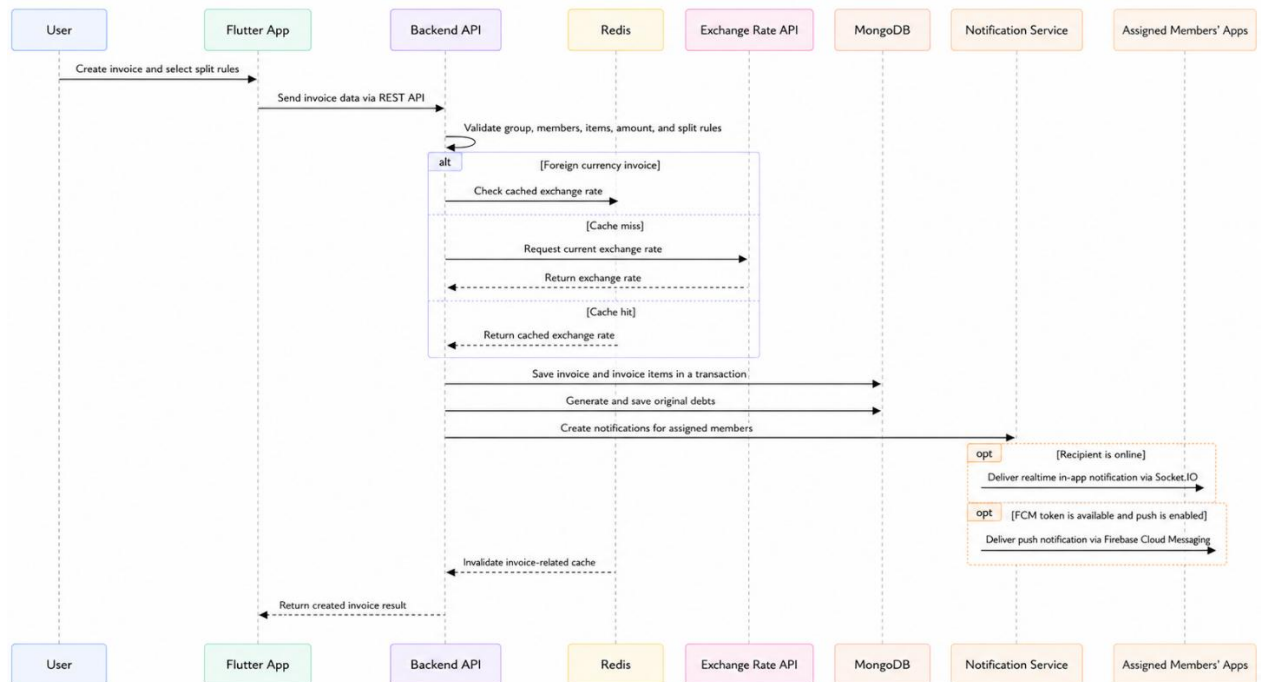


Figure 34 - Create Invoices sequences diagram



Figure 35 - Create Payment request sequences diagram

The mobile application communicates with the backend mainly through REST APIs for standard operations such as authentication, group management, invoice creation, debt tracking, and payment requests. For real-time features such as chat and live notifications, WebSocket communication is used to deliver updates immediately to users. The backend server is the only layer that interacts directly with the database, ensuring that all business rules, validation logic, authorization checks, and financial state changes are controlled centrally.

6 Software implementation

6.1 Development environment

The development environment of SplitPal was designed to support the implementation of a full-stack mobile application with multiple connected financial workflows. Since the system includes a Flutter mobile client, a Node.js/Express backend, MongoDB database, Redis-supported temporary state, and several external services such as payment, notification, email, and AI receipt extraction, the environment had to support more than basic code editing. It needed to allow the team to run, test, debug, inspect, and integrate different system components in a controlled way.

For this reason, the development environment was organised around workflow verification rather than only technology usage. SplitPal is not a simple CRUD application where each feature can be tested independently. A single user action, such as creating an invoice, may trigger several related operations, including validating group members, storing invoice items, generating debts, updating financial records, sending notifications, and displaying the result on the mobile interface. Therefore, the development setup had to make it possible to verify each layer separately before testing the complete end-to-end flow.

6.1.1 Development environment objective

Objective	Implementation in SplitPal	Project value
Independent development	The backend server and Flutter client were placed in separate workspaces	Allows API logic and mobile UI to be developed and tested separately
Workflow verification	APIs were tested before being connected to the mobile client	Reduces integration errors in financial workflows
Data traceability	MongoDB records were inspected after important operations	Confirms whether invoices, debts, transfers, and transactions were created correctly
Safe service integration	External services were configured through testing/sandbox settings where possible	Avoids direct impact on real users, real money, or production credentials
Reproducible setup	Dependencies and configuration values were managed separately	Makes the project easier to run, test, and demonstrate in different environments

This setup was important because many SplitPal features are not isolated actions. For example, creating an invoice may also generate debt records, update group financial data, trigger notifications, and return updated information to the mobile interface. Therefore, the development environment needed to support both isolated testing and complete workflow verification

6.1.2 Local development topology

The project was organised into two main folders: server for the backend and splitpal for the Flutter mobile application. This structure reflects the separation between the business logic layer and the user interaction layer.

Local component	Setup in development	Main role
-----------------	----------------------	-----------

Flutter mobile client	Run on emulator or physical device	Tests user-facing flows such as login, group creation, invoice input, payment requests, and notifications
Express backend server	Run as a local backend process	Handles API requests, validation, financial logic, and service integration
MongoDB	Used as the main local database	Stores users, groups, invoices, debts, transfers, transactions, notifications, and subscriptions
Redis / fallback memory	Used for temporary operational data	Supports OTP, rate limiting, exchange-rate caching, and short-lived control state
Postman	Used for API testing	Verifies backend endpoints before frontend integration
External services	Configured in testing/sandbox mode where possible	Supports payment, notification, email, and AI extraction testing

This topology allowed the team to test backend logic independently before connecting it to the Flutter interface. It also made debugging clearer because errors could be traced through the request payload, backend validation, database record, API response, and frontend display.

6.1.3 Configuration and dependency management

The backend and frontend dependencies were managed separately. The backend used npm-based dependency management for Express, TypeScript, Mongoose, JWT, Socket.IO, Redis, Nodemailer, VNPay, and related libraries. The Flutter client used pubspec.yaml to manage packages such as Provider, Dio, Flutter Secure Storage, Firebase Messaging, Socket.IO Client, image picker, QR-related libraries, and UI support packages.

Configuration area	Development approach	Reason
Backend dependencies	Managed through npm	Keeps API, database, authentication, and integration libraries controlled separately
Flutter dependencies	Managed through pubspec.yaml	Keeps mobile UI, state management, storage, and notification packages organised
Sensitive values	Stored through environment variables or configuration files	Prevents secrets from being hard-coded into the source code
Service credentials	VNPay, Firebase, SMTP, Redis, MongoDB, and Gemini AI settings were separated from business logic	Allows the same codebase to run in local, testing, or demonstration environments

This configuration approach was necessary because SplitPal depends on several services outside the main codebase. Separating configuration from business logic made the system easier to run locally and safer to maintain.

6.1.4 Financial data verification

Because SplitPal manages group finance, the development environment was also used to verify financial data consistency. After important workflows were executed, the generated records were inspected to ensure that the system created the correct financial chain.

Workflow	Records checked	Verification purpose
Invoice creation	Invoice and invoice items	Confirms that the original expense was stored correctly
Debt generation	Original debt records	Confirms who owes whom and how much
Payment request	Payment request and debt status	Confirms that payment requests are linked to valid debts
Wallet transfer	Transfer and transaction records	Confirms that balance changes are traceable
Subscription handling	Subscription, slots, and billing records	Confirms recurring shared services are tracked properly
Notification	Notification record and push event	Confirms users are informed after important actions

6.1.5 External service testing

External services were integrated as supporting components rather than as uncontrolled sources of truth. The backend remained responsible for validating important actions before updating internal financial records.

External service	Development handling	Risk controlled
VNPay	Tested through payment testing/sandbox configuration	Prevents real-money impact during development
Firebase Cloud Messaging	Used to test push notification delivery	Keeps notification delivery separate from core financial logic
SMTP / Nodemailer	Used for OTP and email testing	Supports authentication testing without manual verification
Gemini AI	Used for receipt extraction support	Prevents AI output from being accepted as final financial data without user review
Redis	Used when available, with fallback for local testing	Allows development to continue even when Redis is not running

6.1.6 Development environment limitations

Although the development environment was sufficient for building and verifying the prototype, it did not fully represent a production environment. Local testing can confirm the correctness of individual workflows and basic end-to-end behaviour, but it cannot fully reproduce real production traffic, large-scale concurrent usage, long-term database growth, or real payment risk. Therefore, the current environment should be understood as suitable for implementation and academic demonstration, rather than as complete proof of production readiness.

One limitation is that local or manual testing cannot fully validate high-concurrency financial scenarios. For example, simultaneous invoice updates, repeated payment callbacks, concurrent wallet transfers, or multiple users interacting with the same group at the same time may create edge cases that require deeper stress testing and concurrency testing. These scenarios are difficult to reproduce completely in a simple local development setup.

Another limitation is that sandbox or testing configurations do not capture all risks of real external services. VNPay testing can validate the payment flow structure, but real-world

payment failures, delayed callbacks, bank-side issues, and reconciliation problems would require a more complete staging or production-like environment. Similarly, Firebase notification testing can confirm delivery in simple cases, but large-scale notification reliability would require additional monitoring and operational testing.

The use of fallback logic also introduces a gap between local development and production deployment. For example, in-memory fallback mechanisms can support local testing when Redis is unavailable, but they are weaker than Redis in a horizontally scaled backend environment. If the backend runs across multiple instances, each instance may have separate memory state, reducing the consistency of rate limiting or temporary control data.

Overall, the development environment provided a practical and controlled foundation for implementing SplitPal. It allowed the team to develop frontend and backend components independently, test API workflows before integration, inspect financial records directly, configure external services safely, and verify the main business flows of the prototype. However, before production deployment, the system would require a more advanced staging environment, automated testing pipeline, load testing, security testing, centralised logging, monitoring, and backup strategy.

6.2 Core feartures development

6.2.1 User registration with Email OTP

Aspect	Description
Trigger	POST /auth/signup with email, password, optional displayName.
Validation	Email is normalized (lowercased, trimmed). Duplicate check: if the email exists and is active, throw error; if it exists and is inactive, update credentials and resend OTP instead of creating a duplicate.
Transaction / State Change	Password is bcrypt-hashed. A new User document is created with status: "inactive". If the account already exists but inactive, the password hash and display name are updated in place.
Side Effects	emailService.sendOTP() sends a one-time verification code to the email. No push or in-app notification.
Result Returned	{ userId, email, displayName, avatarUrl, pushNotificationsEnabled, message }. The message differs depending on whether the account was freshly created or reactivated.

```
async SignUpUser(email: string, password: string, displayName?: string):
Promise<any> {
  const normalizedEmail = normalizeEmail(email);
  const existingUser = await User.findOne({ email: normalizedEmail });

  if (existingUser) {
    if (existingUser.status === 'active') {
      throw new Error('Email is already registered');
    }

    const hashedPassword = await this.hashPassword(password);
    existingUser.passwordHash = hashedPassword;
    if (displayName) {
```

```

        existingUser.displayName = displayName;
    }
    await existingUser.save();

    await emailService.sendOTP(normalizedEmail);

    return {
        userId: existingUser._id.toString(),
        email: existingUser.email,
        displayName: existingUser.displayName,
        pushNotificationsEnabled:
toPushNotificationsEnabled(existingUser.pushNotificationsEnabled),
        message: 'Account already exists but was inactive. A new
verification code has been sent to your email.'
    };
}

const hashedPassword = await this.hashPassword(password);
const newUser = await User.create({
    email: normalizedEmail,
    passwordHash: hashedPassword,
    displayName: displayName || normalizedEmail.split('@')[0],
    status: "inactive"
});
await emailService.sendOTP(normalizedEmail);
return {
    userId: newUser._id.toString(),
    email: newUser.email,
    displayName: newUser.displayName,
    avatarUrl: newUser.avatarUrl,
    pushNotificationsEnabled:
toPushNotificationsEnabled(newUser.pushNotificationsEnabled),
    message: 'User registered successfully. Please verify your email to
activate your account.'
};
}

```

This function normalizes email, checks duplicate accounts, hashes the password, creates an inactive user, and sends an OTP for account activation. If the account already exists but is inactive, it updates the password/profile and resends OTP instead of creating a duplicate account.

6.2.2 Login with Optional 2FA

Aspect	Description
Trigger	POST /auth/login with email, password.
Validation	Email normalized → user lookup → account status must be active → bcrypt password comparison.

Transaction / State Change	No write operation. Read-only credential check.
Side Effects	None.
Result Returned	Without 2FA: { user, token } — full JWT and user profile (balance, currency, avatar). With 2FA: { requires2FA: true, tempToken } — a short-lived (5 min) JWT with pending2FA: true claim; the client must complete TOTP verification to obtain a full token.

```

async loginUser(email: string, password: string): Promise<any> {
  const normalizedEmail = normalizeEmail(email);
  const user = await User.findOne({ email: normalizedEmail });
  if (!user) {
    throw new Error('Invalid email or password');
  }
  if (user.status !== 'active') {
    throw new Error('Account is not active. Please verify your email.');
```

```

        email: user.email,
        displayName: user.displayName,
        avatarUrl: user.avatarUrl,
        balance: user.balance,
        currency: user.currency,
        twoFactorEnabled: false,
        pushNotificationsEnabled:
toPushNotificationsEnabled(user.pushNotificationsEnabled),
    },
    token
};
}

```

The login flow validates account status and password. If 2FA is enabled, it returns a short-lived temporary token instead of the full JWT. Otherwise, it returns the normal authentication token and user profile.

6.2.3 Enable two-factor authentication

Two-step process:

Step 1 - generateSetup()

Aspect	Description
Trigger	POST /auth/2fa/setup (authenticated).
Validation	User must exist. 2FA must not already be enabled.
Transaction / State Change	TOTP secret (Speakeasy, 32-byte) is generated and stored in user.twoFactorSecret. 2FA is not enabled yet.
Side Effects	QR code is generated as a Data URL via qrcode library.
Result Returned	{ qrCodeUrl, manualKey } for the authenticator app.

Step 2 - verifyAndEnable()

Aspect	Description
Trigger	POST /auth/2fa/verify with a TOTP token.
Validation	User must exist, 2FA must not be enabled, secret must exist (setup initiated), TOTP token verified with window: 2 (±60 s tolerance).
Transaction / State Change	twoFactorEnabled = true. 10 backup codes are generated, bcrypt-hashed, and stored in twoFactorBackupCodes.
Side Effects	None.
Result Returned	{ backupCodes: string[] } — plain-text codes displayed once for the user to save.

```

sync generateSetup(userId: string): Promise<TwoFactorSetupResponse> {
  const user = await User.findById(userId);
  if (!user) throw new Error('User not found');

  if (user.twoFactorEnabled) {
    throw new Error('Two-factor authentication is already enabled');
  }
}

```

```

    }

    const secret = speakeasy.generateSecret({
      name: `SplitPal (${user.email})`,
      issuer: ISSUER,
      length: 32,
    });

    const qrCodeUrl = await QRCode.toDataURL(secret.otppauth_url!);

    user.twoFactorSecret = secret.base32;
    await user.save();

    return {
      qrCodeUrl,
      manualKey: secret.base32,
    };
  }
}

async verifyAndEnable(userId: string, token: string): Promise<{ backupCodes:
string[] }> {
  const user = await User.findById(userId);
  if (!user) throw new Error('User not found');

  if (user.twoFactorEnabled) {
    throw new Error('Two-factor authentication is already enabled');
  }

  if (!user.twoFactorSecret) {
    throw new Error('Please initiate 2FA setup first');
  }

  const isValid = speakeasy.totp.verify({
    secret: user.twoFactorSecret,
    encoding: 'base32',
    token: token,
    window: 2,
  });

  if (!isValid) {
    throw new Error('Invalid verification code. Please try again.');
```



```

    user.twoFactorEnabled = true;
    user.twoFactorBackupCodes = hashedCodes;
    await user.save();

    return { backupCodes: plainCodes };
}

```

The first function generates a TOTP secret and QR code but does not enable 2FA yet. The second function verifies the authenticator code, enables 2FA, hashes backup codes, and returns the plain backup codes once for the user to save.

6.2.4 Group creation and Invitation

Aspect	Description
Trigger	POST /groups with name, optional description, baseCurrency.
Validation	None beyond authentication. A unique 6-char join code is generated.
Transaction / State Change	Group document created. GroupMember document created with role: 'OWNER'.
Side Effects	None.
Result Returned	GroupResponse with group metadata, join code, and the owner as the sole member.

```

async createGroup(userId: string, data: CreateGroupRequest):
Promise<GroupResponse> {
    const joinCode = await createUniqueJoinCode();

    const group = await Group.create({
        name: data.name,
        description: data.description || '',
        baseCurrency: data.baseCurrency || 'VND',
        joinCode,
        createdBy: userId
    });

    const member = await GroupMember.create({
        groupId: group._id.toString(),
        userId: userId,
        role: 'OWNER'
    });

    const user = await User.findById(userId).select('_id email displayName
avatarUrl');

    return {
        id: group._id.toString(),
        name: group.name,
        description: group.description,

```

```

        joinCode: group.joinCode,
        baseCurrency: group.baseCurrency,
        createdAt: group.createdAt,
        createdBy: group.createdBy,
        memberCount: 1,
        members: [{
            id: member._id.toString(),
            userId: member.userId,
            groupId: member.groupId,
            role: member.role as GroupRole,
            joinedAt: member.joinedAt,
            leftAt: member.leftAt ?? undefined,
            user: transformUser(user!)
        }]
    };
}

async createInvite(userId: string, groupId: string, data: InviteRequest):
Promise<InviteResponse> {
    const membership = await GroupMember.findOne({ groupId, userId, leftAt:
null });

    if (!membership || (membership.role !== 'OWNER' && membership.role !==
'ADMIN')) {
        throw new Error('Permission denied. Only OWNER or ADMIN can create
invites.');
```

```

        expiredAt,
        invitedBy: userId
    });

    const invitedUser = await User.findOne({ email: data.emailInvite });
    if (invitedUser) {
        const group = await Group.findById(groupId);
        const inviter = await User.findById(userId);

        await notificationService.createNotification({
            userId: invitedUser._id.toString(),
            type: NotificationType.INVITE_RECEIVED,
            title: 'Group Invitation',
            message: `${inviter?.displayName || 'Someone'} invited you to join ${group?.name || 'a group'}`,
            data: { groupId, inviteToken: token, groupName: group?.name }
        });
    }

    return {
        id: invite._id.toString(),
        emailInvite: invite.emailInvite,
        role: invite.role as GroupRole,
        status: invite.status as 'PENDING' | 'ACCEPTED' | 'EXPIRED',
        expiresAt: invite.expiredAt,
        createdAt: invite.createdAt,
        token: invite.token
    };
}

```

Group creation automatically creates the owner membership. Invitation creation is restricted to OWNER/ADMIN, prevents duplicate pending invitations, generates a secure token, sets a seven-day expiry, and notifies the invited user if they already exist.

6.2.5 Invoice creation and Debt generation

Aspect	Description
Trigger	POST /invoices/:groupId with title, items[], optional currency, invoiceDate, imageUrl, note.
Validation	Caller must be an active group member. All assignedTo user IDs must be active group members. At least one item required. Split validation per item: EQUAL - no splits needed; PERCENTAGE - splits must sum to 100; CUSTOM - splits must sum to item amount; WEIGHT - all share weights must be positive (proportional split by ratio). Foreign currency invoices trigger exchange rate lookup.
Transaction / State Change	Mongoose session transaction: (1) Invoice created with status: 'SUBMITTED'; (2) InvoiceItem[] created with splitType and splits metadata; (3) OriginalDebt[] created - one per debtor, amount calculated via calculateShareForUser() which supports EQUAL, PERCENTAGE, CUSTOM, and WEIGHT splits, then converted to group base currency using the locked exchange rate.

Side Effects	notificationService.createNotification() sends INVOICE_CREATED notification to all assigned users (except the uploader) via Socket.IO + FCM push. Redis invoice cache invalidated.
Result Returned	Full InvoiceResponse with items, assigned user details, currency conversion info.

```

async createInvoice(userId: string, groupId: string, data:
CreateInvoiceRequest): Promise<InvoiceResponse> {
  const membership = await GroupMember.findOne({ groupId, userId, leftAt:
null });
  if (!membership) {
    throw new Error('NOT_GROUP_MEMBER');
  }

  const amountTotal = data.items.reduce((sum, item) => sum + item.amount,
0);

  const allAssignedUsers = new Set<string>();
  data.items.forEach(item => item.assignedTo.forEach(uid =>
allAssignedUsers.add(uid)));

  for (const assignedUserId of allAssignedUsers) {
    const isMember = await GroupMember.findOne({ groupId, userId:
assignedUserId, leftAt: null });
    if (!isMember) {
      throw new Error(`User ${assignedUserId} is not a member of this
group`);
    }
  }

  if (data.items.length === 0) {
    throw new Error('Invoice must have at least one item');
  }

  for (const item of data.items) {
    validateItemSplits(item);
  }

  const session = await mongoose.startSession();
  let invoice: any;
  let invoiceItems: any[] = [];

  try {
    await session.withTransaction(async () => {
      const group = await Group.findById(groupId).session(session);
      if (!group) throw new Error('Group not found');
      const baseCurrency = group.baseCurrency || 'VND';

```

```

        const invoiceCurrency = data.currency?.toUpperCase() ||
baseCurrency;
        const isForeignCurrency = invoiceCurrency !== baseCurrency;

        let exchangeRate: number | null = null;
        let convertedAmountTotal: number | null = null;

        if (isForeignCurrency) {
            exchangeRate = await
exchangeRateService.getRate(invoiceCurrency, baseCurrency);
            convertedAmountTotal = Math.round(amountTotal * exchangeRate *
100) / 100;
        }

        [invoice] = await Invoice.create([
            {
                groupId,
                title: data.title,
                amountTotal,
                currency: invoiceCurrency,
                convertedAmountTotal: isForeignCurrency ? convertedAmountTotal
: null,
                exchangeRate: isForeignCurrency ? Math.round(exchangeRate! *
10000) / 10000 : null,
                baseCurrency: isForeignCurrency ? baseCurrency : null,
                uploadedBy: userId,
                invoiceDate: data.invoiceDate ? new Date(data.invoiceDate) :
new Date(),
                imageUrl: data.imageUrl,
                note: data.note,
                status: 'SUBMITTED'
            }
        ], { session });

        invoiceItems = await InvoiceItem.create(
            data.items.map(item => ({
                invoiceId: invoice._id.toString(),
                name: item.name,
                amount: item.amount,
                splitType: item.splitType || 'EQUAL',
                assignedTo: item.assignedTo,
                splits: item.splits || []
            }))),
            { session, ordered: true }
        );

        const debtMap = new Map<string, number>();

        for (let i = 0; i < invoiceItems.length; i++) {
            const dbItem = invoiceItems[i];

```

```

        const inputItem = data.items[i];
        if (dbItem.assignedTo.length === 0) continue;

        for (const debtorId of dbItem.assignedTo) {
            if (debtorId === userId) continue;

            const share = calculateShareForUser(
                { amount: dbItem.amount, assignedTo:
dbItem.assignedTo, splitType: dbItem.splitType, splits: inputItem.splits },
                debtorId
            );
            debtMap.set(debtorId, (debtMap.get(debtorId) || 0) +
share);
        }
    }

    const now = new Date();
    const debts = Array.from(debtMap.entries()).map(([debtorId,
amountInCurrency]) => {
        const amountInBase = isForeignCurrency
            ? Math.round(amountInCurrency * exchangeRate! * 100) / 100
            : Math.round(amountInCurrency * 100) / 100;

        return {
            groupId,
            invoiceId: invoice._id.toString(),
            debtorId,
            creditorId: userId,
            originalAmount: amountInBase,
            remainingAmount: amountInBase,
            ...(isForeignCurrency ? {
                originalCurrency: invoiceCurrency,
                originalAmountInCurrency: Math.round(amountInCurrency
* 100) / 100,
                exchangeRateUsed: Math.round(exchangeRate! * 10000) /
10000,
                rateLockedAt: now
            } : {})
        };
    });

    if (debts.length > 0) {
        await OriginalDebt.create(debts, { session, ordered: true });
    }
} finally {
    session.endSession();
}

```

```

    await invalidateInvoiceCache(groupId);
    return this.getInvoiceById(userId, groupId, invoice._id.toString());
}

```

This function validates group membership and invoice splits, locks exchange rates when the invoice uses a foreign currency, creates invoice/items inside a transaction, calculates each debtor's share, and stores original debt records in the group base currency.

6.2.6 Payment request creation and Debt settlement plan

Aspect	Description
Trigger	POST /payment-requests/:groupId (OWNER/ADMIN only).
Validation	Caller must be OWNER or ADMIN. A distributed lock (acquireLock('payment_request', groupId, 10s)) prevents concurrent creation. No existing ISSUED or PARTIALLY_PAID request allowed. At least one unlocked SUBMITTED invoice must exist.
Transaction / State Change	Mongoose session transaction: (1) PaymentRequest created with status: 'ISSUED', 7-day expiry; (2) All submitted invoices locked (isLocked: true, status: 'LOCKED'); (3) debtSettlementEngine.settle() runs net-balance optimization (auto-selects Greedy or MinCostFlow); (4) Transfer[] created with status: 'PENDING'; (5) TransferDebtAllocation[] created mapping each transfer to specific OriginalDebt records.
Side Effects	Redis caches invalidated for both payment requests and invoices. Lock released in finally.
Result Returned	PaymentRequestDetailResponse with transfer summaries, user breakdowns, and net balances.

```

async createPaymentRequest(userId: string, groupId: string):
Promise<PaymentRequestDetailResponse> {
    const membership = await GroupMember.findOne({ groupId, userId, leftAt:
null });
    if (!membership) {
        throw new Error('NOT_GROUP_MEMBER');
    }
    if (membership.role !== 'OWNER' && membership.role !== 'ADMIN') {
        throw new Error('Only OWNER or ADMIN can create payment requests');
    }

    const lock = await acquireLock('payment_request', groupId, 10_000);
    if (!lock) {
        throw new Error('Another payment request is being created. Please
retry in a few seconds.');
```

```

    }).session(session);
    if (existingRequest) {
        throw new Error('There is already an open payment request.
Please complete or cancel it first.');
```

```

    }

    const invoices = await Invoice.find({
        groupId,
        status: 'SUBMITTED',
        isLocked: false
    }).session(session);

    if (invoices.length === 0) {
        throw new Error('No submitted invoices available for payment
request');
```

```

    }

    const invoiceIds = invoices.map(inv => inv._id.toString());
    const netBalances = await
originalDebtService.getNetBalances(groupId);
    const rawDebtDocs = await OriginalDebt.find({
        groupId,
        remainingAmount: { $gt: 0.01 }
    });

    const rawDebts: RawDebt[] = rawDebtDocs.map(d => ({
        debtorId: d.debtorId,
        creditorId: d.creditorId,
        remainingAmount: d.remainingAmount
    }));

    const transfers = await debtSettlementEngine.settle(groupId,
netBalances, rawDebts);

    if (transfers.length === 0) {
        throw new Error('No transfers needed - all balances are
settled');
```

```

    }

    const paymentRequest = await PaymentRequest.create([ {
        groupId,
        createdBy: userId,
        invoiceIds,
        status: 'ISSUED',
        issuedAt: new Date(),
        expiresAt: new Date(Date.now() + 7 * 24 * 60 * 60 * 1000),
        lastReminderAt: null
    } ], { session }).then(res => res[0]);

```



```

    await Invoice.updateMany(
      { _id: { $in: invoiceIds } },
      {
        isLocked: true,
        status: 'LOCKED',
        paymentRequestId: paymentRequest._id.toString()
      },
      { session }
    );

    for (const transfer of transfers) {
      const createdTransfer = await Transfer.create([
        {
          paymentRequestId: paymentRequest._id.toString(),
          groupId,
          fromUserId: transfer.fromUserId,
          toUserId: transfer.toUserId,
          amount: transfer.amount,
          status: 'PENDING'
        }
      ], { session }).then(res => res[0]);

      if (transfer.debtAllocations.length > 0) {
        await TransferDebtAllocation.insertMany(
          transfer.debtAllocations.map(alloc => ({
            transferId: createdTransfer._id.toString(),
            originalDebtId: alloc.originalDebtId,
            allocatedAmount: alloc.amount
          })), { session }
        );
      }
    }

    return paymentRequest._id.toString();
  });

  await invalidatePaymentRequestCache(groupId);
  await invalidateRelatedInvoiceCache(groupId);
  return this.getPaymentRequestById(userId, groupId, result!);
} finally {
  await session.endSession();
  await lock.release();
}
}

```

Only OWNER/ADMIN can create a payment request. A distributed lock prevents concurrent creation. The function locks submitted invoices, calculates optimized settlement transfers from remaining debts, stores transfer allocation records, and returns the full payment request detail.

6.2.7 Transfer payment with OTP and Debt allocation

Aspect	Description
Trigger	POST /payment-requests/:groupId/:requestId/cancel or automatic expiration (scheduler).
Validation	Caller must be OWNER/ADMIN, or a regular member with a pending transfer. Request must be ISSUED or PARTIALLY_PAID. Supports force option for system-triggered expiration.
Transaction / State Change	Full refund/restore logic in a Mongoose session: (1) Completed transfers are refunded: payer balance restored (\$inc: +amount), receiver balance deducted (\$inc: -amount); (2) Original debts are restored via originalDebtService.restoreDebt() using TransferDebtAllocation records; (3) All transfers set to CANCELLED; (4) TransferDebtAllocation records deleted; (5) Invoices unlocked (isLocked: false, status: 'SUBMITTED', paymentRequestId: null); (6) Payment request set to CANCELLED.
Side Effects	Refund transaction records (TRANSFER_REFUND_RECEIVED / TRANSFER_REFUND_SENT) logged. PAYMENT_REQUEST_CANCELLED notification sent to all participants. Individual PAYMENT_REFUNDED notifications sent to both parties of each refunded transfer. Redis caches invalidated.
Result Returned	{ refundedTransfers: number, cancelledTransfers: number }.
Aspect	Description
Trigger	POST /transfers/:transferId/pay with otp, optional categoryTagId.
Validation	Transfer must exist. Caller must be the fromUserId. Transfer must be PENDING (not already COMPLETED). OTP must have been requested, not expired, and must match.
Transaction / State Change	Mongoose session transaction: (1) Transfer status atomically set to COMPLETED (with optimistic lock on status: 'PENDING' + OTP + expiry); (2) Payer balance checked and decreased (\$gte: amount guard prevents overdraft); (3) Receiver balance increased; (4) Allocated debts reduced via originalDebtService.reduceDebt() using TransferDebtAllocation records - if no allocations exist, falls back to reduceDebtsBetweenUsers().
Side Effects	Two transaction records created (TRANSFER_SENT + TRANSFER_RECEIVED). PAYMENT_RECEIVED notification sent to receiver via Socket.IO + FCM push. paymentRequestService.updateRequestStatus() recalculates the request status (ISSUED → PARTIALLY_PAID → PAID).
Result Returned	{ success: true, message, transfer, newBalance }.

```
async verifyOTPAAndPay(userId: string, transferId: string, otp: string,
categoryTagId?: string): Promise<PaymentCompleteResponse> {
  const transfer = await Transfer.findById(transferId);
  if (!transfer) {
    throw new Error('Transfer not found');
  }

  if (transfer.fromUserId !== userId) {
    throw new Error('You can only pay your own transfers');
  }

  if (transfer.status === 'COMPLETED') {
    throw new Error('Transfer is already completed');
  }
}
```

```

if (!transfer.otp || !transfer.otpExpiresAt) {
    throw new Error('No OTP requested. Please initiate payment first.');
```

```

}

if (transfer.otpExpiresAt < new Date()) {
    throw new Error('OTP has expired. Please request a new OTP.');
```

```

}

if (transfer.otp !== otp) {
    throw new Error('Invalid OTP');
```

```

}

const fromUser = await User.findById(transfer.fromUserId);
const toUser = await User.findById(transfer.toUserId);
if (!fromUser || !toUser) {
    throw new Error('User not found');
```

```

}

const amount = Number(transfer.amount);
const allocations = await TransferDebtAllocation.find({ transferId });
const session = await mongoose.startSession();

try {
    await session.withTransaction(async () => {
        const lockedTransfer = await Transfer.findOneAndUpdate(
            {
                _id: transferId,
                status: 'PENDING',
                otp,
                otpExpiresAt: { $gt: new Date() }
            },
            {
                status: 'COMPLETED',
                paidAt: new Date(),
                otp: null,
                otpVerified: true,
                categoryTagId: categoryTagId ?? null
            },
            { session, new: false }
        );

        if (!lockedTransfer) {
            throw new Error('Transfer already processed or OTP invalid');
```

```

        }

        const fromUserDoc = await User.findOneAndUpdate(
            { _id: transfer.fromUserId, balance: { $gte: amount } },
            { $inc: { balance: -amount } },

```

```

        { session, new: false }
    );

    if (!fromUserDoc) {
        throw new Error('Insufficient balance');
    }

    const toUserDoc = await User.findOneAndUpdate(
        { _id: transfer.toUserId },
        { $inc: { balance: amount } },
        { session, new: false }
    );

    if (!toUserDoc) {
        throw new Error('Recipient not found');
    }

    if (allocations.length > 0) {
        const totalAllocated = allocations.reduce((sum, a) => sum +
a.allocatedAmount, 0);
        if (Math.abs(totalAllocated - amount) > 0.01) {
            throw new Error(`Allocation sum (${totalAllocated}) !=
transfer amount (${amount})`);
        }

        for (const alloc of allocations) {
            await originalDebtService.reduceDebt(alloc.originalDebtId,
alloc.allocatedAmount, session);
        }
    } else {
        await
originalDebtService.reduceDebtsBetweenUsers(transfer.groupId,
transfer.fromUserId, transfer.toUserId, amount, session);
    }
});

await transactionService.createTransaction({
    userId: fromUser._id.toString(),
    groupId: transfer.groupId,
    type: TransactionType.TRANSFER_SENT,
    amount: amount,
    balanceBefore: newFromBalance + amount,
    balanceAfter: newFromBalance,
    currency: 'VND',
    description: `Payment to ${toUser.displayName}`,
    referenceId: transferId,
    referenceType: 'TRANSFER'
});

```

```

        await transactionService.createTransaction({
            userId: toUser._id.toString(),
            groupId: transfer.groupId,
            type: TransactionType.TRANSFER_RECEIVED,
            amount: amount,
            balanceBefore: newToBalance - amount,
            balanceAfter: newToBalance,
            currency: 'VND',
            description: `Received from ${fromUser.displayName}`,
            referenceId: transferId,
            referenceType: 'TRANSFER'
        });
    } finally {
        session.endSession();
    }

    await
paymentRequestService.updateRequestStatus(transfer.paymentRequestId);
    await notificationService.createNotification({
        userId: transfer.toUserId,
        type: NotificationType.PAYMENT_RECEIVED,
        title: 'Payment Received',
        message: `${fromUser.displayName} || 'Someone'} sent you
${amount.toLocaleString()} VND`,
        data: { transferId, groupId: transfer.groupId, amount, fromUserId:
fromUser._id.toString() }
    });

    const updatedTransfer = await this.getTransferById(transferId);
    return {
        success: true,
        message: 'Payment completed successfully',
        transfer: updatedTransfer!,
        newBalance: newFromBalance
    };
}

```

The function verifies ownership, OTP validity, transfer status, and payer balance. The payment is completed inside a database transaction: transfer status changes to COMPLETED, payer balance decreases, receiver balance increases, allocated debts are reduced, transactions are logged, and the payment request status is updated.

6.2.8 Wallet Top-Up completion

Aspect	Description
Trigger	VNPay IPN callback → completeTopUp(topUpId, txnRef).

Validation	Top-up must exist. If already COMPLETED, the function returns immediately (idempotent). User must exist.
Transaction / State Change	Mongoose session transaction: Top-up status atomically updated from PENDING → COMPLETED (optimistic lock on status: PENDING). User balance increased by \$inc: amount. If the status was already changed by another callback, the transaction silently skips (idempotent).
Side Effects	Transaction record (TOP_UP) created. BALANCE_UPDATED notification sent to user with the amount and new balance via Socket.IO + FCM push. Both side effects run only if processed === true.
Result Returned	void - callback acknowledgment only.

```

async completeTopUp(topUpId: string, txnRef: string): Promise<void> {
  const topUp = await TopUp.findById(topUpId);

  if (!topUp) {
    throw new Error("Top-up transaction not found");
  }

  if (topUp.status === TopUpStatus.COMPLETED) {
    return;
  }

  const user = await User.findById(topUp.userId);
  if (!user) {
    throw new Error("User not found");
  }

  const balanceBefore = Number(user.balance);
  const amount = Number(topUp.amount);

  const session = await mongoose.startSession();
  let processed = false;
  try {
    await session.withTransaction(async () => {
      const updatedTopUp = await TopUp.findOneAndUpdate(
        { _id: topUpId, status: TopUpStatus.PENDING },
        { status: TopUpStatus.COMPLETED, vnpayTxnRef: txnRef },
        { new: true, session }
      );

      if (!updatedTopUp) {
        return;
      }

      const updateResult = await User.updateOne(
        { _id: topUp.userId },
        { $inc: { balance: amount } },

```

```

        { session }
    );

    if (updateResult.matchedCount === 0) {
        throw new Error("User not found during balance update");
    }

    processed = true;
});
} finally {
    await session.endSession();
}

if (processed) {
    await transactionService.createTransaction({
        userId: topUp.userId,
        type: TransactionType.TOP_UP,
        amount: amount,
        balanceBefore: balanceBefore,
        balanceAfter: balanceBefore + amount,
        currency: "VND",
        description: "Top up via VNPay",
        referenceId: topUpId,
        referenceType: "TOP_UP"
    });

    await notificationService.createNotification({
        userId: topUp.userId,
        type: NotificationType.BALANCE_UPDATED,
        title: 'Balance Updated',
        message: `Your balance was increased by ${amount.toLocaleString()} VND. New balance: ${
            (balanceBefore + amount).toLocaleString()
        } VND`,
        data: { topUpId, amount, newBalance: balanceBefore + amount, transactionType: 'TOP_UP' }
    });
}
}

```

This function finalizes a VNPay top-up callback. It is idempotent: only a PENDING top-up can be marked COMPLETED. The balance update is wrapped in a transaction, then the system creates a transaction log and balance notification.

6.2.9 Withdrawal with Email OTP

Two-step flow:

Step 1 - initiateWithdrawal()

Aspect	Description
--------	-------------

Trigger	POST /withdrawals with amount.
Validation	User must exist. Amount must be > 0. User balance must be $\geq$ amount (pre-check, not locked yet).
Transaction / State Change	Withdrawal document created with status: 'OTP_SENT', a generated OTP, and 10-minute expiry. Balance is not deducted yet.
Side Effects	OTP sent to user's email via emailService.sendOTPEmail().
Result Returned	WithdrawalResponse with withdrawal ID, amount, bank details, status.

Step 2 - verifyOTP()

Aspect	Description
Trigger	POST /withdrawals/:id/verify with otp.
Validation	Withdrawal must exist and be in OTP_SENT status. OTP must not be expired. OTP must match.
Transaction / State Change	Mongoose session transaction: (1) Withdrawal atomically updated from OTP_SENT → COMPLETED (optimistic lock on status + OTP + expiry); (2) User balance decreased with \$gte: amount guard (prevents overdraft from concurrent requests); (3) Transaction record created within the session.
Side Effects	Transaction record (WITHDRAWAL) logged.
Result Returned	{ success: true, message, user: { id, balance } }.

```

async initiateWithdrawal(userId: string, data: CreateWithdrawalRequest):
Promise<WithdrawalResponse> {
  const user = await User.findById(userId);
  if (!user) {
    throw new Error('User not found');
  }

  const requestAmount = Number(data.amount);
  if (requestAmount <= 0) {
    throw new Error('Amount must be greater than 0');
  }

  const userBalance = Number(user.balance);
  if (userBalance < requestAmount) {
    throw new Error(`Insufficient balance. Available: ${userBalance},
Requested: ${requestAmount}`);
  }

  const otp = generateOTP();
  const otpExpiresAt = new Date(Date.now() + 10 * 60 * 1000);

  const withdrawal = await Withdrawal.create({
    userId,
    amount: data.amount,
    currency: 'VND',
    accountNumber: DEFAULT_WITHDRAWAL_BANK.accountNumber,

```



```

        bankName: DEFAULT_WITHDRAWAL_BANK.bankName,
        accountName: DEFAULT_WITHDRAWAL_BANK.accountName,
        status: 'OTP_SENT',
        otp,
        otpExpiresAt
    });

    await emailService.sendOTPEmail(user.email, otp);
    return transformWithdrawal(withdrawal);
}

async verifyOTP(withdrawalId: string, otp: string): Promise<any> {
    const withdrawal = await Withdrawal.findById(withdrawalId);
    if (!withdrawal) {
        throw new Error('Withdrawal not found');
    }

    if (withdrawal.status !== 'OTP_SENT') {
        throw new Error(`Cannot verify OTP for withdrawal in ${withdrawal.status} status`);
    }

    if (!withdrawal.otpExpiresAt || withdrawal.otpExpiresAt < new Date()) {
        throw new Error('OTP has expired. Please request a new OTP.');
```

```

    if (!lockedWithdrawal) {
      throw new Error('Invalid or expired OTP');
    }

    const userDoc = await User.findOneAndUpdate(
      { _id: withdrawal.userId, balance: { $gte: amount } },
      { $inc: { balance: -amount } },
      { session, new: false }
    );

    if (!userDoc) {
      throw new Error('Insufficient balance');
    }

    await transactionService.createTransaction({
      userId: withdrawal.userId,
      type: TransactionType.WITHDRAWAL,
      amount: amount,
      balanceBefore: Number(userDoc.balance),
      balanceAfter: Number(userDoc.balance) - amount,
      currency: 'VND',
      description: `Withdrawal to ${withdrawal.bankName} -
${withdrawal.accountNumber}`,
      referenceId: withdrawalId,
      referenceType: 'WITHDRAWAL',
      session
    });
  });
} finally {
  await session.endSession();
}

const updatedUser = await User.findById(withdrawal.userId);
return {
  success: true,
  message: 'Withdrawal completed successfully',
  user: {
    id: updatedUser!._id.toString(),
    balance: Number(updatedUser!.balance)
  }
};
}

```

Withdrawal is a two-step flow. The first function checks balance and sends a 10-minute OTP. The second verifies OTP and deducts the wallet balance atomically, preventing double processing or overdraft through status and balance conditions.

6.2.10 Recurring bill auto-generation

Aspect	Description
Trigger	Scheduler (cron job) invokes processAutoGenerate() periodically.
Validation	Only ACTIVE templates with nextBillDate ≤ now are processed. Duplicate prevention: checks for existing invoice with the same templateId + billingPeriod. Skips templates where the group has zero active members.
Transaction / State Change	Mongoose session transaction per template: (1) Invoice created as DRAFT (not SUBMITTED) — no debts are generated yet; (2) InvoiceItem[] created with member assignments resolved to current active group members; (3) Template's lastGeneratedAt and nextBillDate updated for the next cycle.
Side Effects	RECURRING_BILL_DRAFT notification sent to the payer (template owner) asking them to review and confirm. Redis invoice cache invalidated per group.
Result Returned	{ generated: number, skipped: number, failed: number }.

```
sync processAutoGenerate(): Promise<{ generated: number; skipped: number;
failed: number }> {
  const now = new Date();
  let generated = 0, skipped = 0, failed = 0;

  const dueTemplates = await BillTemplate.find({
    status: 'ACTIVE',
    nextBillDate: { $lte: now }
  });

  for (const template of dueTemplates) {
    const billingPeriod = computeBillingPeriod(template.billingCycle as
BillTemplateCycle, now);

    const exists = await Invoice.findOne({
      templateId: template._id.toString(),
      billingPeriod
    });
    if (exists) { skipped++; continue; }

    try {
      const activeMembers = await GroupMember.find({
        groupId: template.groupId,
        leftAt: null
      }).select('userId');
      const memberIds = activeMembers.map(m => m.userId);

      if (memberIds.length === 0) {
        skipped++;
        continue;
      }
    }
```

```

const resolvedItems = template.items.map(item => ({
  name: item.name,
  amount: item.amount,
  splitType: item.splitType,
  assignedTo: item.assignedTo.length > 0
    ? item.assignedTo.filter(uid => memberIds.includes(uid))
    : memberIds,
  splits: item.splits ?? []
}));

const title = buildInvoiceTitle(template.name,
template.billingCycle as BillTemplateCycle, now);
const amountTotal = resolvedItems.reduce((s, i) => s + i.amount,
0);

const session = await mongoose.startSession();
let invoiceId: string;
try {
  session.startTransaction();

  const [invoice] = await Invoice.create([
    {
      groupId: template.groupId,
      title,
      amountTotal,
      currency: template.currency,
      uploadedBy: template.payerId,
      invoiceDate: now,
      note: `Hóa đơn định kỳ - tự động tạo từ
"${template.name}"`,
      status: 'DRAFT',
      templateId: template._id.toString(),
      billingPeriod,
      isLocked: false,
      isAdjustment: false,
      groupDeleted: false,
    },
  ], { session });

  invoiceId = invoice._id.toString();

  await InvoiceItem.insertMany(
    resolvedItems.map(item => ({
      invoiceId,
      name: item.name,
      amount: item.amount,
      splitType: item.splitType,
      assignedTo: item.assignedTo,
      splits: item.splits
    })),
  ),

```

```

        { session }
    );

    await BillTemplate.findByIdAndUpdate(template._id, {
        lastGeneratedAt: now,
        nextBillDate: calculateNextBillDate(
            template.billingCycle as BillTemplateCycle,
            template.billingDay,
            now
        )
    }, { session });

    await session.commitTransaction();
} catch (err) {
    await session.abortTransaction();
    throw err;
} finally {
    session.endSession();
}

await notificationService.notify(
    template.payerId,
    NotificationType.RECURRING_BILL_DRAFT,
    'Hóa đơn định kỳ cần xác nhận',
    `Hóa đơn "${title}" đã được tạo tự động. Kiểm tra và xác nhận để gửi cho thành viên.`,
    { invoiceId: invoiceId!, groupId: template.groupId,
templateId: template._id.toString() }
);

await invalidateInvoiceCache(template.groupId);
generated++;
} catch (err) {
    failed++;
}
}

return { generated, skipped, failed };
}

```

The scheduler finds active bill templates whose next bill date has arrived. It prevents duplicate invoices per billing period, snapshots active group members, creates a draft invoice and items in a transaction, updates the next schedule, and notifies the payer to review the draft.

6.2.11 Personal receipt and Budget tracking

Receipt Creation - createReceipt()

Aspect	Description
Trigger	POST /receipts with imageUrl, totalAmount, tagIds[], receiptDate, optional note.
Validation	Ensures user has default tags initialized (ensureUserHasTags). Receipt date normalized. Per-day receipt limit enforced (enforcePerDayLimit). Tags must be owned by the user (ensureTagsOwned). imageUrl is required and non-empty. Note max 500 characters.
Transaction / State Change	Receipt document created with userId, imageUrl, totalAmount, tags (ObjectId refs), and normalized receiptDate.
Side Effects	None.
Result Returned	ReceiptDto with receipt details and expanded tag objects.

Monthly Summary - getMonthSummary()

Aspect	Description
Trigger	GET /receipts/summary/:month (format: YYYY-MM).
Validation	Month format validated against ^\d{4}-\d{2}\$ regex.
Transaction / State Change	None (read-only).
Side Effects	None.
Result Returned	Array of MonthSummaryItem[] — one per day with { date, count, totalAmount, thumbUrls (max 3) }, grouped via MongoDB aggregation pipeline.

```

async createReceipt(userId: string, payload: ReceiptInput):
Promise<ReceiptDto> {
  await ensureUserHasTags(userId);

  const receiptDate = normalizeDate(payload.receiptDate);
  await enforcePerDayLimit(userId, receiptDate);
  const tags = await ensureTagsOwned(payload.tagIds, userId);

  if (!payload.imageUrl || !payload.imageUrl.trim()) {
    throw new AppError('imageUrl is required', 'VALIDATION_ERROR');
  }
  if (payload.note && payload.note.length > 500) {
    throw new AppError('Note too long (max 500)', 'VALIDATION_ERROR');
  }

  const receipt = await Receipt.create({
    userId,
    imageUrl: payload.imageUrl.trim(),
    totalAmount: payload.totalAmount || 0,
    note: payload.note?.trim() || undefined,
    tags: tags.map(t => t._id),
    receiptDate,
  });

  return toReceiptDto(receipt, tags);
}

```

```

async getMonthSummary(userId: string, month: string):
Promise<MonthSummaryItem[]> {
  if (!/^\d{4}-\d{2}$/.test(month)) {
    throw new AppError('Invalid month format. Use YYYY-MM',
'INVALID_MONTH');
  }
  const [year, m] = month.split('-').map(Number);
  const start = new Date(Date.UTC(year, m - 1, 1, 0, 0, 0, 0));
  const end = new Date(Date.UTC(year, m, 1, 0, 0, 0, 0));

  const agg = await Receipt.aggregate([
    { $match: { userId, receiptDate: { $gte: start, $lt: end } } },
    { $sort: { createdAt: 1 } },
    {
      $group: {
        _id: '$receiptDate',
        count: { $sum: 1 },
        totalAmount: { $sum: { $ifNull: ['$totalAmount', 0] } },
        thumbUrls: { $push: '$imageUrl' },
      }
    },
    { $project: { _id: 0, date: { $dateToString: { format: '%Y-%m-%d',
date: '$_id' } } }, count: 1, totalAmount: 1, thumbUrls: { $slice:
['$thumbUrls', 3] } } },
    { $sort: { date: 1 } }
  ]));

  return agg as MonthSummaryItem[];
}

```

Receipt creation validates tags, image URL, note length, date normalization, and daily limits. Monthly summary uses MongoDB aggregation to group receipts by day and calculate daily spending totals.

6.2.12 Realtime, Push, and Email notification

Aspect	Description
Trigger	Internal call from any service via notificationService.createNotification(input).
Validation	None at this level — caller is responsible for providing valid userId, type, title, message.
Transaction / State Change	Notification document saved to MongoDB first (durable-first pattern). This ensures the notification is persisted regardless of downstream delivery failures.
Side Effects	Three best-effort delivery channels: (1) Socket.IO — emits new_notification to user:{userId} room for real-time in-app delivery. (2) Firebase Cloud Messaging (FCM) — sends push notification if user has fcmToken and pushNotificationsEnabled !== false; handles invalid/expired tokens by clearing fcmToken. (3) Email — optional, only if input.sendEmail is true; sends via emailService.sendNotificationEmail() and updates sentEmail flag on the

	notification. All three channels fail independently and silently — no rollback of the saved notification.
Result Returned	void.

```

async createNotification(input: CreateNotificationInput): Promise<void> {
  const notification = await Notification.create({
    userId: input.userId,
    type: input.type,
    title: input.title,
    message: input.message,
    data: input.data,
    sentEmail: false
  });

  try {
    const io = getIO();
    io.to(`user:${input.userId}`).emit('new_notification', {
      id: notification._id.toString(),
      type: notification.type,
      title: notification.title,
      message: notification.message,
      data: notification.data,
      read: notification.read,
      createdAt: notification.createdAt
    });
  } catch (error) {
    console.error('Failed to emit notification via Socket.IO:', error);
  }

  try {
    const user = await User.findById(input.userId);
    if (user?.fcmToken && user.pushNotificationsEnabled !== false &&
isFirebaseInitialized()) {
      const messaging = getMessaging();
      if (messaging) {
        await messaging.send({
          token: user.fcmToken,
          notification: {
            title: input.title,
            body: input.message
          },
          data: input.data ? {
            notificationId: notification._id.toString(),
            type: input.type,
            ...Object.entries(input.data).reduce((acc, [key,
value]) => {
              acc[key] = typeof value === 'string' ? value :
JSON.stringify(value);

```



```

        return acc;
      }, {} as Record<string, string>)
    } : undefined,
    android: { priority: 'high', notification: { sound:
'default' } } },
    apns: { payload: { aps: { sound: 'default' } } }
  });
}
}
} catch (error: any) {
  if (error?.code === 'messaging/invalid-registration-token' ||
    error?.code === 'messaging/registration-token-not-registered') {
    await User.findByIdAndUpdate(input.userId, { fcmToken: null });
  }
}

if (input.sendEmail) {
  try {
    const user = await User.findById(input.userId);
    if (user && user.email) {
      await emailService.sendNotificationEmail(
        user.email,
        user.displayName || user.email,
        input.title,
        input.message
      );

      notification.sendEmail = true;
      await notification.save();
    }
  } catch (error) {
    console.error('Failed to send notification email:', error);
  }
}
}
}

```

Notification creation is durable first: the notification is saved in MongoDB before best-effort delivery through Socket.IO, Firebase FCM, and optional email. Delivery failures do not roll back the saved notification.

6.2.13 AI OCR Invoice Extraction

Aspect	Description
Trigger	POST /invoices/ocr with an uploaded invoice image (Buffer + MIME type).
Validation	If Gemini API is not configured (genAI is null), returns a fallback result with confidence: 'LOW' and an error message.
Transaction / State Change	None — stateless extraction, no database writes.

Side Effects	External API call to Gemini 2.5 Flash (Vision) with the image encoded as base64. Retries up to 3 times with exponential backoff on rate limits (429 / quota errors) and linear backoff on transient failures. Response JSON is cleaned, parsed, and normalized.
Result Returned	{ title, invoiceDate, items[], amountTotal, currency, note, confidence, extractionError }. Confidence is 'HIGH' / 'MEDIUM' / 'LOW' as determined by the AI model. Currency defaults to 'VND', date defaults to today if not extracted.

```
static async extractInvoiceFromImage(
  imageBuffer: Buffer,
  mimeType: string
): Promise<any> {
  const todayDate = new Date().toISOString().slice(0, 10);
  if (!genAI) {
    return {
      title: '',
      invoiceDate: todayDate,
      items: [],
      amountTotal: 0,
      currency: 'VND',
      note: null,
      confidence: 'LOW',
      extractionError: 'Gemini API is not configured'
    };
  }

  const MAX_RETRIES = 3;
  const BASE_DELAY_MS = 1500;

  for (let attempt = 1; attempt <= MAX_RETRIES; attempt++) {
    try {
      const model = genAI.getGenerativeModel({ model: 'gemini-2.5-flash'
    });

      const imagePart = {
        inlineData: {
          data: imageBuffer.toString('base64'),
          mimeType
        }
      };

      const result = await model.generateContent([OCR_PROMPT,
imagePart]);
      const responseText = result.response.text() || '';

      if (!responseText) {
        if (attempt < MAX_RETRIES) {
          await this.delay(BASE_DELAY_MS * attempt);
          continue;
        }
        return {
          title: '',
```

```

        invoiceDate: todayDate,
        items: [],
        amountTotal: 0,
        currency: 'VND',
        note: null,
        confidence: 'LOW',
        extractionError: 'No response from Gemini API'
    });
}

const cleaned = this.cleanJsonResponse(responseText);
const parsed = JSON.parse(cleaned.trim());
const amountTotal = this.parseNumber(parsed.amountTotal ??
parsed.totalAmount ?? parsed.total) ?? 0;
const invoiceDate = typeof parsed.invoiceDate === 'string' &&
parsed.invoiceDate.trim().length > 0
    ? parsed.invoiceDate.trim()
    : todayDate;
const currency = typeof parsed.currency === 'string' &&
parsed.currency.trim().length > 0
    ? parsed.currency.trim().toUpperCase()
    : 'VND';
const confidence = typeof parsed.confidence === 'string' &&
parsed.confidence.trim().length > 0
    ? parsed.confidence.trim().toUpperCase()
    : 'LOW';

return {
    title: typeof parsed.title === 'string' &&
parsed.title.trim().length > 0
        ? parsed.title.trim()
        : 'Scanned Invoice',
    invoiceDate,
    items: Array.isArray(parsed.items) ? parsed.items : [],
    amountTotal,
    currency,
    note: typeof parsed.note === 'string' ? parsed.note : null,
    confidence,
    extractionError: null
};
} catch (error: any) {
    const isRateLimit = error?.status === 429 ||
error?.message?.includes('429') || error?.message?.includes('quota') ||
error?.message?.includes('rate');

    if (attempt < MAX_RETRIES) {
        const delayMs = isRateLimit
            ? BASE_DELAY_MS * Math.pow(2, attempt)

```

```

        : BASE_DELAY_MS * attempt;
        await this.delay(delayMs);
        continue;
    }

    return {
        title: '',
        invoiceDate: todayDate,
        items: [],
        amountTotal: 0,
        currency: 'VND',
        note: null,
        confidence: 'LOW',
        extractionError: `OCR Error: ${error.message} ||
String(error)`
    };
}

return {
    title: '',
    invoiceDate: todayDate,
    items: [],
    amountTotal: 0,
    currency: 'VND',
    note: null,
    confidence: 'LOW',
    extractionError: 'OCR failed after max retries'
};
}

```

The OCR flow sends the uploaded invoice image to Gemini Vision, retries transient failures, cleans JSON output, parses invoice fields, normalizes currency/date, and returns a structured result with confidence and error information.

6.3 Important technical problems & solutions

6.3.1 Problem: Concurrency during billing execution

Problem.

In SplitPal, critical financial operations such as creating payment requests, processing wallet transfers, renewing recurring subscriptions, and handling withdrawals all involve concurrent reads and writes on the same set of financial data. When multiple users or system processes access and modify data at the same time, the system faces the risk of race conditions, where the final result depends on the unpredictable execution order of parallel processes.

For example, if two admins create payment requests for the same group at the same time, the system may generate two duplicate sets of transfers for the same group of invoices, leading to incorrect balances and debt records. To address this problem, SplitPal implements a two-layer concurrency control strategy: distributed locks at the application layer and ACID transactions at the database layer.

Solution.

#1: Two-tier distributed lock

The distributed locking mechanism is implemented in the file server/src/util/lock.ts through the `acquireLock` function. This function accepts three parameters: the name of the resource to lock (`name`), the group identifier (`groupId`), and the maximum time-to-live of the lock (`ttlMs`, defaulting to 15 seconds). The logic follows a Redis-first, MongoDB-fallback model: the system always attempts to acquire the lock from Redis first; if Redis is unavailable or the connection fails, the system automatically falls back to using MongoDB as an alternative locking mechanism

```
async function acquireRedisLock(
  name: string,
  groupId?: string,
  ttlMs: number = 15_000
): Promise<LockHandle | null> {
  const redis = getRedis();
  if (!redis) {
    return null;
  }

  const lockKey = buildRedisKey('lock', name, groupId || 'global');
  const lockToken = randomUUID();

  try {
    const result = await redis.set(lockKey, lockToken, 'PX', ttlMs, 'NX');
    if (result !== 'OK') {
      return null;
    }
  }

  return {
    release: async () => {
      const releaseScript = `
        if redis.call('GET', KEYS[1]) == ARGV[1] then
          return redis.call('DEL', KEYS[1])
        else
          return 0
        end
      `;

      try {
```

```

        await redis.eval(releaseScript, 1, lockKey, lockToken);
    } catch (error) {
        console.error('Failed to release Redis lock:', error);
    }
}
};
} catch (error) {
    console.error('Failed to acquire Redis lock, fallback to Mongo lock:',
error);
    return null;
}
}
}

```

Tier 1 - Redis Lock: The system uses the command `redis.set(lockKey, lockToken, 'PX', ttlMs, 'NX')`. The NX (Not eXists) flag ensures that the SET command only succeeds when the lock key does not already exist in Redis - if another process is already holding the lock, the command returns null and the function immediately refuses to grant the lock without waiting. The PX flag sets the expiration time in milliseconds, ensuring the lock is automatically released even if the holding process crashes or fails to release it manually. Each lock is assigned a unique lockToken (a random UUID), and when releasing the lock, the system uses an atomic Lua script executed directly on the Redis server: the script checks whether the current value of the key matches the original token, and only deletes the key if it matches — this approach prevents a process from accidentally releasing a lock created by another process.

```

async function acquireRedisLock(
    name: string,
    groupId?: string,
    ttlMs: number = 15_000
): Promise<LockHandle | null> {
    const redis = getRedis();
    if (!redis) {
        return null;
    }

    const lockKey = buildRedisKey('lock', name, groupId || 'global');
    const lockToken = randomUUID();

    try {
        const result = await redis.set(lockKey, lockToken, 'PX', ttlMs, 'NX');
        if (result !== 'OK') {
            return null;
        }
    }

    return {
        release: async () => {
            const releaseScript = `
                if redis.call('GET', KEYS[1]) == ARGV[1] then

```

```

        return redis.call('DEL', KEYS[1])
      else
        return 0
      end
    `;

    try {
      await redis.eval(releaseScript, 1, lockKey, lockToken);
    } catch (error) {
      console.error('Failed to release Redis lock:', error);
    }
  }
};
} catch (error) {
  console.error('Failed to acquire Redis lock, fallback to Mongo lock:',
error);
  return null;
}
}

```

Tier 2: MongoDB Fallback Lock: When Redis is unavailable, the system uses the locks collection in MongoDB as an alternative locking mechanism. The `acquireMongoLock` function uses the atomic `findOneAndUpdate` operation with `upsert: true` to create or update a lock document. The search condition includes a `$or` filter that checks whether the current lock has expired (`expiresAt <= now`) or does not exist — only when one of these conditions is met can a process be granted the lock. If another process is holding a lock that has not yet expired, the upsert operation triggers a duplicate key error (error code 11000) due to the unique compound index on `(name, groupId)` defined in the schema, and the function returns `null` to refuse the lock. When releasing, the system sets `expiresAt` to the current time so the lock expires immediately. Additionally, a TTL index on the `expiresAt` field allows MongoDB to automatically clean up expired lock documents without manual intervention, completely preventing deadlock scenarios.

```

async function acquireMongoLock(
  name: string,
  groupId?: string,
  ttlMs: number = 15_000
): Promise<LockHandle | null> {
  const now = new Date();
  const expiresAt = new Date(now.getTime() + ttlMs);

  const filter: any = { name };
  if (groupId) {
    filter.groupId = groupId;
  }
  filter.$or = [
    { expiresAt: { $lte: now } },

```

```

    { expiresAt: { $exists: false } }
  ];

  try {
    const lock = await Lock.findOneAndUpdate(
      filter,
      { name, groupId, expiresAt },
      { new: true, upsert: true, setDefaultsOnInsert: true }
    );

    if (!lock) {
      return null;
    }

    return {
      release: async () => {
        await Lock.findByIdAndUpdate(lock._id, { expiresAt: new Date()
});
      }
    };
  } catch (error: any) {
    // Most likely duplicate key => lock already held
    if (error.code === 11000) {
      return null;
    }
    throw error;
  }
}

```

The Lock model schema defines the unique compound index and TTL index:

```

const LockSchema = new Schema<ILock>({
  name: { type: String, required: true, index: true },
  groupId: { type: String, index: true },
  expiresAt: { type: Date, required: true },
}, {
  timestamps: { createdAt: true, updatedAt: false },
  collection: 'locks'
});

// TTL index so expired locks get cleaned up automatically
LockSchema.index({ expiresAt: 1 }, { expireAfterSeconds: 0 });
LockSchema.index({ name: 1, groupId: 1 }, { unique: true });

```

#2 Applying distributed lock in Payment request creation

The distributed locking mechanism is directly applied in the createPaymentRequest function of paymentRequestService. Before performing any operations on invoices and debts, the system calls acquireLock('payment_request', groupId, 10_000) to acquire a lock with a 10-second TTL for the specific group. If the lock cannot be acquired (another admin is currently

creating a payment request for the same group), the system returns an error asking the user to retry. The entire processing flow - including checking for existing open payment requests, retrieving unlocked invoices, running the debt settlement optimisation algorithm, creating the payment request, locking invoices, and creating transfer records - is executed within a MongoDB transaction to ensure atomicity. The lock is always released in the finally block to guarantee it is never held indefinitely.

```
async createPaymentRequest(userId: string, groupId: string):
Promise<PaymentRequestDetailResponse> {
    // Verify user is OWNER or ADMIN
    const membership = await GroupMember.findOne({ groupId, userId, leftAt:
null });
    if (!membership) {
        throw new Error('NOT_GROUP_MEMBER');
    }
    if (membership.role !== 'OWNER' && membership.role !== 'ADMIN') {
        throw new Error('Only OWNER or ADMIN can create payment requests');
    }

    // Acquire short lock to avoid concurrent creations
    const lock = await acquireLock('payment_request', groupId, 10_000);
    if (!lock) {
        throw new Error('Another payment request is being created. Please
retry in a few seconds.');
```

```
    }

    const session = await mongoose.startSession();
    try {
        const result = await session.withTransaction(async () => {
            // Check if there's already an open request (inside txn)
            const existingRequest = await PaymentRequest.findOne({
                groupId,
                status: { $in: ['ISSUED', 'PARTIALLY_PAID'] }
            }).session(session);
            if (existingRequest) {
                throw new Error('There is already an open payment request.');
```

```
            }

            // ... retrieve invoices, run settlement engine, create transfers
            ...

            // Lock all invoices
            await Invoice.updateMany(
                { _id: { $in: invoiceIds } },
                { isLocked: true, status: 'LOCKED', paymentRequestId:
paymentRequest._id.toString() },
                { session }
            );
        });
    }
}
```

```

        return paymentRequest._id.toString();
    });

    return this.getPaymentRequestById(userId, groupId, result!);
} finally {
    await session.endSession();
    await lock.release(); // Always release the lock
}
}

```

#3. Atomic transactions and Optimistic locking in Wallet transfers

In the `verifyOTPAndPay` function of `transferService`, the system combines a MongoDB transaction with optimistic locking through a conditional `findOneAndUpdate`. Rather than simply updating the transfer status, the system places multiple preconditions in the filter: the transfer must be in `PENDING` status, the OTP code must match, and the OTP must not have expired — if any condition is not satisfied (for example, because another process has already completed this transfer), the `findOneAndUpdate` returns null and the transaction is aborted. Similarly, when deducting the sender's balance, the system uses the condition `balance: { $gte: amount }` to ensure sufficient funds at the exact moment of writing — if another transaction has already deducted the balance causing it to fall below the required amount, the operation also returns null and the entire transaction is rolled back. This approach guarantees that neither double-payment nor overdraft can occur even when two OTP verification requests arrive simultaneously.

```

const session = await mongoose.startSession();

try {
    await session.withTransaction(async () => {
        // Optimistic locking: only update if transfer is PENDING, OTP is
        // correct and not expired
        const lockedTransfer = await Transfer.findOneAndUpdate(
            {
                _id: transferId,
                status: 'PENDING',
                otp,
                otpExpiresAt: { $gt: new Date() }
            },
            {
                status: 'COMPLETED',
                paidAt: new Date(),
                otp: null,
                otpVerified: true,
                categoryTagId: categoryTagId ?? null
            },
            { session, new: false }
        );
    });
}

```

```

    if (!lockedTransfer) {
      throw new Error('Transfer already processed or OTP invalid');
    }

    // Deduct sender's balance — only succeeds if balance >= amount
    const fromUserDoc = await User.findOneAndUpdate(
      { _id: transfer.fromUserId, balance: { $gte: amount } },
      { $inc: { balance: -amount } },
      { session, new: false }
    );

    if (!fromUserDoc) {
      throw new Error('Insufficient balance');
    }

    // Credit receiver's balance
    const toUserDoc = await User.findOneAndUpdate(
      { _id: transfer.toUserId },
      { $inc: { balance: amount } },
      { session, new: false }
    );

    if (!toUserDoc) {
      throw new Error('Recipient not found');
    }

    // Reduce corresponding original debts
    if (allocations.length > 0) {
      for (const alloc of allocations) {
        await originalDebtService.reduceDebt(alloc.originalDebtId,
        alloc.allocatedAmount, session);
      }
    } else {
      await originalDebtService.reduceDebtsBetweenUsers(
        transfer.groupId, transfer.fromUserId, transfer.toUserId,
        amount, session
      );
    }
  });
} finally {
  session.endSession();
}

```

#4. Atomic Transactions in Withdrawals

The `verifyOTP` function in `withdrawalService` applies the same pattern: the entire process of OTP verification, withdrawal status update, user balance deduction, and transaction logging is wrapped within a single MongoDB transaction. The `findOneAndUpdate` operation with the

conditions status: 'OTP_SENT', matching OTP, and non-expired OTP serves as an optimistic lock — if two withdrawal requests arrive simultaneously for the same withdrawal, only the first one succeeds because the second will find that the status has already changed to COMPLETED. The balance: { \$gte: amount } condition when deducting funds also prevents overdraft situations.

```
const session = await mongoose.startSession();
try {
  await session.withTransaction(async () => {
    const lockedWithdrawal = await Withdrawal.findOneAndUpdate(
      {
        _id: withdrawalId,
        status: 'OTP_SENT',
        otp,
        otpExpiresAt: { $gt: new Date() }
      },
      {
        status: 'COMPLETED',
        verifiedAt: new Date(),
        otp: null
      },
      { session, new: true }
    );

    if (!lockedWithdrawal) {
      throw new Error('Invalid or expired OTP');
    }

    const userDoc = await User.findOneAndUpdate(
      { _id: withdrawal.userId, balance: { $gte: amount } },
      { $inc: { balance: -amount } },
      { session, new: false }
    );

    if (!userDoc) {
      throw new Error('Insufficient balance');
    }

    await transactionService.createTransaction({
      userId: withdrawal.userId,
      type: TransactionType.WITHDRAWAL,
      amount: amount,
      balanceBefore: Number(userDoc.balance),
      balanceAfter: Number(userDoc.balance) - amount,
      currency: 'VND',
      description: `Withdrawal to ${withdrawal.bankName} -
${withdrawal.accountNumber}`,
      referenceId: withdrawalId,
      referenceType: 'WITHDRAWAL',
```

```

        session
    });
});
} finally {
    await session.endSession();
}

```

#5. Atomic transactions in Subscription renewals

In the `processSingleMember` function of `subscriptionService`, each time the scheduler automatically renews a subscription for a member, the operations of deducting the member's balance, crediting the subscription owner's balance, and updating the next billing date are all performed within a single MongoDB transaction. If any step fails (for example, if the balance is no longer sufficient at execution time because another transaction has just deducted funds), the entire transaction is rolled back and no changes are persisted. Additionally, the function implements an idempotency guard - it checks whether the member has already been charged for the current billing cycle before proceeding - to prevent duplicate charges when the scheduler runs multiple times within the same cycle.

```

const session = await mongoose.startSession();
try {
    session.startTransaction();
    await User.findByIdAndUpdate(userId, { $inc: { balance: -fee } }, {
session });
    await User.findByIdAndUpdate(subscription.createdBy, { $inc: { balance:
fee } }, { session });

    const nextBillingDate = calculateNextBillingDate(
        new Date(memberDoc.nextBillingDate),
        subscription.billingCycle as BillingCycle,
    );
    await SubscriptionMember.findByIdAndUpdate(memberId, {
        nextBillingDate,
        lastChargedAt: now,
        retryCount: 0,
    }, { session });

    await session.commitTransaction();
} catch (err: any) {
    await session.abortTransaction();
    result.reason = err.message;
} finally {
    session.endSession();
}

```

Similarly, when a member accepts a subscription invitation (`respondToInvitation`), the system performs a double balance check: the first check occurs before the transaction begins (a quick pre-validation), and the second check occurs inside the transaction by re-reading from the database with the session - if between the two checks another transaction has deducted the

balance leaving insufficient funds, the system detects this and aborts the transaction with the message "Insufficient balance (race condition)".

```
// server/src/service/subscriptionService.ts - lines 451-484
const session = await mongoose.startSession();
try {
  session.startTransaction();

  // Re-read balance inside transaction to detect race condition
  const memberDoc = await User.findById(userId).session(session);
  if (!memberDoc || memberDoc.balance < fee) {
    await session.abortTransaction();
    throw new Error('Insufficient balance (race condition). Please try
again.');
```

```
    }

    await User.findByIdAndUpdate(userId, { $inc: { balance: -fee } }, {
session });
    await User.findByIdAndUpdate(subscription.createdBy, { $inc: { balance:
fee } }, { session });

    await SubInvitation.findByIdAndUpdate(invitationId, { status: 'ACCEPTED'
}, { session });

    await SubscriptionMember.create([
      {
        subscriptionId: invite.subscriptionId,
        userId,
        amount: fee,
        status: 'ACTIVE',
        joinedAt: now,
        nextBillingDate,
        lastChargedAt: now,
        retryCount: 0,
        categoryTagId: categoryTagId ?? null,
      },
    ], { session });

    await session.commitTransaction();
  } catch (err) {
    await session.abortTransaction();
    throw err;
  } finally {
    session.endSession();
  }
}
```

In summary, SplitPal employs a combination of application-level distributed locks (Redis lock as the primary mechanism, MongoDB lock as fallback) for coarse-grained mutual exclusion, and MongoDB ACID transactions combined with optimistic locking via conditional findOneAndUpdate for fine-grained atomicity. This layered strategy creates a

multi-level defence system against concurrency errors during billing execution: distributed locks prevent duplicate business-level operations (such as creating two payment requests simultaneously), while database transactions ensure that related financial operations (deducting funds, crediting funds, reducing debts, logging transactions) either all succeed or are all rolled back entirely. However, comprehensive integration testing with simulated concurrent requests remains a necessary area for future improvement to confirm the effectiveness of these mechanisms under realistic load conditions.

6.3.2 Billing cycle handling across time zones

Problem.

For recurring subscription billing, relying solely on the server's local timezone can cause the billing date to drift from the real-world context of the group. This issue becomes especially significant when the system is deployed in a different region or when daylight saving time (DST) changes occur.

Solution.

The system establishes the foundation for timezone-aware billing by storing time-related values as Date/UTC-compatible timestamps while also maintaining a dedicated timezone field for each group. This design separates storage time from business time, allowing billing cycles, cutoff dates, and statement periods to be defined according to the group's actual timezone rather than the physical location of the server.

Evidence in code.

The Group model stores an IANA timezone identifier

```
timezone: {  
  type: String,  
  default: 'Asia/Ho_Chi_Minh'  
},
```

The Subscription model stores billing-related timestamps as Date values

```
nextBillingDate: {  
  type: Date,  
  required: true  
},  
lastBilledAt: {  
  type: Date,  
  default: null  
},
```

Result.

Although the current implementation primarily demonstrates the data-layer foundation, this approach prevents hard dependency on server timezone and prepares the system for correct timezone-based billing boundaries in future iterations.

6.3.3 Currency rounding when splitting shared expenses

Problem.

When implementing shared cost allocation across multiple members, a critical technical issue emerged: rounding error. If a total amount is split using floating-point arithmetic, the sum of the rounded shares may no longer match the original amount exactly. In financial systems, even a very small discrepancy can lead to cumulative inconsistencies over time.

For example, if 100,000 VND is divided equally among 3 members, each share is 33,333.33. After rounding, the recomposed total may not equal the original value precisely. From a business perspective, this creates a situation in which the system effectively “loses” or “creates” money due to arithmetic precision rather than actual transactions.

Solution.

To eliminate this issue, the system does not perform the split directly on floating-point values. Instead, it converts the total amount into the smallest currency unit (minor unit) before carrying out the calculation. For VND, the implementation uses `decimals = 0`, meaning the smallest unit is 1 VND. The system then:

- converts the total amount into an integer value;
- uses integer division to calculate the `baseShare`;
- computes the remainder;
- assigns the remainder according to a deterministic rule.
- This approach guarantees that the sum of all allocated shares always matches the original amount exactly, preventing value loss or artificial value creation caused by rounding.

Evidence in code

In the current codebase, this logic is clearly implemented in the subscription cost-sharing flow. The system converts the amount into an integer-based representation, performs integer division, and computes the remainder:

```
// Integer division to avoid rounding errors (VND has no decimals)
const decimals = 0;
const totalMinor = Math.round(data.amount * Math.pow(10, decimals));
const memberCount = groupMembers.length;
const baseShare = Math.floor(totalMinor / memberCount);
const remainder = totalMinor - (baseShare * memberCount);
```

The remainder is then redistributed to ensure that the final total remains unchanged

```
// First member gets base + remainder; others get base
const memberShares = groupMembers.map((gm, index) => ({
  subscriptionId: subscription._id.toString(),
  userId: gm.userId,
  shareAmount: index === 0 ? baseShare + remainder : baseShare,
  status: 'ACTIVE'
})));
```


The same principle is also applied when a subscription amount is updated

```
if (updates.amount && updates.amount !== subscription.amount) {
  const members = await SubscriptionMember.find({
    subscriptionId,
    status: 'ACTIVE',
    userId: { $ne: subscription.createdBy }
  });

  if (members.length > 0) {
    const decimals = 0;
    const totalMinor = Math.round(updates.amount * Math.pow(10,
decimals));

    const memberCount = members.length;
    const baseShare = Math.floor(totalMinor / memberCount);
    const remainder = totalMinor - (baseShare * memberCount);

    for (let i = 0; i < members.length; i++) {
      const newShare = i === 0 ? baseShare + remainder :
baseShare;

      await SubscriptionMember.findByIdAndUpdate(members[i]._id,
{ shareAmount: newShare });
    }
  }
}
```

Result

By using integer-based calculation with remainder distribution, the system ensures that:

- the sum of all shares always equals the original amount;
- no rounding drift accumulates over time;
- financial allocation remains consistent and auditable.

6.3.4 Problem: Foreign currency invoice entry

Problem.

In real-world usage, many invoices are paid in foreign currencies such as USD, JPY, or EUR. If the system only accepts the group's base currency, users must manually convert invoice values before entering them. This is both inconvenient and error-prone, and it also removes the original monetary context of the invoice from the system.

Solution.

The system allows an invoice to retain its actual currency. If the invoice currency differs from the group's baseCurrency, the system automatically retrieves the exchange rate through exchangeRateService, converts the amount into the group currency, and stores both the original value and the converted value. This preserves the source data while keeping the downstream payment flow aligned with the group's financial standard.

Evidence in code.

The Invoice model includes native multi-currency support

```
convertedAmountTotal?: number; // Amount in group baseCurrency (null if
same currency)
exchangeRate?: number;         // Rate used: 1 currency = X baseCurrency
baseCurrency?: string;         // Group baseCurrency at creation time
```

The conversion logic is implemented in createInvoice()

```
// Determine currency and exchange rate
const group = await Group.findById(groupId).session(session);
if (!group) throw new Error('Group not found');
const baseCurrency = group.baseCurrency || 'VND';
const invoiceCurrency = data.currency?.toUpperCase() ||
baseCurrency;

const isForeignCurrency = invoiceCurrency !== baseCurrency;

let exchangeRate: number | null = null;
let convertedAmountTotal: number | null = null;
```

The invoice is then stored with both its original currency and converted amount

```
[invoice] = await Invoice.create([
  {
    groupId,
    title: data.title,
    amountTotal,
    currency: invoiceCurrency,
    convertedAmountTotal: isForeignCurrency ?
convertedAmountTotal : null,
    exchangeRate: isForeignCurrency ? Math.round(exchangeRate!
* 10000) / 10000 : null,
    baseCurrency: isForeignCurrency ? baseCurrency : null,
```

Result.

This design enables the system to store and display values such as 15,000 JPY ($\approx$ 2,550,000 VND) while preserving both the original invoice data and the normalized value required for internal financial processing.

6.3.5 Exchange rate lock at debt creation time

Problem.

If an invoice is created on one date but the payment request is generated later, the exchange rate may have changed in the meantime. If the system uses the later rate during payment request creation, the final debt value no longer reflects the actual value of the expense at the moment it was incurred.

Solution.

To guarantee transparency and fairness, the system locks the exchange rate at the moment OriginalDebt is created, rather than waiting until a later settlement phase. Each debt record therefore stores:

- the original foreign currency;
- the debt amount in that original currency;
- the exchange rate used;
- the timestamp at which the rate was locked.

The debt itself is still normalized into the group's baseCurrency, so the existing payment flow remains unchanged.

Evidence in code.

The OriginalDebt model contains dedicated exchange-rate-lock fields

```
// Exchange rate lock (set when invoice currency !== group baseCurrency)
originalCurrency?: string;           // Invoice currency (e.g. 'JPY')
originalAmountInCurrency?: number;  // Debt amount in invoice currency
exchangeRateUsed?: number;          // Locked rate: 1 originalCurrency = X
baseCurrency
rateLockedAt?: Date;                // When the rate was locked
```

The rate-lock logic is applied during debt creation

```
// Create original debts (amounts always in baseCurrency)
const now = new Date();
const debts = Array.from(debtMap.entries()).map(([debtorId,
amountInCurrency]) => {
  const amountInBase = isForeignCurrency
    ? Math.round(amountInCurrency * exchangeRate! * 100) /
100
    : Math.round(amountInCurrency * 100) / 100;
  const amountInCurrencyRounded =
Math.round(amountInCurrency * 100) / 100;

  return {
    groupId,
    invoiceId: invoice._id.toString(),
    debtorId,
    creditorId: userId,
    originalAmount: amountInBase,
    remainingAmount: amountInBase,
    // Exchange rate lock (only for foreign currency)
    ...(isForeignCurrency ? {
      originalCurrency: invoiceCurrency,
      originalAmountInCurrency: amountInCurrencyRounded,
      exchangeRateUsed: Math.round(exchangeRate! *
10000) / 10000,
      rateLockedAt: now
    } : {})
  };
});
```

```
});
```

Result.

This approach ensures that once a foreign-currency debt is created, its base-currency value remains stable over time. Consequently, later payment requests do not alter the underlying debt due to exchange-rate fluctuations, which improves auditability and prevents disputes between creditors and debtors.

6.4 Test

6.4.1 Test Strategy

The testing strategy was divided into four main evidence types.

Evidence Type	Description	Used for
Source review	Reviewing route, controller, service, validation, and database update logic in the codebase	Business rules, authorization checks, transaction handling, and fallback logic
API/manual test	Sending API requests and checking returned responses or database records	Authentication, group management, invoice creation, balance viewing, payment requests, wallet operations
Automated test	Running backend test scripts to verify specific service logic	Subscription billing and repeated rule-based checks
Environment-limited test	Verifying internal generation or processing logic when external runtime confirmation is limited	Email OTP delivery, VNPay callback verification, push notification delivery

The result status was interpreted as follows.

Status	Meaning
Passed	The tested behaviour was verified through source review, API/manual test, automated test, or database inspection
Partially Passed / Environment Limited	The internal logic worked, but full runtime proof depended on an external service or production-like environment
Not Passed	The expected behaviour failed or could not be verified at all
Planned / Requires further validation	The test is identified as necessary but requires additional real users, production-like setup, or automation

6.4.2 Functional and API Test Results

ID	Test Case	Input	Description	Priority	Expected Result	Actual Result	Status
----	-----------	-------	-------------	----------	-----------------	---------------	--------

AUTH-001	Register new account	POST /api/auth/signup; body: email, password, displayName	Register a new user account	High	Account created as inactive and OTP is sent	Source verifies inactive user creation and OTP sending logic	Passed
AUTH-002	Verify registration OTP	POST /api/auth/verify-otp; body: email, otp	Activate account using OTP	High	Account status becomes active	Source verifies OTP and updates user status to active	Passed
AUTH-003	Login with email/password	POST /api/auth/login; body: email, password	Authenticate user	High	JWT token and user information returned	Source verifies bcrypt password comparison and JWT generation	Passed
AUTH-004	Login with 2FA enabled	POST /api/auth/login, then POST /api/auth/2fa/verify; body: tempToken, token	Complete login with TOTP	High	Temporary token is returned first, then full JWT after valid 2FA	Source verifies temporary 2FA token and TOTP validation flow	Passed
AUTH-005	Access protected route without token	Any protected API without Authorization: Bearer token	Validate API protection	High	Request rejected with unauthorized response	Source verifies authMiddleware rejects missing/invalid token	Passed
GROUP-001	Create group	POST /api/groups; body: name, baseCurrency, timezone, optional description	Create group workspace	High	Group created and requester becomes owner	Source verifies group and owner membership creation	Passed
GROUP-002	Invite group member	POST /api/groups/:groupId/ invites; body: emailInvite, role	Invite user to group	High	Invite created by owner/admin	Source verifies owner/admin role check before invite creation	Passed
GROUP-003	Reject non-member group access	GET /api/groups/:groupId; token of non-member	Prevent unauthorized group access	High	Request rejected	Source verifies active GroupMember check	Passed
INVOICE-001	Create invoice	POST /api/invoices/:groupId; body: title, currency, invoiceDate, items, splits	Create invoice and debts	High	Invoice, invoice items, and original debt records created	Source verifies invoice creation, split validation, and debt creation	Passed
INVOICE-002	Create invoice with non-member participant	POST /api/invoices/:groupId; body includes non-member user ID	Prevent invalid debt assignment	High	Request rejected	Source verifies assigned users must be group members	Passed

INVOICE-003	View group balance	GET /api/groups/:groupId/balance	Calculate member balances	High	Group balances returned	Source verifies balance calculation from debt records	Passed
PAYMENT-001	Create payment request	POST /api/payment-requests/:groupId	Generate payment request from submitted invoices	High	Payment request and transfer records created	Source verifies owner/admin check, invoice locking, and transfer generation	Passed
PAYMENT-002	Prevent duplicate active payment request	POST /api/payment-requests/:groupId when active request exists	Avoid duplicate settlement	High	Duplicate active request is rejected	Source verifies existing active request check	Passed
TRANSFER-001	Initiate transfer payment	POST /api/transfers/:transferId/pay	Generate payment OTP	High	OTP generated and sent to email	Source verifies OTP generation, but email delivery runtime evidence is not available	Not Pass (Generated Email)
TRANSFER-002	Verify transfer OTP and pay	POST /api/transfers/:transferId/verify-otp; body: otp, optional categoryId	Complete wallet transfer	High	Transfer completed, balances updated, debts reduced	Source verifies OTP, status, balance checks and MongoDB transaction update	Passed
WALLET-001	View wallet balance	GET /api/accounts/balance	Retrieve wallet balance	High	User balance and currency returned	Source verifies authenticated account balance route exists	Passed
WALLET-002	Top up wallet with VNPay	POST /api/accounts/top-up or POST /api/payments/topup; body: amount, returnUrl	Create VNPay top-up flow	High	Signed VNPay payment URL returned	Source verifies VNPay URL signing, but VNPay runtime callback is not verified	Not Pass (Generated Email)
WITHDRAW-001	Initiate withdrawal	POST /api/withdrawals; body: amount	Create withdrawal request with OTP	High	Withdrawal created and OTP sent	Source verifies withdrawal creation, but email delivery runtime evidence is not available	Not Pass (Generated Email)
WITHDRAW-002	Verify withdrawal OTP	POST /api/withdrawals/:withdrawalId/verify-otp; body: otp	Complete withdrawal	High	Balance deducted and transaction recorded	Source verifies OTP validation and transactional balance deduction	Passed
SUB-001	Process subscription billing	Existing due subscription members	Validate subscription charging logic	High	Due subscriptions	Automated backend billing	Passed

					processed correctly	test passed 26 checks	
TEMP LATE-001	Generate recurring bill	POST /api/groups/:groupId/bill-templates/:templateId/generate-now	Generate invoice from bill template	High	Draft invoice generated	Source verifies route and recurring bill generation service	Passed
NOTIF-001	Create notification	Trigger payment/subscription/group event	Store and deliver notification	High	Notification saved and delivery attempted	Source verifies MongoDB notification creation, Socket.IO emit, and FCM attempt	Passed
CHAT-001	Send group message	POST /api/chat/groups/:groupId/messages; body: content	Send group chat message	Medium	Message saved and returned	Source verifies chat API route and group message service	Passed
BUILD-001	Backend build	npm run build inside server	Compile backend TypeScript	High	Build succeeds	Backend build completed successfully	Passed
TEST-001	Backend billing tests	npm run test:billing inside server	Run subscription billing tests	High	All billing checks pass	26 backend billing checks passed	Passed

6.4.3 Financial Workflow and Traceability Testing

Financial workflow testing focused on whether SplitPal could preserve a clear chain of records across money-related actions. This is important because the system does not only store a final balance. It must also explain where a debt came from, which payment request included it, which transfer settled it, and how the wallet balance changed after settlement.

Workflow Checked	Records Verified	Verification Purpose	Result
Invoice creation	Invoice and invoice item records	Confirms that the original expense was stored correctly before debt generation	Passed
Debt generation	OriginalDebt records	Confirms who owes whom, the original amount, remaining amount, and debt status	Passed
Payment request creation	PaymentRequest and Transfer records	Confirms that submitted invoices can be locked and converted into settlement transfers	Passed
Transfer settlement	Transfer, TransferDebtAllocation, User balance, and Transaction records	Confirms that payment affects both debt status and wallet balance history	Passed
Withdrawal completion	Withdrawal, User balance, and Transaction records	Confirms that balance deduction is recorded with an auditable transaction entry	Passed
Subscription billing	Subscription, subscription member, billing, and related invoice records	Confirms that recurring shared expenses can be generated and charged by schedule	Passed
Multi-currency handling	Exchange rate fields and converted debt values	Confirms that exchange rates are locked at debt creation time instead of being recalculated later	Passed

6.4.4 External Service and Environment-limited Testing

External Service	Tested Function	Evidence	Result	Interpretation
SMTP / Nodemailer	OTP email for registration, transfer, and withdrawal	OTP generation and email sending logic were verified	Partially Passed / Environment Limited	The backend logic works, but inbox delivery is dependent on email service configuration
VNPay	Wallet top-up flow	Signed VNPay payment URL was generated	Partially Passed / Environment Limited	Payment URL generation is verified, but full callback processing requires sandbox or production-like runtime confirmation
Firebase Cloud Messaging	Push notification delivery	Notification record, Socket.IO emit, and FCM attempt were verified	Passed with environment limitation	Internal notification creation works, but real device delivery may vary by Firebase configuration
Gemini AI	Receipt OCR support	AI extraction flow and fallback handling were reviewed	Passed with limitation	AI output is treated as assistive data and still requires user review before financial records are created
Redis	Locking and fallback support	Redis-first locking and MongoDB fallback logic were reviewed	Passed	The design supports concurrency control even when Redis is unavailable in local testing

This distinction is important because an external-service limitation does not mean that the implemented feature is broken. It means that the internal system behaviour has been verified, while final runtime proof requires a more complete deployment or third-party sandbox setup.

6.4.5 Usability Feedback from trial users

At the current stage, formal usability testing with a large user group has not been completed. However, a preliminary usability review was considered necessary because SplitPal contains workflows that require user understanding, not only backend correctness. Users need to understand why they owe money, how debts are generated, where payment requests come from, and how wallet transactions can be checked.

The following table presents a limited usability walkthrough format that can be used to record feedback from trial users. The observations focus on practical interaction issues rather than statistical usability measurement.

Review Focus	Tested Workflow	Positive Observation	Issue Observed	Improvement Suggestion
First-time user flow	Register, verify OTP, login, create group	The account and group creation flow is generally understandable because each action is separated into clear steps	Users may not immediately understand why an inactive account needs OTP verification before login	Add a short message explaining that OTP verification protects the account before activation

Expense entry	Create invoice, add items, select participants, choose split method	Users can understand the basic idea of creating an invoice and assigning members to shared items	The difference between equal, percentage, weight, and custom split may require extra explanation for new users	Add short helper text or examples under each split method
Debt transparency	View group balance and generated debts	Users can see who owes whom after invoice creation, which supports financial transparency	Some users may want to see the original invoice directly from the balance or debt screen	Add a link from each debt item to the related invoice detail
Payment request	Create or view payment request	The payment request flow helps group admins organize settlement instead of asking members manually	Users may not understand why submitted invoices are locked after payment request creation	Add a confirmation warning before creating a payment request
Wallet and transaction history	Check balance, transfer, withdrawal, and transaction history	Transaction history helps users check how balance changes after each financial action	Users may need clearer labels for transaction types such as top-up, transfer, withdrawal, and refund	Use clearer transaction labels and add filters by transaction type

6.4.6 Benchmark testing

Benchmark testing was conducted to evaluate the baseline responsiveness and reliability of the backend API endpoints under sequential request conditions. The test was performed against the locally hosted server at <http://localhost:8080>, with 50 sequential HTTP requests issued to each selected endpoint. The benchmark covered eight critical endpoints representing key system functions, including health checking, authentication, group management, dashboard aggregation, invoice retrieval, and balance calculation.

Overall, all tested endpoints achieved a 100% success rate, indicating that no request failures or server-side errors occurred during the sequential benchmark execution. This confirms that the backend remained functionally stable under controlled sequential load conditions.

Endpoint	Main Function	Average Response Time	P95 Latency	Performance Tier	Interpretation
GET /health	Health check	0.5 ms	< 100 ms	Fast	Very lightweight endpoint with minimal processing.
GET /api/auth/me	Retrieve authenticated user	84.9 ms	< 100 ms	Fast	Performs efficiently under sequential requests.
POST /api/auth/login	User authentication	Not specified	162.9 ms	Moderate	Involves authentication logic and database validation.
GET /api/groups	Retrieve user groups	Not specified	346.8 ms	Moderate	Requires moderate database access.
GET /api/groups/:id/members	Retrieve group members	Not specified	285.8 ms	Moderate	Performs acceptably with relational group-member queries.

GET /api/dashboard/home	Dashboard aggregation	Not specified	1,114.7 ms	Slow	High latency due to complex aggregation logic.
GET /api/invoices/:groupId	Retrieve group invoices	Not specified	863.0 ms	Slow	Slower response caused by invoice retrieval and filtering.
GET /api/groups/:id/balance	Calculate group balance	Not specified	586.8 ms	Slow	Requires financial calculation and aggregation across records.

To support clearer performance evaluation, the endpoints were classified into three tiers based on their 95th-percentile response time. The fast tier included endpoints with P95 latency below 100 ms. This group consisted of the health-check endpoint and the authenticated-user endpoint, both of which involve relatively simple processing and limited database interaction.

The moderate tier included endpoints with P95 latency between 100 ms and 500 ms. These endpoints, such as login, group listing, and group-member retrieval, involve authentication logic or moderate database queries. Their response times remain acceptable for normal backend operations.

The slow tier included endpoints with P95 latency equal to or greater than 500 ms. This category consisted of the dashboard home endpoint, invoice listing endpoint, and group-balance endpoint. The elevated latency in these endpoints is mainly attributable to complex aggregation operations, particularly where multiple database collections are joined, filtered, or calculated simultaneously.

Endpoint	Main Function	Average Response Time	P95 Latency	Performance Tier	Interpretation
GET /health	Health check	0.5 ms	< 100 ms	Fast	Very lightweight endpoint with minimal processing.
GET /api/auth/me	Retrieve authenticated user	84.9 ms	< 100 ms	Fast	Performs efficiently under sequential requests.
POST /api/auth/login	User authentication	Not specified	162.9 ms	Moderate	Involves authentication logic and database validation.
GET /api/groups	Retrieve user groups	Not specified	346.8 ms	Moderate	Requires moderate database access.
GET /api/groups/:id/members	Retrieve group members	Not specified	285.8 ms	Moderate	Performs acceptably with relational group-member queries.
GET /api/dashboard/home	Dashboard aggregation	Not specified	1,114.7 ms	Slow	High latency due to complex aggregation logic.
GET /api/invoices/:groupId	Retrieve group invoices	Not specified	863.0 ms	Slow	Slower response caused by invoice retrieval and filtering.

GET /api/groups/:id/balance	Calculate group balance	Not specified	586.8 ms	Slow	Requires financial calculation and aggregation across records.
--------------------------------	-------------------------	---------------	----------	------	--

The benchmark results indicate that most lightweight and moderately complex endpoints meet acceptable responsiveness expectations. However, aggregation-heavy endpoints require further optimisation before production deployment. Recommended improvements include adding compound database indexes on frequently queried fields, optimising MongoDB aggregation pipelines, and introducing Redis-based response caching for read-heavy endpoints such as dashboard and balance-calculation APIs.

6.4.7 Performance testing

In addition to endpoint-level benchmark testing, a broader set of performance tests was conducted to assess the backend system under concurrent, sustained, stressful, and sudden traffic conditions. These tests focused on four key dimensions: concurrency safety, load endurance, stress tolerance, and spike resilience.

Concurrency and race-condition testing was performed to verify whether the backend could preserve data integrity under parallel access. Four concurrency scenarios were executed, and all four passed successfully.

Test Scenario	Test Configuration	Result	Interpretation
Double-spend prevention	10 simultaneous payment requests against the same balance	1 succeeded, 9 rejected	Confirms that the distributed-lock mechanism prevents duplicate financial transactions.
Concurrent invoice creation	10 parallel invoice creation requests	0 HTTP 5xx errors	Demonstrates that Mongoose transactions maintain consistency during simultaneous writes.
Concurrent dashboard reads	30 simultaneous GET requests to dashboard endpoint	All returned HTTP 200 within 3,128 ms	Confirms that concurrent read requests can be handled without functional failure.
Mixed read-write workload	30 simultaneous read and write operations	0 server errors	Indicates that mixed traffic does not cause mutual degradation.

The double-spend prevention test was particularly important because it verified the reliability of the system's financial transaction logic. Only one payment request was allowed to succeed, while the remaining requests were rejected correctly. This confirms that the Redis-based distributed-lock mechanism, supported by MongoDB fallback, provides effective protection against duplicate financial operations.

Load, stress, and spike testing were configured using Artillery, a load-testing framework used to simulate real-world API traffic. These tests were designed to evaluate how the backend behaves under gradually increasing, sustained, and sudden traffic conditions.

Test Type	Traffic Pattern	Purpose
-----------	-----------------	---------

Load test	5 rps during a 60-second warm-up, then 20 rps sustained for 60 seconds	To evaluate steady-state behaviour under expected operating load.
Stress test	Increased from 10 rps to 100 rps over 240 seconds, then held at 100 rps for 60 seconds	To identify the system's degradation point under heavy traffic.
Spike test	Alternated between 5 rps baseline and sudden 80 rps surges, including two 15-second spikes	To assess whether the system can absorb sudden traffic increases and recover afterwards.

The load test demonstrated the system's behaviour under expected operating conditions. The stress test was used to determine how the backend responds when traffic increases beyond normal usage levels. Meanwhile, the spike test evaluated the server's ability to handle abrupt traffic surges and return to baseline operation after short-term demand peaks.

The results were also evaluated against predefined Service Level Agreement thresholds. The system successfully met the sequential error-rate target and passed all concurrency-safety requirements. However, the response-time targets were not fully achieved for the dashboard endpoint.

SLA Metric	Target	Actual Result	Status	Interpretation
Sequential error rate	0%	0%	Met	No errors occurred during sequential benchmark testing.
Concurrency safety	All critical scenarios passed	4/4 scenarios passed	Met	Financial and write operations remained consistent under concurrent access.
Dashboard P95 response time	< 500 ms	1,114.7 ms	Not met	Dashboard aggregation requires optimisation.
Dashboard P99 response time	< 1,000 ms	1,131.8 ms	Not met	Tail latency is above the acceptable production threshold.

Although the backend remained functionally stable, the dashboard endpoint did not meet the required latency thresholds. This suggests that the main performance bottleneck lies not in request failure or concurrency handling, but in aggregation-heavy read operations.

Based on these findings, several optimisation actions are recommended.

Issue Identified	Recommended Improvement	Expected Benefit
Slow dashboard aggregation	Add compound indexes on frequently queried fields such as Invoice.group + status and GroupMember.group + user	Reduces database lookup and filtering time.
High latency in read-heavy endpoints	Implement Redis caching for dashboard and balance-calculation endpoints	Improves response time for repeated queries.
Potential connection bottlenecks under load	Tune MongoDB connection-pool size	Improves concurrent request handling.
Production traffic exceeding 50 rps	Deploy multiple server instances behind a load balancer	Improves scalability, availability, and fault tolerance.
Complex aggregation pipelines	Review and optimise MongoDB aggregation logic	Reduces processing time for dashboard and balance calculations.

Overall, the performance testing results show that the backend is stable under sequential and concurrent conditions, with strong data-integrity protection for financial transactions. However, optimisation is required for aggregation-heavy endpoints, especially the dashboard

and balance-calculation APIs, to ensure that the system can consistently meet production-level latency targets under higher traffic volumes.

6.5 Results

This section summarises the main results achieved after the implementation and testing stages of the SplitPal system. The purpose of this section is not to evaluate the overall strengths and weaknesses of the application, as this is addressed later in the evaluation section. Instead, this section focuses on the factual outcomes of the project, including the implemented features, tested workflows, technical integration results, and the remaining limitations observed during development.

6.5.1 Implementation results

After the development stage, SplitPal successfully implemented most of the core functions defined during the requirement analysis phase. The system was not limited to basic expense recording, but was developed as a more structured group financial management application with multiple connected workflows.

First, the authentication and account management module was implemented to allow users to register, log in, manage profile information, and access the system securely. This module provides the foundation for linking each financial action to a specific user, which is important for transaction traceability and accountability.

Second, the group management module was completed to support group creation, member management, role control, and invitation-related actions. This module acts as the operational context for the remaining financial features, because invoices, debts, payment requests, subscriptions, and notifications are all connected to specific user groups.

Third, the invoice and expense splitting module was implemented as one of the main functions of the system. Users can create invoices, add invoice items, select the original payer, choose participating members, and apply different cost allocation methods. These splitting methods allow the system to support more realistic expense-sharing situations, such as equal splitting, custom amount splitting, and ratio-based allocation.

A key result of the project is the creation of a financial data flow after an invoice is recorded. From the original invoice, the system can generate debts, track outstanding amounts, create payment requests, record transfers, and update transaction history. This means that SplitPal does not only store expense information, but also manages the lifecycle of a shared expense from creation to settlement.

In addition to the core financial functions, SplitPal also implemented several extended features, including internal wallet management, transaction history, real-time notifications, shared subscription management, and AI-assisted receipt extraction. These features help the system move beyond the scope of a simple bill-splitting application and demonstrate its potential as a broader group finance management platform.

6.5.2 Functional testing results

The main functional workflows of SplitPal were tested through API testing and direct application testing. The results show that most core workflows operated according to the expected business logic defined in the system design.

Test area	Main workflows tested	Result
Authentication and account	User registration, login, OTP verification, profile update	Passed
Group management	Create group, view group list, manage members, update roles	Passed
Invoice management	Create invoice, add invoice items, select participants, apply splitting method	Passed
Debt generation	Generate debts from invoice, identify debtor and creditor, calculate outstanding amount	Passed
Payment request	Create payment request, validate debt status, prevent invalid request	Passed
Wallet and transaction	Record transfer, update balance, save transaction history	Passed
Subscription management	Create subscription, assign member slots, manage renewal cycle	Passed
Notification	Send group, payment, and system-related notifications	Passed
AI receipt extraction	Upload receipt image and extract initial invoice data	Passed

The testing results indicate that the main workflows, such as group creation, invoice creation, expense splitting, debt generation, and transaction tracking, functioned correctly at the prototype level. These results directly support the original objective of the project, which was to build a more transparent, structured, and traceable solution for group expense management.

However, some advanced workflows still require further testing. Payment processing, subscription renewal, wallet operations, and AI receipt extraction depend on external services or more complex input conditions. Although these workflows were implemented and tested at a functional level, they still require additional testing under abnormal data, unstable network conditions, concurrent user actions, and repeated financial requests.

6.5.3 Technical integration results

From a technical perspective, the main components of the system were successfully integrated according to the designed architecture. The Flutter mobile application functions as the user interaction layer, allowing users to manage groups, create invoices, view debts, send payment requests, and receive notifications. The frontend was able to communicate with the backend through REST APIs and display updated financial data from the system.

The backend was developed using Node.js, Express, and TypeScript. It acts as the central business logic layer responsible for authentication, group management, invoice processing, debt generation, payment request handling, transaction recording, subscription management, and notification processing. TypeScript helped improve control over data structures across modules, especially in financial workflows where invoices, debts, transfers, and transactions are closely related.

MongoDB was used as the main database for storing users, groups, group members, invoices, invoice items, debts, payment requests, transactions, notifications, and subscriptions. This document-based approach was suitable for SplitPal because several data structures are flexible and may evolve as the product scope expands. Mongoose was also used to provide schema definitions, validation rules, indexing, and clearer model organisation, which helped reduce the risks associated with unstructured data.

Redis was integrated to support temporary and high-speed operations, such as OTP storage, caching, rate limiting, and short-lived locking for sensitive operations. This helped improve system responsiveness and reduced the risk of duplicated processing in selected financial workflows.

External services were also integrated within the scope of the prototype. Firebase Cloud Messaging was used to support push notifications, VNPay was integrated for payment-related workflows in a testing environment, and AI-assisted receipt extraction was implemented to reduce manual input when users upload receipt images. These integrations demonstrate that the system can coordinate multiple services to support a more complete group finance experience.

6.5.4 Results compared with the initial objectives

Compared with the initial objectives, SplitPal achieved most of the core functional scope of a group expense management application. The system supports group creation, expense recording, flexible cost splitting, debt tracking, payment request handling, and transaction history. These features form the foundation needed to prove the feasibility of the proposed solution.

More importantly, the system expanded beyond traditional bill-splitting by introducing internal wallet operations, subscription slot management, notification support, and AI-assisted receipt processing. These results show that SplitPal can handle more complex group finance scenarios, such as shared living groups, recurring digital subscriptions, and repeated expense settlement among members.

Nevertheless, the current result should be understood as an academic prototype rather than a commercial-ready financial product. Several features have demonstrated technical feasibility, but further work is still required in clean architecture, automated testing, scalability validation, security hardening, and production-level monitoring before the system can be deployed in a real operating environment.

Requirement area	What the requirement asks for	How SplitPal handles it	Implementation evidence	Test / proof level
Authentication & account security	Users must register, verify email, log in securely, use JWT, recover password, and optionally enable 2FA	Backend controls authentication, issues JWT for protected APIs, uses OTP/TOTP for sensitive account flows	Auth API, auth service, 2FA controller, auth middleware, Flutter auth screens	Strong: AUTH-001 to AUTH-005
Group management & role control	Users need shared groups with	Groups are the main financial workspace;	Group API, group service, Group,	Strong: GROUP-001 to GROUP-003

	owner/admin/member permissions	owner/admin can manage members and roles while normal users have limited permissions	GroupMember, Invite, Flutter group pages	
Invoice creation & expense splitting	Users must create invoices, add items, choose participants, and split costs fairly	Backend validates group membership and split rules, then converts invoices into traceable original debts	Invoice API, invoice service, original debt service, invoice/debt models, Flutter invoice pages	Strong: INVOICE-001 to INVOICE-003
Debt tracking & payment request	Users must see who owes whom and create settlement requests from unpaid debts	System links invoice → original debt → payment request → transfer → debt allocation, giving a clear financial chain	Payment request service, transfer service, debt settlement engine, PaymentRequest, Transfer, TransferDebtAllocation	Strong/partial: PAYMENT-001, PAYMENT-002, TRANSFER-002; OTP email generation is limited in TRANSFER-001
Wallet, top-up, withdrawal & transaction history	Users need wallet balance, top-up, withdrawal, transfer payment, refund and transaction records	Wallet operations update user balances and store transaction history; VNPay is integrated for payment/top-up flow	Account API, VNPay API, withdrawal API, transaction API, wallet/transaction models	Partial: WALLET-001, WALLET-002, WITHDRAW-001, WITHDRAW-002; VNPay callback/email evidence is limited
Subscription & recurring billing	Users need shared subscriptions, member billing, renewal, billing history and failure handling	Subscription service charges members, records billing history, handles failed billing and recurring cycles	Subscription API, billing service, scheduler, Subscription, SubscriptionMember, BillingHistory	Strong: SUB-001, TEST-001; automated billing script passed 26 checks
Receipt, budget & forecast	Users need personal receipt tracking, budget summaries and cashflow risk visibility	Receipt and tag data feed budget/forecast features; forecast estimates future balance and risk alerts	Receipt API, budget API, forecast API, Flutter receipt and forecast pages	Partial: Flutter receipt model tests exist; no named test found for budget/forecast/risk alerts
AI/OCR invoice assistance	Users should reduce manual input through AI-assisted extraction	AI suggests invoice data from text/image, but output remains assistive and must be reviewed by users	AI API, AI service, OCR upload route, Flutter AI/OCR widgets	Partial: Section 6.5.2 says AI receipt extraction passed; no named Section 6.4 AI test ID
Notification & chat	Users need reminders, status updates and group communication	Backend stores notifications, emits realtime events, supports FCM/push and group chat	Notification API, chat API, Socket.IO setup, FCM service, Flutter notification/chat pages	Strong: NOTIF-001, CHAT-001
Reliability, security & auditability	Financial operations must be protected, validated, traceable and resistant to duplicate processing	Middleware protects APIs; rate limits reduce abuse; locks/transactions protect sensitive financial operations; transaction records support audit trail	Auth middleware, rate limit middleware, login guard, Redis/lock utility, MongoDB transactions in financial services	Partial/strong: AUTH-005, BUILD-001, TEST-001

Instead of treating the requirements as isolated features, SplitPal implements them as connected financial workflows. The table below shows how each requirement area is addressed through design decisions, backend/frontend implementation, and available testing evidence. This helps demonstrate that the project follows a controlled development process from requirement analysis to design, implementation, and verification.

6.5.5 Observed limitations from the implementation results

The implementation results also revealed several limitations. First, automated test coverage remains limited. The current testing process focused mainly on functional testing and API testing, while unit testing, integration testing, concurrency testing, and security testing were not yet covered in sufficient depth.

Second, complex financial workflows require more extensive validation. Processes such as duplicate payment handling, concurrent balance updates, subscription renewal, refund processing, withdrawal approval, and debt allocation may involve multiple edge cases. Without deeper testing, these workflows may create financial inconsistencies when the system is used in a real multi-user environment.

Third, the AI receipt extraction feature still depends heavily on the quality of the uploaded image. If the receipt is blurred, poorly lit, incomplete, or captured from an unsuitable angle, the extracted result may not be fully accurate. Therefore, this feature should currently be treated as an input-support function rather than a complete replacement for user verification.

Finally, the system has not yet been evaluated under a production-scale environment. The current results mainly reflect system behaviour in development and testing conditions. To become production-ready, SplitPal would need additional load testing, security testing, centralised logging, monitoring, backup strategy, and a clearer operational error-handling process.

6.5.6 Summary of results

Overall, the implementation results show that SplitPal successfully achieved the main goal of the project: developing a group expense management system that can record invoices, split expenses flexibly, generate debts, support payment-related workflows, manage transactions, and handle extended scenarios such as shared subscriptions and AI-assisted receipt extraction.

The main modules were integrated into a working prototype involving the mobile frontend, backend APIs, database layer, cache layer, and external services. Although several limitations remain in terms of automated testing, production stability, and advanced financial edge cases, the current result proves the technical feasibility and practical value of the proposed solution. This provides the basis for the following evaluation section, where the strengths, weaknesses, lessons learned, and future improvements of SplitPal are discussed in more detail.

7 Evaluation and conclusion

7.1 Evaluation of results

Pros of application

The deployment results indicate that SplitPal has evolved significantly beyond a basic UI prototype into a comprehensive expense management system. It covers both end-user interactions and complex backend business logic. Looking at the current codebase, the system handles core functionalities: email OTP authentication, two-factor authentication (2FA), group management, flexible invoice splitting, payment requests, transfers, internal wallets, recurring subscriptions, and real-time features via Socket.IO and FCM. It even integrates AI for receipt data extraction. This scope pushes the application far beyond a standard CRUD assignment.

A major strength lies in its deep financial domain modeling. Instead of simply tracking "who owes whom," the system isolates entities like Invoice, OriginalDebt, PaymentRequest, Transfer, TransferDebtAllocation, Subscription, and Transaction. This structure ensures strict data traceability. The system can pinpoint exactly who owes what, which invoice a debt belongs to, and how the actual cash flow is recorded. Implementing features like invoice locking after payment requests and FIFO (First-In-First-Out) debt allocation demonstrates a solid grasp of edge cases that typically corrupt financial data.

Additionally, the application handles shared expenses with high flexibility. The backend supports equal, percentage, weight, and custom amount splits. Tackling multi-currency transactions by locking exchange rates during invoice or debt creation is a highly practical technical decision. It prevents disputes caused by fluctuating rates over time and maintains historical data integrity.

On the operational side, the project balances user experience with system infrastructure. The Flutter app features a clean dashboard, group tabs, receipt calendars, and logical navigation. The Node.js server goes beyond simple REST APIs by incorporating Redis caching, concurrency locks to prevent race conditions, background schedulers for subscriptions, and WebSockets for real-time updates.

Integrating real-world services like the Gemini API, VNPAY, Firebase, and SMTP-complete with fallback mechanisms-shows maturity in system design. This brings the application much closer to a production-ready environment rather than just an academic illustration.

Cons of application

Despite these successes, several limitations remain. The primary issue is architectural. In the Flutter frontend, state classes often make direct API calls via DioClient, leading to "fat providers." While this speeds up initial development, it blurs the line between the presentation layer and business logic. Over time, this will complicate unit testing, code reuse, and maintenance. Similarly, the backend contains monolithic service files handling multiple responsibilities, increasing code coupling and making the system harder to navigate.

Another significant gap is automated testing. Given the system's complexity, the current coverage is insufficient. The Flutter side has basic widget and model tests, but the backend

lacks comprehensive integration or end-to-end (E2E) tests for critical financial flows-like OTP verification, transfer processing, or subscription renewals. Without this safety net, regression bugs are a real risk as the codebase expands.

The AI/OCR module also presents challenges. While it greatly enhances the user experience when Gemini performs well, the fallback regex methods struggle with accuracy. Test scripts indicate that it sometimes misses items or miscalculates totals. This shows that the AI feature is currently best suited as a data-entry assistant rather than a fully autonomous tool in contexts requiring strict financial accuracy.

Finally, relying heavily on third-party integrations introduces operational overhead. Each dependency requires careful configuration, quota monitoring, and failover handling. To scale reliably, SplitPal needs better monitoring layers, structured logging, and a more robust deployment pipeline.

7.2 Conclusion

Lessons learnt

Building SplitPal highlighted several critical aspects of software engineering. The most prominent lesson is that accurate domain modeling is often more important than UI design, especially in financial software. Failing to correctly define entities like original debts, transfers, and billing histories early on leads to an unscalable system. We also learned that currency handling requires strict attention to rounding, minor units, exchange rate locking, and audit trails. Furthermore, when integrating multiple external services, implementing robust error handling and fallback mechanisms is an absolute necessity, not just an optional add-on.

Problems / difficulties

Business logic complexity was the biggest hurdle. Unlike simple bill-splitting apps, SplitPal coordinates multiple splitting methods, multi-currency conversions, invoice locking, real-time synchronization, and recurring tasks. Maintaining data consistency across all these states is highly challenging. Additionally, managing the full stack-Flutter, Node.js, MongoDB, Redis, external APIs-drastically increased the debugging and configuration workload. Time constraints for this academic project also meant that architectural optimization and test coverage could not reach the desired levels.

Future improvements

In the next development stages, SplitPal should be improved from technical, product, and operational perspectives so that it can evolve from a functional prototype into a more reliable group expense management system. The first priority is to refactor the frontend and backend architecture. On the frontend, the system should adopt Clean Architecture by separating the presentation layer, use cases, repositories, and data sources. This prevents the UI from

depending directly on external APIs and allows key workflows such as invoice creation, debt generation, payment confirmation, and balance updates to be managed more consistently. On the backend, large modules should be decomposed into smaller domain services, such as Invoice Service, Debt Service, Payment Request Service, Transfer Service, and Notification Service. This approach allows each service to focus on a specific responsibility and reduces the risk of complexity when the system expands to support more financial transaction types.

Another important improvement is to increase the reliability of the financial pipeline. SplitPal does not only store expense records; it also generates a financial data chain consisting of invoices, debts, payment requests, transfers, and transactions. Therefore, the system needs cross-validation mechanisms to ensure data consistency. For example, the total allocated amount must match the total value of the invoice, the amount allocated from a transfer must not exceed the transferred amount, and the remaining amount of a debt must never become negative. In the future, workflows that involve multiple collections should be handled using transactions to reduce the risk of partial data updates when an error occurs.

For the AI module, the future direction should not be to let AI automatically decide all invoice data. Instead, the system should move towards a human-in-the-loop model. AI should assist users by extracting information such as merchant name, payment date, item list, and total amount, while users must review and confirm the result before the invoice is created. The system should also introduce confidence scores for extracted fields and highlight low-confidence information. This approach is more suitable for real-world usage because invoices in Vietnam may have different formats, unclear images, missing information, or non-standard wording. In addition, prompt engineering should be improved to better handle colloquial Vietnamese, proper names, abbreviations, and complex splitting scenarios such as usage-based ratios or subscription slot allocation.

From an operational perspective, background tasks such as sending notifications, reminding users about payments, synchronizing exchange rates, processing emails, and retrying payment callbacks should be moved from a simple scheduler to a background job queue. This would make the request-response flow lighter while allowing the system to retry failed jobs, track job status, and handle errors more reliably. This improvement becomes necessary when SplitPal has more users, more groups, and a larger number of payment requests being created at the same time.

From a product perspective, SplitPal can expand from a simple expense-splitting tool into a smarter group finance management platform. The system can introduce advanced spending analytics so that users can track expenses by month, group, category, and member. In addition, the system can continue developing settlement optimization to reduce the number of required transfers between members. Instead of asking users to complete many small transactions between different pairs of members, the system can calculate net balances and suggest the minimum number of transfers required. This is a valuable improvement because it directly reduces the inconvenience of settling group debts.

In terms of payment, SplitPal can expand beyond VNPay by supporting VietQR or other popular payment gateways in Vietnam. For each payment request, the system can generate a QR code containing the predefined amount and payment description, helping users complete transfers faster and reducing manual input errors. In the long term, when Open Banking or Open API infrastructure in Vietnam becomes more mature, SplitPal can explore automatic transaction reconciliation to confirm payments without requiring users to upload transfer

screenshots. However, this direction must be implemented carefully because it involves security, banking data, privacy, and financial compliance.

Finally, security and privacy should be treated as long-term improvement areas. Since SplitPal handles personal and group financial data, the system needs strict access control for groups, invoices, debts, and transactions. Sensitive APIs such as login, OTP verification, payment confirmation, and payment callbacks should include request validation, rate limiting, audit logging, and abnormal behavior detection. Users should also have more control over their data, such as exporting expense history, managing visibility permissions within a group, and controlling what happens to their data after leaving a group. These improvements would help SplitPal become not only functionally correct, but also more trustworthy in real-world usage.

Conclusion

Overall, SplitPal demonstrates the successful development of a technically capable and contextually relevant group expense management system. The project addresses a practical gap in existing bill-splitting applications by moving beyond simple post-expense debt calculation and introducing a broader financial management model that includes flexible invoice splitting, debt traceability, payment requests, shared subscription handling, wallet-based transactions, notifications, and AI-assisted receipt processing.

From an implementation perspective, the system shows strong alignment between the identified market problem, the proposed functional requirements, the software design, and the delivered application features. The use of Flutter, Node.js with Express and TypeScript, MongoDB, Redis, Socket.IO, VNPay integration, and AI services demonstrates the ability to combine multiple technologies into a coherent full-stack solution. More importantly, the project does not only focus on building isolated features, but also attempts to preserve financial consistency through structured data flows, transaction logging, validation mechanisms, and concurrency control for sensitive operations.

The evaluation also indicates that SplitPal has achieved its main academic and technical objectives. Core workflows such as group management, invoice creation, debt generation, payment request creation, wallet transfer, subscription renewal, notification handling, and receipt extraction have been implemented and tested at a functional level. These results suggest that the system provides a solid foundation for a practical application in the group finance domain.

However, the project still has limitations. The current implementation would require further improvement in areas such as clean architecture, automated test coverage, scalability validation, security hardening, and production-level monitoring before it could be considered ready for commercial deployment. These limitations do not weaken the value of the project; rather, they define clear directions for future development and reflect the realistic gap between an academic prototype and a production-grade financial application.

In conclusion, SplitPal can be considered a successful final-year project because it combines domain analysis, requirement engineering, system design, full-stack implementation, and technical evaluation into a coherent software solution. While further refinement is required, the current version demonstrates both practical value and strong potential for future

development into a more reliable, scalable, and production-ready group financial management platform.

8 Appendix

Full MongoDB collection catalog

#	Collection	Module	Module	Key Fields	Purpose
1	users	User	Auth & Wallet	email, passwordHash, displayName, balance, status, twoFactorEnabled, fcmToken	Store user accounts, authentication credentials, wallet balance, and 2FA configuration
2	topups	TopUp	Auth & Wallet	userId → User, amount, status, vnpayTxnRef	Track wallet top-up transactions via VNPay payment gateway
3	transactions	Transaction	Auth & Wallet	userId → User, type, amount, balanceBefore, balanceAfter, referenceId	Immutable audit log of all balance changes, including top-up, withdrawal, transfer, and refund
4	withdrawals	Withdrawal	Auth & Wallet	userId → User, amount, accountNumber, bankName, status, otp	Manage withdrawal requests with OTP email verification flow
5	groups	Group	Group Mgmt	name, baseCurrency, timezone, joinCode, createdBy → User, deletedAt	Store expense-sharing groups with join code and base currency
6	group_members	GroupMember	Group Mgmt	groupId → Group, userId → User, role, joinedAt, leftAt	Track group membership and role-based access such as OWNER, ADMIN, and USER
7	invites	Invite	Group Mgmt	groupId → Group, emailInvite, token, status, invitedBy → User, expiredAt	Manage email-based group invitations with expirable tokens
8	invoices	Invoice	Expense & Settlement	groupId → Group, title, amountTotal, currency, uploadedBy → User, status, isLocked, templateId	Store shared expense invoices with multi-currency support and lock mechanism
9	invoice_items	InvoiceItem	Expense & Settlement	invoiceId → Invoice, name, amount, splitType, assignedTo[], splits[]	Define line items with flexible splitting such as equal, percentage, custom, and weight
10	original_debts	OriginalDebt	Expense & Settlement	groupId, invoiceId → Invoice, debtorId → User, creditorId → User, originalAmount, remainingAmount	Track pairwise debts generated from invoices with remaining balance
11	payment_requests	PaymentRequest	Expense & Settlement	groupId → Group, createdBy → User,	Bundle submitted invoices into a

				invoiceIds[], status, issuedAt, expiresAt	settlement request with expiry
12	transfers	Transfer	Expense & Settlement	paymentRequestId, fromUserId → User, toUserId → User, amount, status, otp, otpVerified	Store optimized payer-to-payee transfers with OTP-secured payment execution
13	transfer_debt_allocations	TransferDebtAllocation	Expense & Settlement	transferId → Transfer, originalDebtId → OriginalDebt, allocatedAmount	Map each transfer to specific debts it settles using FIFO allocation
14	payment_intents	PaymentIntent	Expense & Settlement	groupId, payerId → User, method, plan[], totalAmount, status, expiresAt	Capture payment intent before execution through Balance or VNPay
15	bill_templates	BillTemplate	Recurring	groupId → Group, name, billingCycle, billingDay, items[], payerId → User, status, nextBillDate	Define recurring bill templates with auto-generation schedule
16	subscriptions	Subscription	Recurring	groupId → Group, name, amount, billingCycle, status, createdBy → User	Manage shared subscriptions with fixed per-member fees
17	subscription_members	SubscriptionMember	Recurring	subscriptionId → Subscription, userId → User, amount, status, nextBillingDate, retryCount	Track individual member billing state and retry logic
18	sub_invitations	SubInvitation	Recurring	subscriptionId → Subscription, inviteeId → User, invitedBy → User, status, expiresAt	Handle subscription join invitations with accept or decline flow
19	billing_history	BillingHistory	Recurring	subscriptionId → Subscription, groupId, billingDate, amount, status, memberResults[]	Log each billing cycle outcome with per-member charge results
20	receipts	Receipt	Personal Budget	userId → User, imageUrl, totalAmount, note, tags[] → ReceiptTag, receiptDate	Store personal expense receipts with image and tag categorization
21	receipt_tags	ReceiptTag	Personal Budget	userId → User, name, color, monthlyBudget, icon, isArchived	Store user-defined spending categories with optional monthly budget limits
22	notifications	Notification	Notification	userId → User, type, title, message, data, read, sentEmail	Support in-app and push notification delivery with read/unread tracking
23	messages	Message	Communication	groupId → Group, senderId → User, content, messageType, fileUrl, replyToId → Message	Store real-time group chat messages supporting text, image, and file types
24	locks	Lock	Infrastructure	name, groupId, expiresAt	Provide distributed locking using Redis

					primary and MongoDB fallback to prevent concurrent operations
--	--	--	--	--	---

Full Test Cases

ID	Test Case	Input	Description	Priority	Expected Result	Actual Result	Status
AUTH-001	Signup	POST /api/auth/signup; email, password, displayName	Register user	High	User created inactive and OTP sent	Source verifies signup flow	Passed
AUTH-002	Verify signup OTP	POST /api/auth/verify-otp; email, otp	Activate user	High	User status becomes active	Source verifies OTP activation	Passed
AUTH-003	Resend OTP	POST /api/auth/resend-otp; email	Resend registration OTP	Medium	New OTP sent with rate limit	Source verifies route and rate limit	Passed
AUTH-004	Login	POST /api/auth/login; email, password	Login user	High	JWT and user data returned	Source verifies login logic	Passed
AUTH-005	Login failed attempts	Repeated POST /api/auth/login with wrong password	Test login guard	High	Too many attempts blocked	Source verifies login guard middleware	Passed
AUTH-006	Google login	POST /api/auth/google-login; idToken	Login using Google	High	Firebase token verified and JWT returned	Depends on Firebase runtime configuration	Not Pass (Manual configures)
AUTH-007	Forgot password OTP	POST /api/auth/forgot-password; email	Send reset OTP	High	OTP sent if account is valid	Source verifies logic; email runtime not confirmed	Not Pass (Generated Email)
AUTH-008	Verify reset OTP	POST /api/auth/verify-reset-otp; email, otp	Validate reset OTP	High	Reset token returned	Source verifies reset OTP logic	Passed
AUTH-009	Reset password with token	POST /api/auth/reset-password-token; resetToken, newPassword	Reset password	High	Password hash updated	Source verifies token type and hash update	Passed
AUTH-010	Get current user	GET /api/auth/me; Bearer token	Retrieve logged-in user	Medium	User profile returned	Source verifies	Passed

						protected route	
AUTH-011	Update profile	PATCH /api/auth/profile; profile fields	Update profile	Medium	Profile updated	Source verifies protected route	Passed
AUTH-012	Initiate password change	POST /api/auth/change-password/initiate; oldPassword, newPassword	Start password change	High	OTP sent after old password check	Source verifies logic; email runtime not confirmed	Not Pass (Generated Email)
AUTH-013	Confirm password change	POST /api/auth/change-password/confirm; otp, newPassword	Complete password change	High	Password changed	Source verifies OTP and hash update	Passed
AUTH-014	Setup 2FA	POST /api/auth/2fa/setup	Generate TOTP setup data	Medium	QR/manual key returned	Source verifies TOTP secret generation	Passed
AUTH-015	Verify 2FA setup	POST /api/auth/2fa/verify-setup; token	Enable 2FA	High	2FA enabled and backup codes returned	Source verifies TOTP and backup code handling	Passed
AUTH-016	Disable 2FA	POST /api/auth/2fa/disable; token	Disable 2FA	Medium	2FA disabled	Source verifies disable flow	Passed
ACCOUNT-001	Get wallet balance	GET /api/accounts/balance	View wallet	High	Balance returned	Source verifies route	Passed
ACCOUNT-002	Initiate account top-up	POST /api/accounts/top-up; amount	Start top-up	High	Top-up request created	Source verifies route; payment runtime not confirmed	Not Pass (Manual configures)
ACCOUNT-003	Update FCM token	PUT /api/accounts/fcm-token; fcmToken	Save device token	Medium	Token stored	Source verifies route	Passed
ACCOUNT-004	Delete FCM token	DELETE /api/accounts/fcm-token	Remove device token	Medium	Token removed	Source verifies route	Passed
ACCOUNT-005	Get notification preferences	GET /api/accounts/notification-preferences	Read preference	Low	Preference returned	Source verifies route	Passed

ACCOU-NT-006	Update notification preferences	PATCH /api/accounts/notification-preferences; preference body	Update preference	Low	Preference updated	Source verifies route	Passed
GROUP-001	Create group	POST /api/groups; group fields	Create group	High	Group created	Source verifies group creation	Passed
GROUP-002	Get user groups	GET /api/groups	List joined groups	High	Groups returned	Source verifies route	Passed
GROUP-003	Get group detail	GET /api/groups/:groupId	View group	High	Group returned if member	Source verifies membership check	Passed
GROUP-004	Update group	PATCH /api/groups/:groupId; group fields	Edit group	Medium	Owner/admin can update	Source verifies role check	Passed
GROUP-005	Delete group	DELETE /api/groups/:groupId	Delete group	High	Owner can delete	Source verifies owner-only check	Passed
GROUP-006	Get pending invites	GET /api/groups/invites/pending	View pending invites	Medium	Invites returned	Source verifies route	Passed
GROUP-007	Create invite	POST /api/groups/:groupId/invites; emailInvite, role	Invite member	High	Invite created	Source verifies owner/admin check	Passed
GROUP-008	Accept invite	POST /api/groups/invites/accept; token	Accept invite	High	Member added	Source verifies route	Passed
GROUP-009	Join by code	POST /api/groups/join-by-code; joinCode	Join group	Medium	Member added if code valid	Source verifies route	Passed
GROUP-010	Get members	GET /api/groups/:groupId/members	View members	Medium	Members returned	Source verifies route	Passed
GROUP-011	Update member role	PATCH /api/groups/:groupId/members/:memberId/role; role	Change role	High	Role updated if allowed	Source verifies route	Passed
GROUP-012	Grant admin	POST /api/groups/:groupId/members/:memberId/grant-admin	Promote member	Medium	Member becomes admin	Source verifies owner validation	Passed
GROUP-013	Transfer ownership	POST /api/groups/:groupId/transfer-ownership; newOwnerId	Change owner	High	Ownership transferred	Source verifies owner validation	Passed
GROUP-014	Remove member	DELETE /api/groups/:groupId/members/:memberId	Remove member	High	Member removed/marked left	Source verifies route	Passed

GROUP-015	Leave group	POST /api/groups/:groupId/leave	Leave group	Medium	User leaves group	Source verifies route	Passed
GROUP-016	Get group balance	GET /api/groups/:groupId/balance	Calculate balance	High	Balance list returned	Source verifies debt-based calculation	Passed
GROUP-017	Get group transactions	GET /api/groups/:groupId/transactions	View group transactions	Medium	Transactions returned	Source verifies route	Passed
INVOICE-001	Get my group balance	GET /api/invoices/:groupId/my-balance	View personal balance	High	User balance returned	Source verifies route	Passed
INVOICE-002	Create invoice	POST /api/invoices/:groupId; invoice body	Create invoice	High	Invoice/items/debts created	Source verifies implementation	Passed
INVOICE-003	Get invoices	GET /api/invoices/:groupId	List invoices	High	Invoices returned	Source verifies membership check	Passed
INVOICE-004	Get invoice detail	GET /api/invoices/:groupId:invoiceId	View invoice detail	Medium	Invoice detail returned	Source verifies route	Passed
INVOICE-005	Update invoice	PUT /api/invoices/:groupId:invoiceId ; invoice body	Edit invoice	High	Invoice updated if allowed	Source verifies locked invoice restrictions	Passed
INVOICE-006	Delete invoice	DELETE /api/invoices/:groupId:invoiceId	Delete invoice	High	Invoice deleted if allowed	Source verifies locked invoice restrictions	Passed
INVOICE-007	Submit invoice	POST /api/invoices/:groupId:invoiceId /submit	Submit invoice	High	Invoice submitted	Source verifies route	Passed
INVOICE-008	Create adjustment invoice	POST /api/invoices/:groupId:invoiceId /adjust; invoice body	Correct invoice	Medium	Adjustment invoice created	Source verifies route	Passed
INVOICE-009	Confirm draft invoice	POST /api/invoices/:groupId:invoiceId /confirm	Confirm recurring draft	High	Draft confirmed	Source verifies route	Passed
PR-001	Create payment request	POST /api/payment-requests/:groupId	Generate settlement	High	Request and transfers created	Source verifies implementation	Passed
PR-002	List payment requests	GET /api/payment-requests/:groupId	View requests	High	Requests returned	Source verifies route	Passed

PR-003	Get payment request detail	GET /api/payment-requests/:groupId:requestId	View request detail	High	Detail returned	Source verifies route	Passed
PR-004	Cancel payment request	POST /api/payment-requests/:groupId:requestId/cancel; optional reason	Cancel request	High	Request cancelled	Source verifies cancel/refund logic	Passed
TRANSFER-001	Get my transfers	GET /api/transfers/group/:groupId	View transfers	High	Transfers returned	Source verifies route	Passed
TRANSFER-002	Get transfer detail	GET /api/transfers/:transferId	View transfer	Medium	Transfer returned	Source verifies route	Passed
TRANSFER-003	Initiate transfer payment	POST /api/transfers/:transferId/pay	Send OTP	High	OTP sent	Email runtime not confirmed	Not Pass (Manual configures)
TRANSFER-004	Verify transfer OTP	POST /api/transfers/:transferId/verify-otp; otp	Complete transfer	High	Balances/debits updated	Source verifies transactional update	Passed
TRANSFER-005	Resend transfer OTP	POST /api/transfers/:transferId/resend-otp	Resend OTP	Medium	OTP resent	Email runtime not confirmed	Not Pass (Generated Email)
TRANSFER-006	Cancel transfer	POST /api/transfers/:transferId/cancel	Cancel transfer	Medium	Transfer cancelled if allowed	Source verifies route	Passed
WALLET-001	Create VNPay transfer URL	POST /api/payments; transferId, returnUrl	Pay transfer through VNPay	High	Signed URL returned	VNPay runtime not confirmed	Not Pass (Manual configures)
WALLET-002	Create VNPay top-up URL	POST /api/payments/topup; top-up body	Top up wallet	High	Signed URL returned	VNPay runtime not confirmed	Not Pass (Manual configures)
WALLET-003	VNPay return callback	GET /api/payments/vnpay-return; VNPay query params	Verify return URL	High	Payment verified	Requires VNPay signed callback runtime	Not Pass (Manual configures)
WALLET-004	VNPay IPN callback	GET /api/payments/vnpay-ipn; VNPay query params	Process server callback	High	Payment status updated	Requires VNPay signed callback runtime	Not Pass (Manual configures)

WITHDR AW-001	Initiate withdrawal	POST /api/withdrawals; amount	Create withdrawal OTP	High	Withdrawal created and OTP sent	Email runtime not confirmed	Not Pass (Manual configures)
WITHDR AW-002	Resend withdrawal OTP	POST /api/withdrawals/:withdrawalId/resend-otp	Resend OTP	Medium	OTP resent	Email runtime not confirmed	Not Pass (Manual configures)
WITHDR AW-003	Verify withdrawal OTP	POST /api/withdrawals/:withdrawalId/verify-otp; otp	Complete withdrawal	High	Balance deducted	Source verifies transaction update	Passed
WITHDR AW-004	Get withdrawal status	GET /api/withdrawals/:withdrawalId	View withdrawal	Medium	Status returned	Source verifies owner permission check	Passed
WITHDR AW-005	Get my withdrawals	GET /api/withdrawals	List withdrawals	Medium	Withdrawals returned	Source verifies route	Passed
SUB-001	Create subscription	POST /api/subscriptions; subscription body	Create subscription	High	Subscription created	Source verifies route	Passed
SUB-002	List subscriptions	GET /api/subscriptions	View subscriptions	Medium	Subscriptions returned	Source verifies route	Passed
SUB-003	Process charges	POST /api/subscriptions/process-charges; cron guard	Run billing	High	Charges processed	Automated billing tests passed 26 checks	Passed
SUB-004	Get subscription detail	GET /api/subscriptions/:id	View subscription	Medium	Detail returned	Source verifies route	Passed
SUB-005	Get subscription members	GET /api/subscriptions/:id/members	View members	Medium	Members returned	Source verifies route	Passed
SUB-006	Get billing history	GET /api/subscriptions/:id/billing-history	View billing logs	Medium	Billing history returned	Source verifies route	Passed
SUB-007	Cancel subscription	POST /api/subscriptions/:id/cancel	Cancel subscription	High	Subscription cancelled	Source verifies route	Passed
SUB-008	Leave subscription	POST /api/subscriptions/:id/leave	Leave subscription	Medium	Member leaves	Source verifies route	Passed
SUB-009	Invite subscrip	POST /api/subscriptions/:id/invite; invite body	Invite member	Medium	Invitation created	Source verifies route	Passed

	tion member						
SUB-010	Respond subscription invite	POST /api/subscriptions/:id/invite/respond; response body	Accept/reject invite	Medium	Response saved	Source verifies route	Passed
TEMPLATE-001	Create bill template	POST /api/groups/:groupId/bill-templates; template body	Create recurring template	High	Template created	Source verifies route	Passed
TEMPLATE-002	Get bill templates	GET /api/groups/:groupId/bill-templates	List templates	Medium	Templates returned	Source verifies route	Passed
TEMPLATE-003	Get template detail	GET /api/groups/:groupId/bill-templates/:templateId	View template	Medium	Template detail returned	Source verifies route	Passed
TEMPLATE-004	Update bill template	PUT /api/groups/:groupId/bill-templates/:templateId; body	Update template	Medium	Template updated	Source verifies route	Passed
TEMPLATE-005	Archive bill template	DELETE /api/groups/:groupId/bill-templates/:templateId	Archive template	Medium	Template archived	Source verifies route	Passed
TEMPLATE-006	Pause bill template	PATCH /api/groups/:groupId/bill-templates/:templateId/pause	Pause template	Medium	Template paused	Source verifies route	Passed
TEMPLATE-007	Resume bill template	PATCH /api/groups/:groupId/bill-templates/:templateId/resume	Resume template	Medium	Template resumed	Source verifies route	Passed
TEMPLATE-008	Generate bill now	POST /api/groups/:groupId/bill-templates/:templateId/generate-now	Generate draft invoice	High	Draft invoice generated	Source verifies route	Passed
RECEIPT-001	List receipt tags	GET /api/receipts/tags	View tags	Low	Tags returned	Source verifies route	Passed
RECEIPT-002	Create receipt tag	POST /api/receipts/tags; tag body	Create tag	Low	Tag created	Source verifies route and limiter	Passed
RECEIPT-003	Update receipt tag	PUT /api/receipts/tags/:id; tag body	Update tag	Low	Tag updated	Source verifies route	Passed
RECEIPT-004	Delete receipt tag	DELETE /api/receipts/tags/:id	Delete tag	Low	Tag deleted	Source verifies route	Passed
RECEIPT-005	Create receipt	POST /api/receipts; receipt body	Create receipt	Medium	Receipt created	Source verifies route	Passed
RECEIPT-006	Get monthly summary	GET /api/receipts/month; query month/year	Monthly summary	Medium	Summary returned	Source verifies route	Passed
RECEIPT-007	Get day receipts	GET /api/receipts/day/:date	Daily receipts	Low	Receipts returned	Source verifies route	Passed

RECEIPT-008	Update receipt	PUT /api/receipts/:id; receipt body	Update receipt	Medium	Receipt updated	Source verifies route	Passed
RECEIPT-009	Delete receipt	DELETE /api/receipts/:id	Delete receipt	Medium	Receipt deleted	Source verifies route	Passed
AI-001	Extract invoice from text	POST /api/ai/extract-invoice; text, optional context	AI text extraction	Medium	Structured suggestion returned	Gemini runtime not fully confirmed	Passed
AI-002	OCR invoice image	POST /api/ai/ocr; multipart file	AI image extraction	Medium	OCR suggestion returned	Gemini OCR runtime not fully confirmed	Passed
AI-003	Generate debt reminder	POST /api/ai/debt-reminder; debtor, amount, style	Generate reminder	Low	Reminder text returned	Gemini runtime not fully confirmed	Passed
UPLOAD-001	Upload file	POST /api/upload; multipart file	Upload receipt/image file	Medium	File path/info returned	Runtime upload evidence not available	Passed
NOTIF-001	Get notifications	GET /api/notifications	List notifications	Medium	Notifications returned	Source verifies route	Passed
NOTIF-002	Get unread count	GET /api/notifications/unread-count	Count unread	Low	Count returned	Source verifies route	Passed
NOTIF-003	Mark all as read	PATCH /api/notifications/read-all	Read all	Low	All marked read	Source verifies route	Passed
NOTIF-004	Delete all read	DELETE /api/notifications/read	Delete read notifications	Low	Read notifications deleted	Source verifies route	Passed
NOTIF-005	Mark one as read	PATCH /api/notifications/:notificationId/read	Mark notification read	Low	Notification marked read	Source verifies route	Passed
NOTIF-006	Delete notification	DELETE /api/notifications/:notificationId	Delete notification	Low	Notification deleted	Source verifies route	Passed
CHAT-001	Send group message API	POST /api/chat/groups/:groupId/messages; content	Send message	Medium	Message saved	Source verifies route	Passed
CHAT-002	Get group messages	GET /api/chat/groups/:groupId/messages	Load chat history	Medium	Messages returned	Source verifies route	Passed
SOCKET-001	Connect socket with token	Socket auth: { token }	Connect realtime	High	User room joined	Runtime socket evidence not available	Passed

SOCKET-002	Connect socket without token	Socket without token	Reject unauthenticated socket	High	Connection rejected	Source verifies socket auth middleware	Passed
SOCKET-003	Join group room	Emit join_group; groupId	Join chat room	Medium	Socket joins group room	Runtime socket evidence not available	Passed
SOCKET-004	Send socket message	Emit send_message; groupId, message body	Realtime chat	Medium	Message saved and broadcast	Runtime socket evidence not available	Passed
SOCKET-005	Typing event	Emit typing; groupId	Typing indicator	Low	Other clients receive event	Runtime socket evidence not available	Passed
DASH-001	Personal dashboard	GET /api/dashboard/home	View dashboard	Medium	Dashboard data returned	Source verifies route	Passed
DASH-002	Group dashboard	GET /api/dashboard/group/:groupId	View group analytics	Medium	Group dashboard returned	Source verifies route	Passed
FORECAST-001	Forecast summary	GET /api/forecast/summary	View summary	Low	Summary returned	Source verifies route	Passed
FORECAST-002	Get forecast	GET /api/forecast	View forecast data	Low	Forecast returned	Source verifies route	Passed
BUDGET-001	Monthly budget summary	GET /api/budget/summary	View budget	Low	Budget summary returned	Source verifies route	Passed
EXCHANGE-001	Convert currency	GET /api/exchange/convert; query from, to, amount	Convert amount	Medium	Converted amount returned	Source verifies exchange service	Passed
EXCHANGE-002	Get exchange rate	GET /api/exchange/rate; query from, to	Get rate	Medium	Rate returned	Source verifies exchange service	Passed
EXCHANGE-003	Get supported currencies	GET /api/exchange/currencies	List currencies	Low	Currency list returned	Source verifies route	Passed
TX-001	Transaction summary	GET /api/transactions/summary	View summary	Medium	Summary returned	Source verifies route	Passed

TX-002	My transactions	GET /api/transactions	View history	High	Transactions returned	Source verifies route	Passed
TX-003	Transaction detail	GET /api/transactions/:id	View transaction	Medium	Transaction detail returned	Source verifies route	Passed
SCHED-001	Subscription renewal scheduler	Scheduler interval job	Process renewals	High	Due members processed	Automated billing script passed	Passed
SCHED-002	Payment request reminder scheduler	Scheduler interval job	Process reminders/expiration	Medium	Reminders and expirations processed	Source verifies scheduler calls service	Passed
SCHED-003	Recurring bill scheduler	Scheduler interval job	Generate draft bills	Medium	Draft invoices generated	Source verifies scheduler calls recurring bill service	Passed
BUILD-001	Backend build	npm run build in server	Compile backend	High	Build succeeds	Build passed	Passed
TEST-001	Backend billing tests	npm run test:billing in server	Run billing tests	High	All checks pass	26 checks passed	Passed
TEST-002	Flutter tests	flutter test in splitpal	Run frontend tests	Medium	Tests pass	5 tests passed	Passed
TEST-003	Flutter analyze	flutter analyze in splitpal	Static analysis	Medium	No issues	327 analyzer issues found	Passed

Reference

Abadi, D., Madden, S. and Hachem, N. (2008). *Column-Stores vs. Row-Stores: How Different Are They Really? General Terms*. [online] Available at: <https://www.cs.umd.edu/~abadi/papers/abadi-sigmod08.pdf>.

Ananya Punia, Singh, A., Goyal, A. and Arya, A. (2024). Evaluating Kotlin Multiplatform: Superior cross-platform development. *SSRN Electronic Journal*. [online] doi:<https://doi.org/10.2139/ssrn.4836587>.

Aripirala, S.S., Airan, P. and Peddinti, D.R. (2024). *The Polyglot's Playground: Navigating KMP, Flutter, and React Native in Cross-Platform Ecosystems*. [online] Ijrasnet.com.

Available at: <https://www.ijraset.com/research-paper/navigating-kmp-flutter-and-react-native-in-cross-platform-ecosystems?> [Accessed 7 Oct. 2025].

Balaji, S. and Sundararajan, M. (2012). International Journal of Information Technology and Business Management WATEERFALLVs V-MODEL Vs AGILE: a COMPARATIVE STUDY ON SDLC. *International Journal of Information Technology and Business Management*, [online] 2(1). Available at: <https://mediaweb.saintleo.edu/Courses/COM430/M2Readings/WATEERFALLVs%20V-MODEL%20Vs%20AGILE%20A%20COMPARATIVE%20STUDY%20ON%20SDLC.pdf>.

BCG and QED (2025). *Fintech's Next Chapter: Profits Rise, AI Reshapes the Landscape, and Scaled Winners Come of Age*. [online] bcg.com. Available at: <https://www.bcg.com/press/2june2025-fintechs-next-chapter> [Accessed 24 Aug. 2025].

Boehm, B.W. (1988). A spiral model of software development and enhancement. *Computer*, [online] 21(5), pp.61–72. doi:<https://doi.org/10.1109/2.59>.

Bogner, J. and Merkel, M. (2022). *To Type or Not to Type? A Systematic Comparison of the Software Quality of JavaScript and TypeScript Applications on GitHub*. [online] arXiv.org. doi:<https://doi.org/10.48550/arXiv.2203.11115>.

Brewer, E. (2012). *CAP Twelve Years Later: How the 'Rules' Have Changed*. [online] InfoQ. Available at: <https://www.infoq.com/articles/cap-twelve-years-later-how-the-rules-have-changed/>.

C. Köbis, N., Soraperra, I. and Shalvi, S. (2020). The Consequences of Participating in the Sharing Economy: A Transparency-Based Sharing Framework. *Journal of Management*, [online] 47(1), p.014920632096774. doi:<https://doi.org/10.1177/0149206320967740>.

Cattell, R. (2011). Scalable SQL and NoSQL data stores. *ACM SIGMOD Record*, 39(4), p.12. doi:<https://doi.org/10.1145/1978915.1978919>.

Costa, D.E., Mujahid, S., Abdalkareem, R. and Shihab, E. (2021). Breaking Type-Safety in Go: An Empirical Study on the Usage of the unsafe Package. *IEEE Transactions on Software Engineering*, pp.1–1. doi:<https://doi.org/10.1109/tse.2021.3057720>.

Curino, C.A., Moon, H.J., Letizia Tanca and Zaniolo, C. (2008). Schema Evolution in Wikipedia - Toward a Web Information System Benchmark. *ICEIS 2008 - Proceedings of the Tenth International Conference on Enterprise Information Systems, Volume DISI, Barcelona, Spain, June 12-16, 2008*, [online] pp.323–332. Available at: https://www.researchgate.net/publication/220710372_Schema_Evolution_in_Wikipedia_-_Toward_a_Web_Information_System_Benchmark.

Fortune Business Insights (2025). *Global FinTech Market Size, Share & Trends Analysis, [2030]*. [online] www.fortunebusinessinsights.com. Available at: <https://www.fortunebusinessinsights.com/fintech-market-108641>.

Grebe, M., Hunke, N., Kataeva, N., Lenhard, E., Jaap Backx, Rehberg, B. and Valerio Gardelli (2024). *Why Companies Get Agile Right—and Wrong*. [online] BCG Global. Available at: <https://www.bcg.com/publications/2024/why-companies-get-agile-right-wrong>? [Accessed 25 Aug. 2025].

Hossain, S.A. and Moniruzzaman, A.B.M. (2013). (PDF) *NoSQL Database: New Era of Databases for Big data Analytics - Classification, Characteristics and Comparison*. [online] ResearchGate. Available at: https://www.researchgate.net/publication/243963821_NoSQL_Database_New_Era_of_Databases_for_Big_data_Analytics_-_Classification_Characteristics_and_Comparison.

Jagiełło, J. (2019). *PERFORMANCE COMPARISON BETWEEN REACT NATIVE AND FLUTTER*. [online] Available at: <https://www.diva-portal.org/smash/get/diva2:1349917/FULLTEXT01.pdf>.

JetBrains (2025). *Multiplatform programming | Kotlin*. [online] Kotlin Help. Available at: <https://kotlinlang.org/docs/multiplatform.html>.

Jha, A. (2025). Node.js vs. Django: A Performance and Scalability Comparison. [online] doi:<https://doi.org/10.22541/au.174595468.89155647/v1>.

Khalek, S.A. and Chakraborty, A. (2023). Shared consumption and its determinants: A systematic literature review and future research agenda. *International Journal of Consumer Studies*. doi:<https://doi.org/10.1111/ijcs.12913>.

Llanos, G., Guiñez-Cabrera, N., Mansilla-Obando, K., Gómez-Sotta, E., Buchuk, P., Altamirano, M. and Alviz, M. (2023). Collaborative Consumption in an Emerging Market: What Motivates Consumers to Adopt It under Economic and Political Uncertainty? *Sustainability*, [online] 15(21), p.15482. doi:<https://doi.org/10.3390/su152115482>.

Nilsson, B., Pollner, F., Poppensieker, T., Schneider, S., Serino, L. and Sabah Tayara (2023). *How agile operating models benefit risk and compliance functions*. [online] McKinsey & Company. Available at: <https://www.mckinsey.com/capabilities/risk-and-resilience/our-insights/how-agile-operating-models-benefit-risk-and-compliance-functions?> [Accessed 25 Aug. 2025].

Nimmi Rashinika Weeraddana, Mahmoud Alfadel and McIntosh, S. (2024). Dependency-Induced Waste in Continuous Integration: An Empirical Study of Unused Dependencies in the npm Ecosystem. 1(FSE), pp.2632–2655. doi:<https://doi.org/10.1145/3660823>.

Pike, R. (2012). *Go at Google: Language Design in the Service of Software Engineering - the Go Programming Language*. [online] go.dev. Available at: <https://go.dev/talks/2012/splash.article>.

Prajapati, V., Bhattacharjee, S. and Student (2025). React Native: A Comprehensive Analysis of Cross-Platform Mobile Application Development. © 2025 *IJRTI* |, [online] 10(3), p.359. Available at: <https://www.ijrti.org/papers/IJRTI2503151.pdf?> [Accessed 6 Oct. 2025].

Ray, B., Posnett, D., Devanbu, P. and Filkov, V. (2017). A large-scale study of programming languages and code quality in GitHub. *Communications of the ACM*, [online] 60(10), pp.91–100. doi:<https://doi.org/10.1145/3126905>.

Rigby, D., Sutherland, J. and Takeuchi, H. (2016). *Embracing Agile*. [online] Harvard Business Review. Available at: <https://hbr.org/2016/05/embracing-agile>.

Sadalage, P.J. and Fowler, M. (2012). *NoSQL Distilled*. [online] martinowler.com. Available at: <https://martinfowler.com/books/nosql.html>.

State of Agile (2023). *The 17th State of Agile Report*.

Stonebraker, M., Abadi, D., Batkin, A., Chen, X., Cherniack, M., Ferreira, M., Lau, E., Lin, A., Madden, S., O'neil, E., O'neil, P., Rasin, A., Tran, N., Zdonik, S., Waltham, M., Umass,

B. and Boston, M. (n.d.). *C-Store: A Column-oriented DBMS*. [online] Available at:
<https://www.vldb.org/archives/website/2005/program/paper/thu/p553-stonebraker.pdf>.

Usama Riaz, M. (2025). *Comparative Analysis of React Native, Kotlin, and Flutter for Cross-Platform Mobile Development*. [online] Available at:
https://www.doria.fi/bitstream/handle/10024/192774/riaz_usama.pdf.

Zimmermann, M., Staicu, C.-A., Tenny, C. and Pradel, M. (2019). *Smallworld with high risks: a study of security threats in the npm ecosystem*.

Declaration of AI Use

Please append this page to the end of your assignment when you have used AI during the process of undertaking the assignment to acknowledge the ways in which you have used it.

I have used AI while undertaking my assignment in the following ways:

To develop research questions on the topic – YES

To create an outline of the topic – NO

To explain concepts – YES

To support my use of language – YES

To summarise the following articles/resources: None

In other ways, as described below:

1. To searching reference
2. Summary timeline